

Code Completion unter eigener Kontrolle: Self-Hosted Open-Source-Alternativen für datensensible Unternehmen

Bachelorarbeit

vorgelegt am 05.05.2025

Fakultät Wirtschaft und Gesundheit

Studiengang Wirtschaftsinformatik

Kurs WWI2022G

von

FILIP HERCEG

Betreuung in der Ausbildungsstätte:

S-Management Services GmbH

Thilo Schaber

IT-Architekt mit fachlicher Führung

DHBW Stuttgart:

Maximilian Ott

Unterschrift

Inhaltsverzeichnis

Abkürzungsverzeichnis	IV
Abbildungsverzeichnis	V
Tabellenverzeichnis	VI
1 Einleitung	1
1.1 Hintergrund	1
1.2 Ziel der Arbeit	2
1.3 Aufbau der Arbeit	3
1.4 Methodik	5
2 Marktentwicklung und technologische Trends von Chatbots und Code-Completion-Systemen	7
2.1 Die Evolution intelligenter Assistenzsysteme	7
2.2 Die Entwicklung von Code-Completion-Systemen im Kontext dialogorientierter KI	13
2.3 Marktanforderungen und Bedarfe von Unternehmen an moderne Code-Completion-Systeme	17
2.4 Technologische Trends im Kontext von Code Completion	18
3 Relevanz und Zielgruppen von Code-Completion-Systemen	21
3.1 Relevanz in der heutigen Softwareentwicklung	21
3.2 Potenziale für Unternehmen	21
3.3 Zielgruppen und Einsatzkontexte	22
4 Datenschutz, regulatorische und rechtliche Aspekte	24
4.1 Einleitung und Problemaufriss	24
4.2 Grundlagen des Datenschutzes im Kontext von Softwareentwicklung	24
4.3 Datenschutztechnische Risiken bei Code-Completion-Systemen	29
4.4 Interne vs. externe Server und deren rechtliche Bewertung	30
5 Methoden der Projektkonzeptionierung	32
5.1 Ziel und Bedeutung der Konzeptionierung	32
5.2 Methodisches Rahmenwerk: Design Science Research (DSR)	34
5.3 Phase 1: Problemanalyse und Zieldefinition	36
5.4 Phase 2: Artefaktentwicklung — Konzeption und Prototyping	36
5.5 Phase 3: Evaluation mit Fokus auf Nutzerfeedback	37
5.6 Fazit der Konzeptionierung und DSR-Anwendung	37
6 Analyseverfahren von Plattformen	39
6.1 Zielsetzung der Plattformanalyse	39
6.2 Bewertungskriterien für Plattformen	39
6.3 Kriterienbasierte Bewertungsverfahren	40
6.4 Feature-Mapping	40
7 Evaluationsverfahren für KI-Modelle	42
7.1 Zielsetzung der Evaluationsverfahren	42

7.2	Evaluationsdimensionen	42
7.3	Evaluationsmethoden für KI-Modelle	42
7.4	Kombination und Gewichtung der Ergebnisse	43
7.5	Fazit	44
8	Marktforschung und Unternehmensanforderungen	45
8.1	Ziel der Marktforschung	45
8.2	Vorstellung der analysierten Plattformen	45
8.3	Vorstellung der Marktsituation für Code-Completion-Modelle	48
8.4	Unternehmensanforderungen und -prioritäten	50
9	Analyse der aktuellen Plattform Refact.ai	52
9.1	Schwächenanalyse der bestehenden Plattform Refact.ai	54
10	Analyse alternativer Plattformen	56
10.1	Präemptiver Ausschluss ungeeigneter Plattformen	56
10.2	Verbesserungspotenziale durch Continue.dev (On-Premise)	57
11	Evaluation vielversprechender Modelle	59
11.1	Zielsetzung der Evaluation	59
11.2	Methodik: Prüfung der Modelle durch Code-Challenges	59
11.3	Methodik: Nutzerfeedback zum Praxiseinsatz	60
12	Ergebnisse und Entscheidung	61
12.1	Ergebnisse der Plattformanalyse	61
12.2	Durchführung und vorläufige Ergebnisse der Modellevaluation	61
12.3	Erkenntnisse und Ableitungen	63
13	Reflexion und Ausblick	65
13.1	Grenzen der Evaluation	65
13.2	Offene technische Perspektiven	65
13.3	Fazit und Ausblick	66
	Anhang	67
	Literaturverzeichnis	81

Abkürzungsverzeichnis

A.L.I.C.E Artificial Linguistic Internet Computer Entity

API Application Programming Interface

BDSG Bundesdatenschutzgesetz

CI/CD Continuous Integration/Continuous Delivery

DNN Deep Neural Network

DSGVO Datenschutz-Grundverordnung

DSFA Datenschutz-Folgenabschätzung

DSR Design Science Research

DX Developer Experience

GPT Generative Pre-trained Transformer

KI Künstliche Intelligenz

LD SG Landesdatenschutzgesetz

LLM Large Language Model

NLP Natural Language Processing

ML Machine Learning

RAG Retrieval-Augmented Generation

RNN Recurrent Neural Network

S-MS S-Management Services GmbH

SVM Support Vector Machine

IDE Integrated Development Environment

Abbildungsverzeichnis

1	Beispiel für einen Fehler bei Codestral (Bildschirmaufnahme).	62
2	Vergleich der Java Testergebnisse mehrerer Modelle (Bildschirmaufnahmen)	78
3	Vergleich der Python Testergebnisse mehrerer Modelle (Bildschirmaufnahmen) . . .	79
4	Vergleich der TypeScript Testergebnisse mehrerer Modelle (Bildschirmaufnahmen) .	80

Tabellenverzeichnis

1	Bewertungskriterien für Plattformen	40
2	Evaluationsdimensionen für die Bewertung von KI-Modellen zur Code Completion .	43
3	Vergleich von KI-gestützten Code Completion Tools: Teil 1	68
4	Vergleich von KI-gestützten Code Completion Tools: Teil 2	69
5	Vergleich von KI-gestützten Code Completion Tools: Teil 3	70
6	Vergleich von KI-gestützten Code Completion Tools: Teil 4	71
7	Vergleich von KI-gestützten Code Completion Tools: Teil 5	72
8	Vergleich von KI-gestützten Code Completion Tools: Teil 6	73
9	Beispiel: Speicherbedarf eines 2B-Parameter-Modells bei verschiedenen Präzisionsfor- maten	74
10	Maximale Anzahl der Parameter bei 24 GB VRAM für verschiedene Präzisionsformate	74
11	Feature-Matrix Refact.AI (Ist-Analyse)	75
12	Feature-Matrix Continue.dev (On-Premise-Nutzung)	76
13	Bewertung der Plattformen anhand definierter Ausschlusskriterien	77

1 Einleitung

In nur wenigen Jahren hat sich ein regelrechter Boom in der Entwicklung von Künstliche Intelligenz (KI)-Assistenten vollzogen. Darunter sind insbesondere Chatbots und Code-Completion-Systeme rasant vorangeschritten¹. Während Chatbots in der Regel für allgemeine Anwendungsfälle im Dialog entwickelt werden, sind Code-Completion-Systeme spezialisiert auf die Vervollständigung von Code-Snippets und somit der Unterstützung von Softwareentwicklern und Softwareentwicklerinnen bei ihrer täglichen Arbeit². Damit verändert sich die Art und Weise, wie Softwarecode entwickelt wird, nachhaltig. Die vorliegende Arbeit beschäftigt sich mit der Frage, wie ein vorhandenes Code-Completion-System an die spezifischen Bedürfnisse und Anforderungen eines Unternehmens mit hohen Datenschutzvorgaben angepasst werden kann. Dies soll bewerkstelligt werden, indem geprüft wird, inwiefern eine Anpassung der Lösung notwendig ist, um die Anforderungen zu erfüllen. Anschließend soll die Lösung nach ihrer Implementierung evaluiert werden, um die Qualität der Anpassung zu überprüfen und weitere Verbesserungsmöglichkeiten zu dokumentieren.

1.1 Hintergrund

Genau wie die Digitalisierung im KI-getriebenen Kontext die Gesellschaft bereits in vielen anderen Arbeitsbereichen verändert hat, etwa in der Medizin mit *AlphaFold*³ oder im Journalismus mit Chatbot-Assistenten wie *ChatGPT* oder *Gemini*⁴, so verändert sie auch die Softwareentwicklung. Während Chatbots heute schon in der Lage sind, komplexe Programmieraufgaben zu lösen⁵, so fehlt ihnen die direkte Integration in die Entwicklungsumgebung⁶. Der Anspruch im Softwareentwicklungsprozess wird mit fortlaufender Zeit immer komplexer, während gleichzeitig schneller gearbeitet werden muss. Folglich ist es im Interesse von Softwareentwicklern und Softwareentwicklerinnen, so viel Zeit und Arbeit wie möglich zu sparen. Hierbei unterstützen Code-Completion-Systeme. Sie sollen dabei helfen, die Qualität des Codes zu verbessern, die Effizienz zu steigern und die Wartbarkeit zu erhöhen⁷. Durch die erleichterte Arbeit befindet sich die Rolle von Entwicklern ebenfalls im Wandel. Es wird weniger getippt und mehr kuratiert⁸.

Allerdings haben Code-Completion-Systeme auch Eigenschaften, die sie für Unternehmen problematisch machen. Der Datenschutz ist hierbei ein zentrales Thema, da Quellcode als sensible Daten betrachtet wird und sich darin oftmals weitere sensible Informationen befinden. Es ist nicht ohne Weiteres möglich, diese Daten an Drittanbieter zu übermitteln, da diese die gesammelten Daten für ihre oder fremde Zwecke wie zum Beispiel dem Training ihrer Modelle verwenden

¹Vgl. Casheekar et al. 2024, S. 2 Z. 40 ff.

²Vgl. Contreras, Guerra, De Lara 2024, S. 2 f.

³Vgl. Guo et al. 2024, S. 1

⁴Vgl. Bettis 2024, S. 1f., 48–51

⁵Vgl. Sergeyuk, Titov, Izadi 2024, S. 97

⁶Vgl. Xu, Vasilescu, Neubig 2022, S. 26 f.

⁷Vgl. Solanke 2024, S. 247

⁸Vgl. Treude, Gerosa 2025, S. 2 f.

könnten⁹. Somit stellt dies ein Datenleck dar, das für Unternehmen nicht tragbar ist. Ein zweites Problem ist die Erklärbarkeit und Nachvollziehbarkeit der Systeme. Dies entspricht dem sogenannten Black-Box-Charakter vieler KI-Modelle¹⁰. Das bedeutet, dass nur das Eingangs- und Ausgangsverhalten der Modelle sichtbar ist, jedoch nicht nachvollzogen werden kann, wie genau das Modell zu seinem Ergebnis gekommen ist. Das gilt im besonderen Maße für Systeme, die von Drittanbietern bereitgestellt werden.

Daraus ergibt sich für Unternehmen das Interesse, die Kontrolle über das System und damit auch über die verarbeiteten Informationen zu maximieren.

Ein wichtiger Ansatz ist damit der Einsatz von Open-Source Plattformen und Modellen, sodass eine eigene und unabhängige Implementierung des Systems möglich ist¹¹. Dadurch entsteht die Möglichkeit, sowohl die Modelle, als auch die Plattform an die eigenen Anforderungen anzupassen und bestimmte Einstellungen zu konfigurieren.

Die S-Management Services GmbH (S-MS) stellt sich der Herausforderung für ihre internen Kunden ein Code-Completion-System zu entwickeln und hat dazu bereits die Plattform Refact.ai in Verwendung. Damit existiert eine Einbindung eines Plugins in JetBrains Integrated Development Environments (IDEs), das die nötigen Daten von der IDE abgreift und an das Backend weiterleitet. Dieses Backend kommuniziert dann mit dem KI-Modell, das die Vervollständigung vornimmt und das Ergebnis zurücksendet. Der Entwickler oder die Entwicklerin erhält das Ergebnis dann als Vorschlag, der angenommen oder abgelehnt werden kann. Das KI-Modell wird derzeit auf einem AWS-Server mit strikten Datenschutzvorgaben betrieben.

Diese Lösung erwies sich im Laufe der Zeit als unzureichend: Sie entspricht nicht den Anforderungen und ist daher nicht in der Lage, die gewünschten Ergebnisse zu liefern.

1.2 Ziel der Arbeit

Das Ziel der Arbeit ist es, ein methodisch fundiertes Konzept zur Evaluation und Verbesserung der aktuellen Code-Completion-Lösung zu entwickeln. Dabei sollen Ursachen für bestehende Defizite identifiziert und geeignete Alternativen oder Optimierungen erarbeitet werden. Die Ergebnisse münden in einen Prototyp, der abschließend evaluiert und im Hinblick auf zukünftige Entwicklungen reflektiert wird.

Konkret soll identifiziert werden, welche Faktoren zum Misserfolg der aktuellen Lösung geführt haben. Die identifizierten Schwachstellen werden dann genauer untersucht. So soll ermittelt werden, ob es sinnvolle Alternativen gibt oder ob Anpassungen zu verbesserter Qualität führen. Die gesammelten Erkenntnisse bilden dann die Grundlage für die Entwicklung eines Prototyps, der ebenfalls auf seine Eignung auf die Anforderungen des Unternehmens getestet wird. Zuletzt kann

⁹Vgl. Al-Kharusi et al. 2024, S. 2-12

¹⁰Vgl. Schaaf, Wiedenroth, Wagner 2021, S. 11-15

¹¹Vgl. Vake et al. 2025, S. 1 f.

so ein Ausblick gegeben werden, welche Schritte noch notwendig sind und welche Forschungsfragen für zukünftige Arbeiten relevant sein könnten.

1.3 Aufbau der Arbeit

In dieser Sektion wird die inhaltliche Struktur der Arbeit erläutert. Die Gliederung ist ausgerichtet am Ziel, sowohl die nötigen theoretischen Grundlagen zu schaffen, als auch eine praxisnahe Umsetzung des Projekts durchzuführen.

Da es sich bei der Thematik dieser Arbeit um ein umfangreiches und breit gefächertes Gebiet handelt, ist es notwendig, den Lesern und Leserinnen eine detaillierte theoretische Grundlage zu schaffen. Diese umfasst sowohl die Definition zentraler Konzepte als auch die Erläuterung der Arbeitsweise, nach der die Arbeit durchgeführt wird.

Es handelt sich hierbei um eine Arbeit, die parallel eine praktische Umsetzung beinhaltet. Folglich werden auch Konzeptionen der Wirtschaftsinformatik behandelt, die für die praktische Umsetzung von Bedeutung sind.

Um dieser Vielfalt gerecht zu werden, ist es notwendig, die Arbeit in zwei grundlegende Teile zu gliedern. Der erste Teil befasst sich mit den theoretischen Hintergründen und Voraussetzungen der Arbeit. Hierzu gehören die Kapitel von 2 (Marktentwicklung und technologische Trends von Chatbots und Code-Completion-Systemen) bis 7 (Evaluationsverfahren für KI-Modelle). Neben den konzeptionellen Grundlagen werden auch vergangene und aktuelle Entwicklungen im Bereich KI-gestützter Chatbots und Code-Completion-Systeme geschildert. Darunter fallen zudem die rechtlichen Aspekte und der Datenschutz, die für die Arbeit von Bedeutung sind. Darüber hinaus werden relevante Modelltypen vorgestellt sowie die Funktionsweise von Plattformen, ihren Limitationen und möglichen Evaluationsverfahren.

Der zweite Teil hingegen umfasst die Kapitel von 8 (Marktforschung und Unternehmensanforderungen) bis 13 (Reflexion und Ausblick) und befasst sich mit der praktischen Umsetzung des Projektes anhand der Konzepte aus dem ersten Teil. Ziel dieses Teils ist die schrittweise Entwicklung eines unternehmensspezifischen Code-Completion-Systems — von der Konzeption über die Implementierung bis zur Evaluation der Ergebnisse. Abschließend wird eine Reflexion der Erkenntnisse und der Durchführung der Arbeit vorgenommen, um einen Ausblick auf zukünftige Entwicklungen zu geben. Dieser Teil beginnt mit der Marktforschung und einer Analyse des Ist-Zustandes. Durch die klare Trennung zwischen theoretischem Fundament und praktischer Umsetzung wird eine konsistente Leserführung gewährleistet. Anschließend folgt der Aufbau eines Soll-Zustandes durch die Auswahl und Anpassung geeigneter Modelle und Plattformen.

Um einen Überblick zu schaffen, gibt die folgende Übersicht die einzelnen Kapitel und deren Inhalte wieder:

Marktentwicklung und technologische Trends von Chatbots und Code-Completion-Systemen

Es werden wichtige historische Ereignisse beschrieben und deren Einfluss auf die allgemeine

Entwicklung bis hin zu modernen Chatbots und Code-Completion-Systemen erläutert. Ein besonderes Augenmerk wird auf die kürzlichen Entwicklungen gelegt, die den Durchbruch solcher Systeme in der heutigen Zeit ermöglicht haben.

Relevanz und Zielgruppen von Code-Completion-Systemen

Die Relevanz von Code-Completion-Systemen wird hier beschrieben. Erläutert werden hierzu die Zielgruppen und übliche Einsatzkontexte. Zudem wird zwischen Code-Completion-Systemen und allgemeinen Chatbots differenziert, die in dieser Arbeit eine untergeordnete Rolle spielen.

Datenschutz, regulatorische und rechtliche Aspekte

Oft werden sensible Daten in Code-Completion-Systemen verarbeitet. Eine missbräuchliche Verwendung oder Weitergabe dieser Daten kann schwerwiegende Folgen für das Unternehmen und seine Kunden haben. Daher wurden vom Gesetzgeber Regelungen getroffen, um den Umgang mit solchen Daten zu regeln. In diesem Kapitel werden zum einen die besagten Regelungen beschrieben und zum anderen weitere Herausforderungen, Risiken und mögliche Strategien zur Risikominimierung erläutert.

Methoden der Projektkonzeptionierung

In diesem Kapitel werden jene Methoden zur Projektkonzeptionierung beschrieben, die für die Durchführung der Arbeit benötigt werden. Unter anderem wird auf die Vorgehensweise zur Analyse von Ist- und Soll-Zuständen eingegangen, zum anderen das Vorgehen zur Realisierung eines Prototypen des Produktes.

Analyseverfahren von Plattformen

Die ausgewählten Plattformen müssen anhand ihrer Eignung für den Unternehmenskontext analysiert werden. Um dies anhand von objektiven Kriterien festzustellen, werden in diesem Kapitel Analyseverfahren beschrieben, die in den Kapiteln 9 (Analyse der aktuellen Plattform Refact.ai) und 10 (Analyse alternativer Plattformen) zur Anwendung kommen.

Evaluationsverfahren für KI-Modelle

Ähnlich wie bei den Analyseverfahren für Plattformen, werden auch Verfahren zur Evaluation von KI-Modellen benötigt, um deren Eignung für den Unternehmenskontext zu bestimmen. Sie finden im Kapitel 11 (Evaluation vielversprechender Modelle) Anwendung. Insbesondere die Frage nach der Qualität der Evaluation ist von Bedeutung, da viele öffentliche Benchmarks und Leaderboards nur begrenzte Aussagekraft und Glaubwürdigkeit genießen, besonders im Unternehmenskontext.

Marktforschung und Unternehmensanforderungen

Die Marktforschung markiert den Beginn des praxisorientierten Teils der Arbeit. Hier wird zunächst umfassend erhoben, welche Anforderungen der Markt an Code-Completion-Systeme stellt und welche Plattformen und Modelle momentan zur Verfügung stehen. Es kann erst dann ein systematischer Vergleich stattfinden.

Analyse der aktuellen Plattform Refact.ai

Das Produkt nutzt zu Beginn dieser Arbeit die Plattform RefactAi. Dabei soll der aktuelle Ist-Zustand beschrieben werden. Im Fokus stehen die vorhandenen Funktionen sowie die Stärken der Plattform. Gleichzeitig werden mögliche Schwächen und Kritikpunkte aufgezeigt. Es soll festgestellt werden, weshalb das Unternehmen einen Bedarf an einer Anpassung des Systems hat und welche Anforderungen nicht erfüllt werden.

Analyse alternativer Plattformen

Analog zu Kapitel 9 wird in diesem Kapitel die Soll-Analyse durchgeführt. Nachdem eine Auswahl an Kandidaten getroffen wurde, werden diese Plattformen wie die RefactAi-Plattform analysiert. Ziel ist es, die Unterschiede und Gemeinsamkeiten der Plattformen klar herauszustellen und zu sammeln. Anschließend werden die Ergebnisse auf die Anforderungen angewendet und eine Entscheidung getroffen. Die neue Plattform bildet dann die Basis für die weitere Entwicklung.

Evaluation vielversprechender Modelle

Nachdem der Prototyp erstellt wurde, wird die Lösung anhand von Evaluationsverfahren getestet. Ziel ist es, die Auswahl der KI-Modelle zu prüfen. So kann sichergestellt werden, dass die Modelle den maximalen Nutzen für die Kunden bieten und maximale Wirtschaftlichkeit für das Unternehmen erzielt wird.

Ergebnisse und Entscheidung

Die neue Lösung wird in diesem Kapitel vorgestellt. Alle wichtigen Informationen werden erneut aufgegriffen und zusammengefasst. Die Ergebnisse der Evaluationen und dem Erfolg der Anpassung werden hier zusammengefasst.

Reflexion und Ausblick

Damit eine fortlaufende Verbesserung stattfinden kann, wird in diesem Kapitel die Lösung auf mögliche Schwächen untersucht und gegebenenfalls werden Verbesserungen vorgeschlagen. Außerdem wird die allgemeine Umsetzung der Arbeit sowie der Projektverlauf reflektiert. Zuletzt wird ein Ausblick auf das weitere Vorgehen gegeben. Es wird vorausschauend auf mögliche nächste Schritte und Entwicklungen eingegangen, die sowohl für das Unternehmen, als auch für die Gesellschaft von Bedeutung sein könnten.

1.4 Methodik

In der vorliegenden Arbeit wird anwendungsorientiert geforscht. Ziel ist es, eine bestehende Code-Completion-Lösung im Unternehmenskontext zu evaluieren und gezielt zu verbessern. Anschließend wird ein darauf basierender Prototyp entwickelt. Methodisch wird dabei ein Design Science Research (DSR)-Ansatz verfolgt, ein in der Wirtschaftsinformatik etabliertes Rahmenwerk zur schrittweisen Entwicklung und Bewertung von sogenannten Artefakten. Dieser Ansatz eignet sich besonders für praxisnahe Forschungsprojekte, die sowohl technische Artefakte, als auch organisatorische Aspekte berücksichtigen. Obwohl sich dieses Rahmenwerk eher aus der Praxis entwi-

ckelt hat, als dass es direkt erfunden wurde, so haben Alan Hevner et al. im Jahre 2004 einen theoretischen Grundstein gelegt, der auch in dieser Arbeit den Grundstein für die methodische Herangehensweise bildet¹².

Angewandt auf dieses Projekt kann der iterative Arbeitsprozess nach Hevner et al. in drei Phasen unterteilt werden:

1. Problemanalyse und Zieldefinition:

Zuerst wird eine Schwachstellenanalyse auf Basis einer Ist-Analyse der bestehenden Refact.ai Lösung durchgeführt. Es werden Schwachstellen basierend auf technischen Aspekten, bereits dokumentierten Einschränkungen und Rückmeldungen interner Kunden identifiziert.

2. Artefaktentwicklung und Anpassung:

Nachdem Defizite identifiziert wurden, wird erarbeitet, wie diese behoben werden können. Daraufhin wird ein anpassbarer Prototyp entwickelt auf einer speziell eingerichteten Testumgebung. So können Veränderungen und Alternativen ausprobiert werden, ohne die Entwicklung am Hauptsystem zu stören. Verschiedene Backend-Komponenten, wie die KI-Modelle können schnell und problemlos getauscht werden.

3. Evaluation durch Nutzerfeedback:

Im Mittelpunkt steht in dieser Phase das Feedback von ausgewählten Testern und Testerrinnen, die den Prototypen erhalten haben und in realitätsnahen Szenarien testen. Durch ein Feedback-Formular wird zu jeder Veränderung eine Rückmeldung zum Nutzererlebnis und der wahrgenommenen Qualität des Vorschlags eingeholt. Darüber hinaus werden sekundär auch technische Metriken von bestehenden Benchmarks herangezogen, jedoch steht die Nutzerperspektive im Vordergrund.

Es werden für die Bearbeitung der Forschungsfrage abseits des DSR-Ansatzes keine weiteren externen Normen oder Standards verwendet, aber es wird im Laufe der Arbeit auf etablierte Kriterien der Softwarequalität geachtet. Die Testumgebung muss mit begrenzten Ressourcen auskommen, sodass statt geeigneteren GPUs lediglich eine *Nvidia A10G-GPU* zur Verfügung steht mit 24GB VRAM. Das führt zu Einschränkungen in der Auswahl der Modelle, die im Praxisteil verwendet werden können.

Der Praxisbezug der Arbeit verfolgt insgesamt ein explorativ-evaluiertes Vorgehen. Zwar sind die zu erwartenden Ergebnisse spezifisch auf die S-MS ausgerichtet, jedoch steht es nicht außer Frage, dass diese für ähnliche Einsatzszenarien wichtige Hinweise und Erkenntnisse in anderen Organisationen liefern können. Das gilt insbesondere in Bezug auf den Datenschutz und lokal betreibbare Lösungen.

¹²Vgl. Hevner et al. 2004

2 Marktentwicklung und technologische Trends von Chatbots und Code-Completion-Systemen

Durch die immer weiter beschleunigende Entwicklung von KI-gestützten Assistenzsystemen in den letzten Jahren hat sich ein neues Kapitel der heutigen Zeit aufgetan¹³. Vor allem Chatbots haben sämtliche Bereiche des alltäglichen Lebens und der Arbeit verändert. Auf dieser Basis konnte auch die Entwicklung von Code-Completion-Systemen ins Rollen kommen¹⁴. Dieses Kapitel zeigt auf, wie historische Meilensteine auf die Entwicklung und Verbreitung von Code-Completion-Systemen, Chatbots sowie weiteren KI-Modellen Einfluss genommen haben. Außerdem wird konkret darauf eingegangen, wie sich der gewöhnliche Chatbot zu einem Code-Completion-System entwickelt hat und in welchen Eigenschaften und Aufgaben sie sich unterscheiden. Das geschieht aus Sicht der Marktbedürfnisse und der technologischen Trends und Durchbrüche, die den Markt prägen.

Das Kapitel hilft dem Leser bzw. der Leserin zu verstehen, woher die aktuellen Ansprüche und Möglichkeiten der Code-Completion-Systeme kommen, sodass die Relevanz und der Nutzen im nachfolgenden Kapitel nachvollzogen werden kann.

2.1 Die Evolution intelligenter Assistenzsysteme

Es scheint ein alter menschlicher Traum zu sein, Maschinen mit menschenähnlicher oder gar übermenschlicher Intelligenz zu erschaffen, sogenannter KI¹⁵. Für die meiste Zeit der Menschheitsgeschichte blieb dieser Wunsch unverwirklicht. Die Anfänge der KI liegen erst in den 1950er Jahren¹⁶, als die ersten theoretischen Grundlagen intelligenter Systeme entwickelt wurden, darunter auch der Turing-Test. Darunter ist ein von Alan Turing entwickelter Test zu verstehen, der die Fähigkeit untersucht, ob ein Computer menschliches Kommunikationsverhalten simulieren kann¹⁷. Es stellt einen großen Meilenstein dar, diesen Test zu bestehen, da dann der Computer nicht mehr von einem Menschen zu unterscheiden ist.

Zu den bekanntesten frühen Systemen gehört das von Joseph Weizenbaum entwickelte *ELIZA*, eines der ersten Programme, das heute als Chatbot bezeichnet werden würde¹⁸. Die womöglich bekannteste Konfiguration des Systems hatte die Aufgabe, Patienten mit psychologischen Problemen in der Rolle eines Therapeuten zu helfen.

Zu dieser Zeit wurde noch keine KI verwendet, stattdessen ist *ELIZA* ein symbolisches und regelbasiertes Pattern-Matching-System. Das bedeutet, dass das Programm die Eingaben der

¹³Vgl. Aslam 2023, S. 70

¹⁴Vgl. Smutny, Bojko 2024, S. 1

¹⁵Vgl. Farrow 2019, S. 62 f.

¹⁶Vgl. Muggleton 2024, S. 2

¹⁷Vgl. Turing 1950, S. 433 f.

¹⁸Vgl. Natale 2019, S. 9f

Nutzerinnen und Nutzer erhält, daraufhin ein Skript ausführt und anschließend eine Antwort generiert. *ELIZA* arbeitet mit Keywords und Pattern-Matching, um aus einer Eingabe ein im Vorfeld definiertes Satzbaumuster zu erkennen und wichtige Wörter zu extrahieren¹⁹.

Bei der Ausgabe gibt es dann für jedes identifizierte Pattern eine Liste an Vorlagen mit Platzhaltern. Gibt es mehrere passende Vorlagen für die Ausgabe, wird zufällig einer der Kandidaten ausgewählt. Die Platzhalter werden dann mit den extrahierten Wörtern gefüllt und transformiert, also grammatikalisch angepasst. Darauf folgt dann eine Antwort, die daraufhin den Nutzerinnen und Nutzern präsentiert wird.

Obwohl *ELIZA* nicht tatsächlich versteht, was mit der Eingabe oder Ausgabe gemeint ist, konnte es zu dieser Zeit bereits täuschend echte Gespräche führen. Es war allerdings noch weit davon entfernt, den Turing-Test zu bestehen.

Die stark begrenzte oder gar fehlende Lernfähigkeit dieser Programme war eine große technische Limitierung, die für eine lange Zeit nicht überwunden wurde.

Die nächsten Schritte in der Entwicklung solcher Systeme waren die in den 1970er- und 1980er-Jahren aufkommenden Expertensysteme. Sie entstanden in einem Umfeld stark geförderter Forschung maschineller Wissensrepräsentationen. Die hohe Nachfrage solcher Forschung kam hauptsächlich durch militärische und industrielle Problemstellungen wie *XCON* bei der Digital Equipment Corporation²⁰ oder *MYCIN* in der Medizin²¹. Auch sie sind regelbasiert und symbolisch, aber im Gegensatz zu *ELIZA* beruhen die Antworten nicht auf festgelegten Mustern, sondern auf einer Wissensdatenbank. Expertensysteme sind in der Lage, mit WENN-DANN-Regeln logische Schlussfolgerungen zu ziehen. Solche logischen Schlüsse werden dann miteinander verkettet, um komplexe Probleme zu lösen. Darüber hinaus konnte nachvollzogen werden, wie die Ergebnisse zustande kamen, denn jede logische Schlussfolgerung wurde durch die zugrunde liegenden Regeln nachvollziehbar erklärt²². Trotzdem blieb das Problem der begrenzten Lernfähigkeit bestehen und mit ansteigender Komplexität der manuell angefertigten Wissensdatenbanken stieg auch die benötigte Rechenleistung exponentiell an²³. Ein weiteres zentrales Defizit war die Unfähigkeit, aus Erfahrung zu lernen, wie heutige datengetriebene Ansätze es können²⁴.

Der nächste große Schritt geschah nach der Jahrtausendwende dank einer massiven Zunahme der verfügbaren Rechenleistung²⁵. Grund für den Anstieg waren die Taktfrequenzsteigerungen der Prozessoren in den frühen 2000er Jahren²⁶ sowie das Aufkommen spezialisierter Server²⁷. Ab Mitte der 2000er Jahre ermöglichten Mehrkernprozessoren zusätzlich die Parallelisierung im großen Stil²⁸. Zudem konnten mit der Vorstellung von *CUDA* im Jahr 2006 auch Grafikkar-

¹⁹Vgl. Natale 2019, S. 10 f.

²⁰Vgl. Barker et al. 1989, S. 301

²¹Vgl. Buchanan, Smith 1988, S. 27, 46

²²Vgl. Abraham 2005, S. 10 f.

²³Vgl. Buchanan, Smith 1988, S. 35, 38

²⁴Vgl. Hubert, Dreyfus 1986, S. 96, 98

²⁵Vgl. Danowitz et al. 2012, S. 1

²⁶Vgl. Danowitz et al. 2012, S. 9 ff.

²⁷Vgl. Hamilton 2009, S. 1 f.

²⁸Vgl. Sutter, Larus 2005, S. 54 f.

ten zunehmend für die Berechnung von KI-Algorithmen genutzt werden²⁹. Parallel dazu legte Geoffrey Hinton mit dem Revival von Deep Neural Networks (DNNs) (tiefen neuronalen Netzen) den Grundstein für das Deep Learning³⁰, das klassische symbolische Ansätze zunehmend verdrängte. Es war zudem eine Zeit, in der das Internet bereits weit verbreitet war. Zeitgleich stieg die Menge an öffentlich zugänglichen digitalen Textdaten durch Foren, Wikis und andere Online-Quellen, die als Trainingsdaten für KI-Modelle genutzt werden konnten, dramatisch an. Durch das Internet konnte Zugriff auf große Datenmengen ohne Ortsbeschränkungen ermöglicht werden, was ein enormer Erfolgsfaktor von datengetriebenen Systemen war. Dadurch erlebten die 2000er Jahre einen Umstieg von symbolischer zu datengetriebener KI und markieren damit den Beginn von *Machine Learning* als dominierenden KI-Ansatz³¹. Solche Modelle konnten weitaus mehr Aufgaben lösen als ihre regelbasierten Vorgänger. So konnten beispielsweise Klassifikationsprobleme für Texte gelöst werden oder Dialogsysteme in Call-Centern Anwendung finden. Ein Resultat daraus waren Spamfilter in E-Mail-Programmen mit deutlich gesteigerter Leistungsfähigkeit. Zwar stellten diese Fortschritte einen großen Schritt in der Entwicklung dar, dennoch blieben die meisten Chatbots in dieser Zeit weiterhin regelbasiert.

Das erklärt, warum die zu dieser Übergangszeit populären Chatbots wie *Artificial Linguistic Internet Computer Entity (A.L.I.C.E)* und *SmarterChild* selbst ohne diese Fortschritte große Relevanz hatten³².

Obwohl dies eine große Verbesserung darstellte, reichte es nicht aus, um Chatbots maßgeblich zu verbessern. Diese arbeiteten weiterhin überwiegend regelbasiert. Ein beispielhafter Chatbot aus dieser Zeit ist *A.L.I.C.E*. Dieser Chatbot wurde zwar im Jahr 1995 veröffentlicht, wurde aber erst 1998 bis 2001 wirklich bekannt. Er gewann drei Mal den Loebner Prize, was die Popularität des Systems steigerte³³. Der Loebner Prize basiert auf einer stark vereinfachten Form des Turing-Tests und war ein jährlicher Wettbewerb, bei dem Chatbots gegeneinander antraten³⁴.

Der Unterschied zu früheren Systemen wie *ELIZA* und den Expertensystemen ist, dass *A.L.I.C.E* durch das Internet massentauglich und Open-Source war. So konnte *A.L.I.C.E* von einer großen Community weiterentwickelt und Experimente mit dem System durchgeführt werden³⁵. Community-Entwickler konnten eigene Regeln bauen und so ganze Persönlichkeiten erschaffen, dank eines eigenen XML-Dialekts, der *AIML (Artificial Intelligence Markup Language)*. Der *A.L.I.C.E*-Chatbot markierte den Höhepunkt der regelbasierten Chatbots³⁶.

Das Ende der regelbasierten Systeme lässt sich mit ihrer fehlenden semantischen Tiefe begründen. Diese Tiefe ist notwendig, um Bedeutung in Sprache angemessen zu erfassen. Aufgrund dieser Einschränkung sind regelbasierte Systeme nicht in der Lage, bei komplexen Konversatio-

²⁹Vgl. Nickolls et al. 2008, S. 41-45

³⁰Vgl. Hinton et al. 2012, S. 82 ff.

³¹Vgl. Jordan, Mitchell 2015, S. 255

³²Vgl. Al-Amin et al. 2024, S. 9 ff.

³³Vgl. Deryugina 2010, S. 144

³⁴Vgl. Wallace 2009, S. 1f, 5

³⁵Vgl. Wallace 2009, S. 1

³⁶Vgl. Singh, Joesph, Jabbar 2019, S. 2

nen sinnvolle Antworten zu geben³⁷. Aufgrund dessen stieg der Druck auf die Entwickler von Chatbots, ihre Systeme weiterzuentwickeln und neue Ansätze zu finden. So gewannen ab Mitte der 2000er Jahre die datengetriebenen Methoden deutlich an Bedeutung.

Wenige Jahre später entstanden die ersten Chatbots, die nicht ausschließlich regelbasiert arbeiteten. So hat zum Beispiel das Unternehmen ActiveBuddy (heute Colloquis) im Jahr 2001 den frühen Vertreter dieser Chatbots *SmarterChild* veröffentlicht. *SmarterChild* war populär auf Plattformen wie dem *AOL Instant Messenger* und dem *MSN Messenger*^{38,39}. Neben den regelbasierten Elementen, wie sie auch bei *ELIZA* und *A.L.I.C.E* vorkommen, arbeitete *SmarterChild* vermutlich mit einfachen heuristischen Verfahren⁴⁰, die an n-gram-basierte Wahrscheinlichkeiten erinnerten⁴¹. Diese Methoden ermöglichten es dem System, aus einer großen Datenbank vordefinierter Antworten, die am besten passende auszuwählen — auch unter Einbeziehung des unmittelbaren Kontexts⁴². Dies markierte einen frühen Schritt in Richtung eines kontextsensitiven Verhaltens, das als Vorläufer moderner Natural Language Processing (NLP)-Systeme betrachtet werden kann⁴³. Dadurch wirkten die Antworten im Vergleich zu früheren Chatbots deutlich natürlicher und dynamischer. Nennbare Schwächen sind jedoch die Unfähigkeit, ein echtes Verständnis der Konversation zu entwickeln, und der fehlende Erhalt des Kontexts über frühere Sätze hinaus⁴⁴. Trotz dieser Schwächen stellt *SmarterChild* einen Meilenstein in der Nutzerakzeptanz dar⁴⁵. Auf diese etablierten Nutzergewohnheiten bauten nachfolgende Systeme wie *Siri* (2011) und *Google Now* (2012) auf — die ersten virtuellen Assistenten, die auf Spracherkennung und NLP basieren⁴⁶. *Siri* selbst entstand aus dem *DARPA*-finanzierten *CALO*-Projekt⁴⁷.

Die Grundlagen für datengetriebene, lernfähige Systeme steckten noch in den Kinderschuhen. Das änderte sich mit den ersten NLP-Tools mit Machine Learning (ML)-Unterbau. Die Stanford NLP Group begann ab dem Jahr 2006 mit dem Entwickeln essenzieller Tools, die bis heute von bedeutender Relevanz sind⁴⁸. Darunter gehören Preprocessing-Tools wie dem *Stanford POS Tagger*⁴⁹, dem *Stanford NER (Named Entity Recognizer)* sowie dem *Stanford Parser (Statistical Parser)*. Diese weit verbreiteten Open-Source NLP-Tools verhelfen zu sauberen und testbaren Modellen, robusten Baselines, sowie der Vergleichbarkeit von Modellen über Datensätze hinweg.

Darauf folgend wurden einige wichtige Modellarchitekturen entwickelt. Bereits ab den frühen 2000er Jahren kamen Support Vector Machines (SVMs) verstärkt in der Textverarbeitung zum Einsatz, ab etwa 2007 dominierten sie viele Klassifikationsprobleme⁵⁰. Obwohl das Konzept der

³⁷Vgl. Bangalore, Johnston 2003, S. 221, 224

³⁸Vgl. Levitan 2015

³⁹Vgl. Silberling 2023

⁴⁰Vgl. Sultan 2022, S. 21-24

⁴¹Vgl. Wetstein 2017, S. 26, 62 ff.

⁴²Vgl. Al-Amin et al. 2024, S. 8f

⁴³Vgl. Tsikora 2023, S. 11-15

⁴⁴Vgl. Sultan 2022, S. 8f

⁴⁵Vgl. Silberling 2023

⁴⁶Vgl. Jampala et al. 2024, S. 1333 f.

⁴⁷Vgl. Canbek, Mutlu 2016, S. 596 f.

⁴⁸Vgl. Hirschberg, Manning 2015, S. 261

⁴⁹Vgl. The Stanford Natural Language Processing Group 2025

⁵⁰Vgl. Moraes, Valiati, Gavião Neto 2013, S. 621 f., 624

SVM schon in den 1990er Jahren entwickelt wurde, erlebte sie ihren breiten Durchbruch durch die Verfügbarkeit großer Datenmengen, Rechenleistung und offene Tools. Die SVM trennt Datenpunkte mithilfe einer optimalen Hyperplane (Hyperfläche), welche den Abstand zwischen den Datenpunkten der verschiedenen Klassen maximiert. So entsteht eine klare Definition, die Datenpunkte einer bestimmten Klasse zuordnet. Konkret konnte diese Architektur bei Chatbots zur Klassifikation des Intents (Absicht) des Nutzers (Frage, Befehl, Aussage etc.) verwendet werden.

Mit den aufkommenden 2010er Jahren rückten allmählich die DNNs in den Fokus der Forschung und Anwendung. 2013 wurde mit der Veröffentlichung von *Word2Vec* durch ein Team bei Google eine grundlegende Methode für die Erstellung verteilter Wortrepräsentationen eingeführt⁵¹. Damit wurde eine der zentralen Technologien für die fortlaufende Entwicklung von NLP gelegt. *Word2Vec* bereitete den Boden für die Entwicklung von Modellen, die ein Verständnis für den Kontext sowie die Bedeutung von Wörtern im Satz ermöglichen. Erstmals war es möglich, semantische Zusammenhänge in Vektoren darzustellen. Dadurch konnte eine mathematische Modellierung von Wörtern und deren Bedeutung erfolgen. Ein oft zitiertes Beispiel zur Veranschaulichung dieses Konzepts ist: „König - Mann + Frau = Königin“⁵².

Darauf aufbauend entstanden weitere Embedding-Modellkonzeptionen wie *GloVe* (*Global Vectors for Word Representation*) (2014), *fastText* (2016) und *ELMo* (*Embeddings from Language Models*) (2018)⁵³. Sie verbesserten insbesondere die Kontexttiefe und semantische Kohärenz. *ELMo* führte erstmals kontextsensitive Wortvektoren ein, indem es ganze Sätze analysierte⁵⁴. Aus dem Aufschwung der DNN-Modelle wurden ab etwa 2013 auch Recurrent Neural Networks (RNNs) (rekurrente neuronale Netze) wieder stärker erforscht, obwohl sie ursprünglich bereits in den 1980er Jahren (z. B. Elman-Netzwerke) entwickelt wurden⁵⁵. Anders als herkömmliche Feed-Forward-DNNs sind RNNs mit Speicherzellen ausgestattet, um Informationen aus vorherigen Zeitschritten zu behalten⁵⁶. Das erlaubt ihnen, sequenzielle Daten wie Texte zu verarbeiten und dabei den Kontext vorheriger Wörter zu berücksichtigen. Ihre Kontexttiefe ist jedoch begrenzt durch die *Vanishing Gradient*-Problematik, die insbesondere bei langen Sequenzen auftritt. Technisch bedeutet das, dass die Gradienten beim Backpropagation-Prozess exponentiell gegen Null tendieren und Gewichte somit nicht mehr effektiv angepasst werden können⁵⁷. Langfristige Abhängigkeiten bleiben dadurch schwer erfassbar. Als Weiterentwicklung der RNN-Architektur wurde *LSTM* (*Long Short-Term Memory*) eingeführt, die 1997 von Hochreiter und Schmidhuber entwickelt wurde. LSTM Modelle sind durch sogenannte *Gates* (Tore) in der Lage, den Informationsfluss und -verlust gezielt zu steuern, wodurch sie besser mit langen Abhängigkeiten umgehen können⁵⁸.

Bahnbrechend war anschließend der Einsatz von Attention-Mechanismen. Diese erlauben es,

⁵¹Vgl. Naseem et al. 2021, S. 21 f.

⁵²Vgl. Mikolov et al. 2025, S. 2

⁵³Vgl. Naseem et al. 2021, S. 21

⁵⁴Vgl. Naseem et al. 2021, S. 13 ff.

⁵⁵Vgl. Davis 2016, S. 1

⁵⁶Vgl. Davis 2016, S. 16-19

⁵⁷Vgl. Davis 2016, S. 10 f.

⁵⁸Vgl. Davis 2016, S. 18

die Relevanz von Beziehungen zwischen Wörtern in einem Satz explizit zu gewichten. Dadurch kann der Kontext über deutlich längere Sequenzen hinweg effektiv erfasst werden. Das *Sequence-to-Sequence (Seq2Seq)*-Modell mit Attention wurde 2015 von Bahdanau et al. eingeführt⁵⁹. Es kombiniert die Encoder-Decoder-Architektur mit einem Aufmerksamkeitsmechanismus, wodurch Übersetzungen und andere sequenzielle Aufgaben deutlich verbessert wurden⁶⁰.

Die Reaktion auf den Attention-Mechanismus war die bahnbrechende Einführung des *Transformer*-Modells von Google im Jahr 2017. In der Arbeit *Attention is All You Need* von Vaswani et al. wurde die Architektur definiert. Sie basiert auf dem Encoder-Decoder-Prinzip, verzichtet dabei allerdings vollständig auf rekurrente Strukturen und nutzt anstatt dessen ausschließlich Attention-Mechanismen und Feed-Forward DNNs. Das erlaubt ihnen im Gegensatz zu ihren Vorgängern eine parallele Verarbeitung von Daten. Das verkürzt die Trainingszeit massiv und erlaubt die Verarbeitung von größeren Datenmengen⁶¹. Bis heute setzen Transformer-Modelle die Messlatte für viele Bereiche des ML und insbesondere des NLP immer höher und höher⁶².

Auf diesem Grundstein entstanden bedeutende Modelle, die die Chatbots revolutionierten. Einige Durchbrüche stellen die *BERT (Bidirectional Encoder Representations from Transformers)* Architektur von Google (2018) sowie die *Generative Pre-trained Transformer (GPT)-2* Architektur von OpenAI (2019) dar⁶³. Heute existieren viele Varianten von Modellen, die auf dieser Transformer-Architektur beruhen, sowohl proprietär, als auch Open Source. Zum heutigen State-of-the-Art Repertoire gehört das global leistungsfähigste Modell *GPT-4.5* von OpenAI⁶⁴, dazu *Claude 3.7 Sonnet* von Anthropic⁶⁵, *Gemini 2.5 Pro Experimental* von Google DeepMind⁶⁶, *Qwen 2.5-Max* von Alibaba⁶⁷ und *LLaMA 4 Behemoth* von Meta AI⁶⁸.

Diese Modelle sind nicht nur bemerkenswert kompetent darin natürlich wirkende Gespräche zu führen, sondern zeigen auch eine beachtliche Problemlösungsfähigkeit. So sind sie oftmals schon in der Lage Fachexperten der Mathematik, Informatik und Naturwissenschaften in spezialisierten Tests zu übertreffen⁶⁹. Dies verdeutlicht das Potenzial von modernen Chatbots als Assistenzsysteme in diversen Bereichen zu fungieren. Damit verändert sich unweigerlich die Art und Weise, wie wir Menschen am Arbeitsplatz und im Alltag arbeiten und damit auch die Gesellschaft als Ganzes.

⁵⁹Vgl. Bahdanau, Cho, Bengio 2014, S. 1-4

⁶⁰Vgl. Puspitaningrum 2021, S. 271 f.

⁶¹Vgl. Vaswani et al. 2017, S. 1 f.

⁶²Vgl. Patwardhan, Marrone, Sansone 2023, S. 1 f.

⁶³Vgl. Keshamoni 2023, S. 448-252

⁶⁴Vgl. OpenAI 2025

⁶⁵Vgl. Anthropic 2025

⁶⁶Vgl. Google DeepMind 2025

⁶⁷Vgl. Qwen 2025

⁶⁸Vgl. Meta 2025

⁶⁹Vgl. Zhong et al. 2024, S. 3

2.2 Die Entwicklung von Code-Completion-Systemen im Kontext dialogorientierter KI

Dialogorientierte Assistenzsysteme wurden durch *Siri* in mobilen Apple Geräten und *Google Now* in Android-Geräten erstmals flächendeckend eingesetzt. Beide nutzten zunächst einfache NLP-Architekturen und regelbasierte Algorithmen, um Antworten auf Anfragen zu generieren oder bestimmte Aktionen auszuführen⁷⁰. Ihre Fähigkeiten waren jedoch durch die vordefinierten Aktionsmöglichkeiten beschränkt. Sie waren damit nur wirklich hilfreich, um die Bedienung des Geräts selbst zu erleichtern oder einfache Fragen zu beantworten. Bei komplexeren Aufgaben konnten diese Systeme keine Hilfe leisten, weshalb sie noch nicht als wirklich intelligent angesehen werden konnten. Dass es sich um eine Maschine handelt, wird bei der Benutzung schnell deutlich⁷¹.

Aufgrund der vielen Umfelder, in denen solche Assistenzsysteme theoretisch eingesetzt werden könnten, liegt es nahe, dass es nicht unbedingt sinnvoll ist, ein System für alle Bereiche zu entwickeln. Vielmehr könnte es hilfreich sein, ein System für eine spezifische Aufgabendomäne zu entwickeln, sodass mehr Fachwissen, Aktionsmöglichkeiten und Integration in bestehende Systeme möglich ist⁷².

Im Bereich der Softwareentwicklung stellen allgemeine Chatbots bereits eine ideale Grundlage dar, da sie mittlerweile in der Lage sind, Kontext hinter den Wörtern der Anfrage zu erkennen und zu verstehen⁷³. Diese Fähigkeit ist zentral in einem logikbasierten Bereich wie der Softwareentwicklung. Ein präzises Verständnis von Projektstrukturen, Syntax und Semantik sind in diesem Fall unverzichtbar⁷⁴. Allerdings besteht dennoch das Problem, dass diese Systeme nicht fokussiert auf technische Kontexte trainiert wurden.

Das Potenzial solcher spezialisierten KI-gestützter Systeme für Entwickler haben Unternehmen längst erkannt⁷⁵. Es liegt schließlich im Interesse von IT-Unternehmen, die Effizienz des Entwicklungsprozesses zu maximieren, um so Ressourcen zu sparen und gleichzeitig die Qualität durch Automatisierung alltäglicher Aufgaben zu verbessern. Eine erste Demonstration dieses Potenzials in der Praxis waren diverse Tools wie *Kite* und *TabNine*⁷⁶. Im Gegensatz zu allgemein trainierten Chatbots, wurden diese Modelle gezielt auf Programmierdaten und typische Softwareentwicklungsprobleme trainiert. Sie sind außerdem nicht wie Chatbots für den Dialog konzipiert, sondern für die Vervollständigung von Code am Cursor in einer IDE.

Diese Schritte der Spezialisierung markieren einen Paradigmenwechsel. Der erfolgte Wandel dieser Systeme wurde insbesondere durch die Auffassungsgabe der Transformer-Architektur ins Rollen

⁷⁰Vgl. Jampala et al. 2024, S. 1333 f.

⁷¹Vgl. Canbek, Mutlu 2016, S. 598 f.

⁷²Vgl. Schmidt, Alt, Zimmermann 2021, S. 4026-4030

⁷³Vgl. Melo et al. 2020, S. 1, 7

⁷⁴Vgl. Maletic, Marcus 2001, S. 103

⁷⁵Vgl. Daniel Ajiga et al. 2024, S. 1897 ff.

⁷⁶Vgl. Hassan 2024, S. 12-14

gebracht⁷⁷. Ein solches Beispiel stellt *GitHub Copilot* dar, dem heutigen State-of-the-Art Code-Completion-System. Dahinter steckt die sogenannte *Codex*-Architektur, die wiederum auf dem *GPT-3* Modell von OpenAI basiert⁷⁸.

Die genannten frühen Code-Completion-Systeme basierten auf domänenspezifischen Trainingsdaten, doch ihre Leistung blieb trotzdem hinter den Erwartungen zurück. Transformer-basierte Modelle dagegen erfüllen die Anforderungen deutlich besser. Die Idee des Einsatzes der Transformer-Architektur ergab sich daraus, dass OpenAIs *GPT-2* und *GPT-3* Modelle auffällig gut darin waren, nicht nur natürliche Sprache zu verarbeiten, sondern auch Code zu generieren⁷⁹. Doch es fehlte noch die Tiefe für das Verständnis von Code, sodass dieser sowohl logisch sinnvoll, als auch syntaktisch korrekt ist⁸⁰.

Im Jahr 2021 wurde auf *GPT-3* ein Fine-Tuning mit GitHub-Repositories durchgeführt und das Modell *Codex* geboren. Es ist gleichzeitig in der Lage natürliche Sprache und Code zu verstehen⁸¹. Für die Softwareentwicklung stellte dies einen Quantensprung dar.

Es ist außerdem zu bemerken, dass sich die Transformer-Architektur oft sogar besser für Code eignet als für natürliche Sprache. Das entspringt der deterministischen Natur von Code, die feste Regeln und standardisierte Strukturen und Benennungen vorgibt⁸². Transformer arbeiten nach dem „Next-Token-Prediction“-Prinzip, weshalb diese Deterministik ideal ist⁸³.

Obwohl sich Chatbots und Code-Completion-Systeme in vielen Aspekten ähneln und z. T. auf den gleichen Technologien basieren, sind Code-Completion-Systeme keine bloße Variation von Chatbots. Da sich die Anforderungen, Einsatzstrategie, Interaktionsmuster, sowie der Kontext grundlegend unterscheiden, ist es angebracht, sie als eine eigenständige Kategorie einzustufen⁸⁴.

Chatbots arbeiten in der Regel mit kurzen Dialogen, während es bei Code-Completion-Systemen nicht nur auf die aktuelle Zeile ankommt, sondern meistens auf das gesamte Dokument oder sogar das gesamte Projekt. Jedoch sind Projekte oft sehr groß und vielschichtig. Das erhebt den Anspruch des Systems, ein Code-Search-Tool zu haben, das z. B. mithilfe von Indexierung der IDE den gesamten Code analysiert und die aktuelle Relevanz verschiedener Dokumente und Codeabschnitte festzulegen⁸⁵.

Code-Completion-Systeme müssen außerdem mit einer großen Menge an statischem Kontext auskommen. Darunter zählen geregelte Strukturen wie Importe oder projektweite Variablen- und Funktionsnamen oder -definitionen⁸⁶.

Bei Code-Completion-Systemen spielt die Reaktionszeit eine wichtige Rolle für das Nutzerinnen-

⁷⁷Vgl. Dewey 2024, S. 10

⁷⁸Vgl. Kantek 2023, S. 26, 31 f.

⁷⁹Vgl. Lajko et al. 2024, S. 34-37

⁸⁰Vgl. Savelka et al. 2023, S. 2, 10

⁸¹Vgl. Lomshakov et al. 2023, S. 171 f.

⁸²Vgl. Tipirneni, Zhu, Reddy 2022, S. 2

⁸³Vgl. Tipirneni, Zhu, Reddy 2022, S. 8

⁸⁴Vgl. Abdellatif et al. 2022, S. 3087 ff.

⁸⁵Vgl. Smutny, Bojko 2024, S. 4

⁸⁶Vgl. Blinn et al. 2024, S. 468 ff.

und Nutzererlebnis. Softwareentwicklerinnen und -entwickler sollen in ihrem Workflow nicht durch lange Wartezeiten gestört werden. Für die Akzeptanz des Systems ist die Reaktionszeit folglich bedeutend wichtiger als bei Chatbots, bei denen oft noch während der Generierung der Antwort mit dem Lesen begonnen werden kann⁸⁷.

Durch die Integration in IDEs kommen andere Prompts zustande als bei Chatbots, beispielsweise wird die Cursorposition eingespeist⁸⁸. Die Prompts sind außerdem keine klar formulierten Texte, sondern lediglich Bruchstücke von Code und unter Umständen mitten in einem syntaktischen Ausdruck. Auf diese Prompts muss das System gezielt ausgerichtet werden, um sinnvolle Vorschläge zu generieren⁸⁹.

Entwickler und Entwicklerinnen benötigen jedoch nicht nur Code-Vervollständigung, sondern auch Erklärungen und Hilfestellungen. Aus diesem Grund ist es sinnvoll, sowohl Code-Completion-Systeme als auch Chatbots in einem System zu vereinen. Somit entsteht ein hybrides System mit maximalem Kundennutzen⁹⁰. Um eine solche hybride Lösung bereitzustellen, benötigt es eine Plattform, die in die IDE integriert werden kann und als Schnittstelle zwischen dem Nutzer und den Modellen fungiert. Es muss in der Lage sein, für beide Modelltypen passende Oberflächen und Funktionalitäten bereitzustellen.

Das bis heute leistungsfähigste und populärste Code-Completion-System ist *GitHub Copilot*. Gleichzeitig ist es aber auch das System, das überhaupt erst für den Durchbruch von Code-Completion-Systemen gesorgt hat. Seine Veröffentlichung im Jahr 2021 beinhaltete eine Kombination des *Codex*-Modells und einer engen Integration in die Visual Studio Code IDE⁹¹. Es ist damit nicht nur architekturbedingt leistungsstark, sondern bietet auch die Fähigkeit, in Echtzeit kontextrelevante Vervollständigungen vorzuschlagen. Damit wurde die Messlatte erheblich höher gelegt und der Standard für sämtliche Akzeptanzkriterien wie der Usability oder dem Komfort in der Nutzung gesetzt⁹².

Ein Jahr später demonstrierte Google DeepMind mit ihrem Modell *AlphaCode*, dass generative Modelle hohe Kompetenz für einfache Codevervollständigungen ausweisen, viel wichtiger aber darüber hinaus auch komplexe algorithmische Aufgaben lösen können⁹³.

Darauf folgten einige Open-Source-Projekte, die es ermöglichten, die zugrunde liegende Technologie für die Allgemeinheit zugänglicher zu machen. Somit entstand ein Gegengewicht zu den proprietären Lösungen, die von großen Unternehmen wie Microsoft und Google angeboten werden. Heute findet sich auf Plattformen wie HuggingFace eine breite Palette an ML-Modellen aller Art, darunter auch viele Code-Completion-Systeme. So ist es möglich völlig ohne eine Angewiesenheit auf Cloud-Services Code-Completion-Systeme entweder privat oder im Unternehmen zu betreiben.

⁸⁷Vgl. Pinto et al. 2024, S. 82 f.

⁸⁸Vgl. Desolda et al. 2025, S. 7

⁸⁹Vgl. Li, Youjia, Shi, Zhang, Z. 2024, S. 53078

⁹⁰Vgl. Desolda et al. 2025, S. 12-16

⁹¹Vgl. Kantek 2023, S. 31

⁹²Vgl. Szolderits 2025, S. 23 ff.

⁹³Vgl. Li, Yujia et al. 2022, S. 1 f.

Mit der Zeit verbesserten sich insbesondere zwei Aspekte. Der erste ist die Integration in verschiedene IDEs. Somit ist es heute egal, ob nun mit JetBrains *IntelliJ*, *Visual Studio Code* oder einer anderen IDE gearbeitet wird. Die Plattformen werden zunehmend generalisiert, sodass sie in alle gängigen IDEs integriert werden können⁹⁴. Die funktionalen Unterschiede sollten dabei möglichst gering bleiben. Das heißt in der Praxis, dass beispielsweise in JetBrains *IntelliJ* und *Visual Studio Code* identische Funktionalitäten erwarten werden können. Der zweite Aspekt ist die Optimierung der Reaktionszeit die z. B. durch technische Fortschritte im Caching, Streaming oder der Kontextverarbeitung ermöglicht wurde⁹⁵.

Während bei Chatbots eine gewisse Kreativität nicht nur erlaubt, sondern oft sogar erwünscht ist, kann dies bei Code-Completion-Systemen nur bedingt behauptet werden. Es ist zwar wichtig, dass ein gewisser Einfallsreichtum beim Lösen von Codeproblemen vorhanden ist, jedoch darf das System nicht von den vorgegebenen Regeln der Programmiersprache abweichen⁹⁶. Erfundene Imports, Variablen oder Funktionen, die es so nicht gibt oder unter einem anderen Namen aufzufinden sind, führen unweigerlich zu einem Fehler, den der Entwickler oder die Entwicklerin manuell beheben muss. Die Vorschläge müssen damit funktional fehlerfrei sein und zudem hohe Anforderungen erfüllen. Darunter fallen auch die Sicherheitsaspekte, die Kontexttiefe und die Erklärbarkeit und Dokumentation der Vorschläge.

In den letzten Jahren des rasanten Voranschreitens generativer KI hat sich unter den zahlreichen Einsatzgebieten die Code-Completion als hochgradig praxisrelevante und einflussreiche Speerspitze erwiesen⁹⁷. Andere Bereiche der generativen KI sind z.B. die Bildgenerierung, Textgenerierung oder sogar Audio- und Videogenerierung; doch diese sind überwiegend kreativer Natur und werden im professionellen Umfeld nur begrenzt eingesetzt oder sogar vermieden⁹⁸.

Die Code-Completion unterscheidet sich in dieser Hinsicht, da sie konkret in produktive Prozesse eingreift. Sie spart dort Zeit, wo in der Vergangenheit viel Zeit mit der Informationsrecherche verschiedenster Natur verbracht wurde. Es werden durch Code-Completions stattdessen passgenaue Lösungsvorschläge generiert und der Informationsbeschaffungsprozess wird vollständig oder überwiegend umgangen.

Ein Blick über den Tellerrand des heutigen Standes der Technik hinaus, könnte zu der Überzeugung führen die Kompetenzen von Code-Completion-Systemen nicht weiter ausbauen zu müssen. In Zukunft könnten sie nicht nur eine Assistenz darstellen, sondern auch (teil-)autonome Akteure im Entwicklungsprozess werden. Wünschenswerte nächste Meilensteine wären dann z. B. die Übernahme der Testerstellung, Refactoring oder sogar das vollständig autonome Erstellen ganzer Module oder Projekte. Solch eine Weiterentwicklung hätte das Potenzial den gesellschaftlichen Stand in der Softwareentwicklung in ein neues Zeitalter zu befördern.

⁹⁴Vgl. Zhang, M. et al. 2024, S. 297

⁹⁵Vgl. Zhang, M. et al. 2024, S. 306-311

⁹⁶Vgl. Smutny, Bojko 2024, S. 10

⁹⁷Vgl. Xiao et al. 2024, S. 1 f.

⁹⁸Vgl. Yu et al. 2024, S. 1-3, 12

2.3 Marktanforderungen und Bedarfe von Unternehmen an moderne Code-Completion-Systeme

Der Markt ist ständig im Wandel, damit auch seine Anforderungen und die Bedürfnisse seiner Akteure. Zu den Beginnen der Code-Completion stand vor allem die Vermeidung von Syntaxfehlern und die Verbesserung des Komforts beim Schreiben von Code im Vordergrund. Dafür reichten simple Modelle auf statistischer Basis aus. Einfache Vervollständigungen von Methoden und Variablen waren möglich, die Entwicklung dieser war dennoch federführend vom Entwickler abhängig. Lösungen wie *IntelliSense* erfüllten zunächst diese Anforderungen, insofern die Entwickler mit Microsoft *Visual Studio* arbeiteten⁹⁹. Doch immer deutlicher wurde, dass das nur die Spitze des Eisbergs war. Vielmehr geht es darum, einen wirtschaftlichen Erfolg zu erzielen durch eine Produktivitätssteigerung der Entwickler.

Als jedoch klar wurde, dass mit der Transformer-Architektur weitaus mehr möglich ist, als nur einfache Syntaxhilfe, sondern dass sogar komplexe Aufgaben teilautonom gelöst werden können¹⁰⁰, erhob sich die Erwartungshaltung des Marktes drastisch¹⁰¹. Immer mehr Unternehmen haben allerdings auch damit einhergehende Datenschutzbedenken. Viele der leistungsstarken Modelle auf dem Markt sind proprietär oder Cloud-Services¹⁰². Code wird aber in der Regel als vertrauliche Information behandelt, die nicht ohne klare vertragliche Regelungen in die Hände Dritter gegeben werden kann. Als Alternative suchen sich daher viele Unternehmen Modelle, die sie im eigenen Rechenzentrum betreiben können. Daten sollen so nicht abgegeben werden und es kann volle Kontrolle über den Informationsfluss und Einstellungen gewährleistet werden¹⁰³.

Heute wird oben drauf gefordert, dass Code-Completion-Systeme diese fehlerfreien und sinnvollen Vorschläge in kürzester Zeit generieren und den Nutzerinnen und Nutzern demonstrieren. Idealerweise soll das innerhalb von Sekundenbruchteilen geschehen, sodass der Schreibfluss der Entwicklerinnen und Entwickler nicht gestört wird.

Es gibt einige Branchen mit besonderem Anliegen an Datenschutz und Compliance. Zu solchen hochregulierten Branchen gehören z. B. die Banken- und Versicherungsbranche. Ein Datenleck hätte in diesen nicht nur negative Auswirkungen für das Unternehmen, sondern auch für die Kundinnen und Kunden¹⁰⁴. Damit das nicht passiert drängt es nach Nachvollziehbarkeit der Systeme und nach maximaler Systemkontrolle. On-Premise und Private-Cloud Lösungen werden deshalb von diesen Unternehmen bevorzugt, mit denen alle Datenflüsse genau beobachtet und damit transparent werden können¹⁰⁵.

Ein weiterer relevanter Faktor für die Nachfrage an effektiven und effizienten Code-Completion-Systemen ist die aktuelle Lage am Arbeitsmarkt. Die Arbeitsmenge in der Softwareentwicklung

⁹⁹Vgl. Gao 2023, S. 233 f.

¹⁰⁰Vgl. Annepaka, Pakray 2025, S. 2977 f., 3010

¹⁰¹Vgl. Annepaka, Pakray 2025, S. 2971 f.

¹⁰²Vgl. Choi, Chang 2025, S. 1 f.

¹⁰³Vgl. Choi, Chang 2025, S. 2 f.

¹⁰⁴Vgl. Annepaka, Pakray 2025, S. 3001

¹⁰⁵Vgl. Geng 2023, S. 3

steigt ohne Halt, kann aber nicht ausreichend von neuen, qualifizierten Softwareentwicklern und -entwicklerinnen bedient werden¹⁰⁶. Das zwingt die Unternehmen, stattdessen die Mitarbeiterproduktivität statt der Mitarbeiteranzahl zu erhöhen. Solche Systeme tragen maßgeblich dazu bei, dieses Ziel zu erreichen, indem sie Zeit sparen und Anfängern dieses Berufsfeldes in ihrem Lernprozess unterstützen¹⁰⁷.

Zudem ist es wichtig, dass die verwendeten Plattformen einfach wartbar sind und deployed werden können. Das gilt für alle gängigen Infrastrukturen, etwa containerbasierten Infrastrukturen, mit denen eine zentrale Verwaltung und Skalierung erlaubt wird. Wird schließlich das Unternehmen größer, wächst die Anzahl der Nutzerinnen und Nutzer des Systems, so muss auf die wachsende Nachfrage problemlos reagiert werden können.

Insgesamt liegen die Anforderungen des Marktes folglich auf funktionalen Aspekten, wie der Geschwindigkeit und Qualität der Modelle, den rechtlichen und organisatorischen Aspekten der Plattformen, sowie der Integration in Softwareentwicklungsprozesse¹⁰⁸.

In dieser Arbeit bilden die Anforderungen des Marktes die Grundlage für die Entscheidungsfindung und die Analyse von Plattformen, Modellen, dem Ist- sowie dem Soll-Zustand.

2.4 Technologische Trends im Kontext von Code Completion

In diesem Abschnitt wird genauer untersucht, welche technologischen Trends am aktuellen Markt und in der Forschung zu beobachten sind. Es werden dazu wichtige Fortschritte bei den Code-Completion-Modellen selbst betrachtet, darüber hinaus aber auch deren Integration in bestehende Systeme, Personalisierungsmöglichkeiten und weitere Aspekte wie die Ethik. Diese werden im Zusammenhang mit ihren Implikationen für die Entwicklerproduktivität analysiert.

Wie sich aus der Historie der Code-Completion-Systeme ableiten lässt (siehe Abschnitt 2.2 Die Entwicklung von Code-Completion-Systemen im Kontext dialogorientierter KI), haben sich Large Language Models (LLMs), die auf großen Mengen an Code trainiert wurden, als besonders leistungsfähig erwiesen. Sie können heute geschriebenen Code verstehen und Code-Schnipsel, Zeilen oder gar ganze Funktionen vervollständigen.

Was zunächst klar wird, ist der immer weitergehende Trend von proprietären zu Open-Source-Modellen. Durch die Natur von Open-Source-Modellen werden diese von einer breiten Masse an Entwicklern und Entwicklerinnen manipuliert und sogar weiterentwickelt. Damit wird eine kontinuierliche Verbesserung der KI-Kernmodelle sichergestellt und durch Experimente werden neue Wege der Nutzung und Integration gefunden¹⁰⁹. Ein wichtiger Trend ist heute nicht nur die Integration in die IDE allein, sondern in das gesamte Ökosystem der Softwareentwicklung. So wird die Integration mit anderen Entwicklungswerkzeugen sowie -plattformen immer nahtloser und

¹⁰⁶Vgl. Souza et al. 2024, S. 499 f.

¹⁰⁷Vgl. Hassan 2024, S. 15 ff.

¹⁰⁸Vgl. Celia Dolores Benitez, Montes Serrano 2024, S. 284-287

¹⁰⁹Vgl. Vake et al. 2025, S. 1-3

tiefer. Darunter gehört unter anderem die Integration mit *Git* oder anderen Versionierungssystemen, sowie mit Continuous Integration/Continuous Delivery (CI/CD) Pipelines¹¹⁰. Ein gutes Beispiel für ein tiefes Level der Integration mit dem gesamten Ökosystem ist *Zencoder*. Es kann weitere Aufgaben übernehmen als nur die Vervollständigung von Code. Unter diesen Aufgaben fallen das Management von Unit Tests, Reviews und Feedback zum Code (Sicherheit, Qualität, Best-Practices), sowie die Docstring-Generierung und die Websuche. Das Tool ist außerdem tief mit *Jira* integriert, sodass die Codebase direkt mit den Tickets verknüpft werden kann, die Jira verwaltet¹¹¹. Somit kann KI in den gesamten Workflow der Entwickler und Entwicklerinnen integriert werden und positive Einflüsse auf die Entwicklerproduktivität, aber auch auf die Developer Experience (DX) haben.

Genau so individuell wie einzelne Entwickler und Entwicklerinnen, Abteilungen oder ganze Unternehmen in ihrem Codierungsstil sind, so facettenreich sind auch die dementsprechenden Anforderungen an die Code-Completion-Systeme, sich an den Codierungsstil und die Präferenzen der Nutzerinnen und Nutzer anzupassen. Die Modelle lernen zu verstehen, welche Muster und Konventionen in einem Projekt zu finden sind und verändern ihre Vorschläge dementsprechend. So können die Vorschläge nicht nur syntaktisch qualitativ hochwertig sein, sondern auch im stilistischen Sinne konsistent mit dem bestehenden Code sein. Diese Anpassung führt zu einer höheren Akzeptanz der Vorschläge. Um diese Lernfähigkeit zu ermöglichen, bieten verschiedene Tools wie z. B. *Tabnine* das Feature an, ein lokales Training der eingesetzten Modelle auf Basis der eigenen Codebase durchzuführen¹¹².

Durch Technologien wie Retrieval-Augmented Generation (RAG) können darüber hinaus immer besser kontextbezogene Vorschläge generiert werden. Das funktioniert, indem das System das gesamte Projekt analysiert und auf Basis der aktuellen Cursorposition die Relevanz verschiedener Dokumente und Codeabschnitte festlegt¹¹³. Mit der Zeit finden Entwickler und Entwicklerinnen solcher Systeme immer bessere Möglichkeiten, um die Genauigkeit solcher Algorithmen zu erhöhen.

Ein fundamentales Problem, das sich über das gesamte Gebiet der KI zieht, ist die Verzerrung (Bias) in den Modellen. Problematisch daran ist, dass diese zu Diskriminierung, Vorurteilen oder unwissenschaftlichen Ergebnissen führen können. Letzteres ist besonders kritisch im Bereich der Softwareentwicklung¹¹⁴. Um Verzerrungen zu erkennen und einzudämmen, wird zunehmend Forschung betrieben und neue Technologien entwickelt.

Umstritten ist ebenfalls die Frage nach geistigem Eigentum. Da Code-Completion-Systeme auf öffentlich verfügbaren Codedaten trainiert werden, kann nicht ausgeschlossen werden, dass einige vom System generierten Code direkte Kopien von diesen Daten sind¹¹⁵. Es fehlt dabei jede Information zur Lizenzierung und damit auch zur Erlaubnis der Nutzung. Sollte es dazu kommen, dass

¹¹⁰Vgl. Rautiainen 2023, S. 6-13

¹¹¹Vgl. For Good AI Inc. 2025

¹¹²Vgl. tabnine 2025

¹¹³Vgl. Liang et al. 2024, S. 1 ff.

¹¹⁴Vgl. Oyekanle Claudius Oyeniran et al. 2022, S. 115-118

¹¹⁵Vgl. Stalnaker et al. 2025, S. 25 f.

Code ohne eine solche Erlaubnis wiederverwendet wird, kann das ernste rechtliche Konsequenzen mit sich ziehen, was für Unternehmen ein großes Risiko darstellt¹¹⁶. Daher drängen Unternehmen mit großem Interesse nach Tools oder Modellen, die im Vorfeld oder nachträglich solche Risiken vermeiden können. Für dieses Vorhaben werden klare Richtlinien und Best-Practices etabliert, sodass eine rechtliche Absicherung kontrolliert werden kann. Dieses Problem löst beispielsweise *GitHub Copilot* mit seinem *Code Referencing* Feature. Hierbei werden bei Vorschlägen gleich die Quellen angegeben und erwähnt, dass der entsprechende Code aus öffentlich zugänglichen Repositories stammt¹¹⁷.

Beim Betrachten der Entwicklung in der heutigen Zeit, ist ein deutlicher Trend zu „Agentic AI“ (Agentischer KI) erkennbar. Das bedeutet einen Wandel von passiver Unterstützung zu aktiver Mitarbeit und -gestaltung bei komplexen Entwicklungsaufgaben¹¹⁸. Exemplarisch experimentiert *GitHub Copilot* mit einem „Agentenmodus“, in dem das System die Codebase selbstständig analysiert, relevante Dokumente liest, sie editiert und sogar mit dem Terminal und Test interagiert. Ausgehend davon, kann es auf Kompilierfehler und Informationen von Linting-Tools reagieren und den Code entsprechend iterativ anpassen, bis die Aufgabe fehlerfrei abgeschlossen ist. Geplant sind für die Zukunft weitere Funktionalitäten wie unter anderem die Fähigkeit bestimmte Änderungen rückgängig zu machen oder Erweiterungen zu installieren.¹¹⁹

Nicht nur sollen Modelle extern fortlaufend verbessert werden, sondern auch intern durch menschliches Feedback der Nutzerinnen und Nutzer. Deswegen wird heutzutage ein Feedback-Loop eingebaut, sodass die Modelle aus der Projekthistorie und deren Einfluss auf den Code lernen können¹²⁰. Von Nutzerinnen und Nutzern werden dazu Code-Reviews abgegeben und so spezifische Bedürfnisse erfasst.

Zusammenfassend ergibt sich aus der Recherche heutiger Entwicklungen und Trends, dass nicht nur ein passiver Unterstützer, sondern ein aktiver Partner bei der Softwareentwicklung angestrebt wird. Das führt jedoch zu mehr Verantwortung und damit auch zu mehr Aufmerksamkeit und Sorgfalt bei der Entwicklung und Nutzung solcher Systeme.

¹¹⁶Vgl. Stalnaker et al. 2025, S. 3 f., 31–33

¹¹⁷Vgl. Liao 2025

¹¹⁸Vgl. Acharya, Kuppan, Divya 2025, S. 18912

¹¹⁹Vgl. Isidor and the VS Code team 2025

¹²⁰Vgl. Wu et al. 2022, S. 368 f.

3 Relevanz und Zielgruppen von Code-Completion-Systemen

3.1 Relevanz in der heutigen Softwareentwicklung

Besonders in den vergangenen Jahren, mit dem Boom der Transformer-Modelle, hat sich durch den Durchbruch zunächst die Code-Completion als „nice-to-have“ etabliert. Inzwischen ist jedoch klar, dass sie keineswegs mehr nur eine nette Spielerei darstellt, sondern in den Vordergrund der Anforderungen von Unternehmen sowie Entwicklerinnen und Entwicklern gerückt ist. Es wandelt sich regelrecht die Rolle der oder des Entwickelnden selbst — weg vom Tippen und immer mehr zum Orchestrieren der Softwareentwicklung. Von der Code-Completion werden zum Teil schon heute Routineaufgaben übernommen; es wird die Qualität und Effizienz der Softwareentwicklung verbessert und der Entwickler oder die Entwicklerin konzentriert sich mehr auf die kreativen und komplexen Aufgaben. Durch die Automatisierung repetitiver Aufgaben und im Hinblick auf den Fachkräftemangel werden Code-Completion-Systeme daher unentbehrlich. Unternehmen, die diese Technologie nicht einsetzen, könnten voraussichtlich im Wettbewerb des Marktes zurückfallen. Hinzu kommt, dass die Softwareentwicklung mit der Zeit nur komplexer und umfangreicher wird, was für einzelne Entwickler und Entwicklerinnen nicht mehr allein zu bewältigen ist. Für eine einzelne entwickelnde Person ist es nicht länger tragbar, dieses Volumen an Wissen zu beherrschen und bewältigen¹²¹. Code-Completion ist somit ein „must-have“ in modernen Projekten und IT-Unternehmen.

3.2 Potenziale für Unternehmen

Wie bereits im vorangegangenen Abschnitt beschrieben, leisten Code-Completion-Systeme einen entscheidenden Beitrag zur Steigerung der Produktivität und Codequalität, indem sie Routineaufgaben übernehmen und sich an den Trainingsdaten orientieren, wodurch gängige Coding Conventions, sichere Bibliotheken und Best Practices berücksichtigt werden. Insbesondere bei der Einhaltung von Sicherheitsstandards, sowie beim Wissens- und Dokumentationsmanagement bieten solche Systeme einen signifikanten Mehrwert¹²². Gegebenenfalls können nachträglich auch Fine-Tunings des Codes mit spezifischen Secure-Coding-Standards durchgeführt werden. So wird die Codequalität zusätzlich erhöht. Bei komplexen Plattformen ist es auch denkbar, das verwendete Linter-System in die Code-Completion zu integrieren. Das führt zu Echtzeit-Feedback, das in die Vorschläge einfließt¹²³.

Die Entwicklerproduktivität wird außerdem durch eine schnellere Einarbeitung in neue Themen und Technologien erhöht. Das System agiert hierbei als Orientierungshilfe, die in komplexen

¹²¹Vgl. Solanke 2024, S. 234 f., 245

¹²²Vgl. Nettur et al. 2025, S. 6 ff.

¹²³Vgl. Li, Yichen et al. 2024, S. 72 f.

Gebieten einen besonderen Mehrwert bietet. Menschliche Fehler wie Tippfehler, Syntaxfehler oder Leichtsinnfehler werden vom System nicht gemacht, da sich diese nicht oder kaum in den Trainingsdaten wiederfinden. Zeitaufwendige Fehlersuchen und Debuggings werden so massiv reduziert.

Nicht zu vernachlässigen ist auch der Nutzen für das Unternehmen durch erhöhte Mitarbeiterzufriedenheit, die mit einem guten Code-Completion-System einhergeht¹²⁴. Es ist zu erwarten, dass diese Technologie zum Standard in der Branche wird; Unternehmen ohne ein solches System werden einen negativen Eindruck auf potenzielle und bestehende Mitarbeiter hinterlassen.

3.3 Zielgruppen und Einsatzkontexte

Eine Vielzahl an Gruppen kann an der Einführung eines Code-Completion-Systems interessiert sein. Entwickler und Entwicklerinnen, die die tatsächlichen Anwenderinnen und Anwender eines solchen System sind, profitieren wohl am meisten durch die Erleichterung ihrer eigenen Arbeit. Unter ihnen befinden sich auch Junior-Entwickler und -Entwicklerinnen sowie Auszubildende, die mithilfe der Vervollständigungen die Grundlagen des Programmierens erlernen können. Z. B. wird der Syntax und die miteingehende Semantik der Programmiersprache vermittelt, sodass sie schnell eingepreßt werden können. Durch die vielen Vorschläge, die sie im Laufe ihres Einsatzes sehen, erhalten Anfängerinnen und Anfänger auch eine große Hilfe beim Einstieg in die lösungsbasierte Denkweise eines Entwickler oder einer Entwicklerin.

Das DevOps Mindset, das seit den 2010er Jahren immer fester in den Alltag der Entwickler und Entwicklerinnen eingebaut wird, wird ebenfalls von diesen Systemen unterstützt. Das geschieht, indem Automatisierungsskripte und automatisierte Tests schneller und zuverlässiger erstellt werden können. So erhöht sich die Testabdeckung und die Geschwindigkeit des gesamten Deployment-Prozesses.

Wurde ein bestimmter Code committed, so ist es in modernen Unternehmen üblich, dass mindestens eine weitere Person den eingereichten Code überprüft. Das wird auch Code Review genannt. Diejenigen, die den Code überprüfen, könnten mithilfe eines Code-Completion-Systems in ihrer Genauigkeit und ihrem Verständnis des Codes unterstützt werden. So werden möglicherweise auch Fehler oder Schwächen entdeckt, die für das menschliche Auge unscheinbar oder nicht erkennbar sind. Davon profitieren insbesondere Branchen mit hohen Sicherheitsanforderungen, wie in diesem Fall die Sparkassen.

Ein weiterer Einsatzkontext ist die Migration von alten Systemen auf moderne Infrastrukturen und Technologien, sowie deren Refactoring. Als Beispiel kann genannt werden, dass oftmals alte Programmiersprachen wie *COBOL* oder *Fortran* in alten Systemen genutzt werden, die heutzutage nur noch wenige Entwickler und Entwicklerinnen beherrschen und bei welchen die Dokumentation oftmals vollständig fehlt oder minimal eingeführt wurde¹²⁵. Unter den jungen

¹²⁴Vgl. Pettersson 2025, S. 18 f., 47

¹²⁵Vgl. Podduturi 2025, S. 2 ff.

Kollegen und Kolleginnen ist es nicht länger üblich, solche Sprachen überhaupt zu erlernen und die spezifische Ausbildung zu solchen Sprachen könnte ein großer Kostenfaktor für Unternehmen sein. Mit solchen Systemen können jedoch alle in der Lage sein, auch diesen alten Code zu verstehen und in einen neuen Kontext zu übersetzen. Gleichzeitig wird aufgeführt, wie die veralteten Strukturen, Muster etc. heute umgesetzt werden würden.

Das gleiche Prinzip kann auch für modernen Code angewendet werden, jedoch von neuen Teammitgliedern, die sich in die bestehende Codebasis einarbeiten müssen. Sie sparen sich so viel Zeit und sie vermeiden eventuell auftretende Interpretationsfehler, die beim Menschen natürlich vorkommen können. Die Kollegen und Kolleginnen müssen auch weniger Zeit für Onboarding-Prozesse aufwenden. Fachspezifisches und kontextspezifisches Wissen kann einfacher und zeiteffizienter innerhalb des Teams propagiert werden. Darunter z. B. regulatorische und interne Standards, die eingehalten werden sollen.

4 Datenschutz, regulatorische und rechtliche Aspekte

Das Kapitel Datenschutz, regulatorische und rechtliche Aspekte beschäftigt sich mit relevanten Datenschutzaspekten, sowie rechtlichen Vorgaben, die für IT-Dienstleister im Umgang mit personenbezogenen Daten von Bedeutung sind. Es wird erläutert, welche Rahmenbedingungen für herkömmliche IT-Dienstleister gelten und welche spezifischen Anforderungen im Umgang mit Kunden des öffentlichen Sektors gelten.

4.1 Einleitung und Problemaufriss

Das Thema Datenschutz ist zentral bei Code-Completion-Systemen. Das liegt daran, dass Produktivcode direkt extrahiert und an das System übertragen wird, wo er dann weiterverarbeitet wird. Quellcode beinhaltet eine Reihe sensibler Informationen wie z. B. interne Schnittstellen, die Geschäftslogik, Zugangsdaten oder spezifische Konfigurationen und Application Programming Interface (API)-Schlüssel¹²⁶. Es ist von höchster Bedeutung, dass diese Informationen nicht an unbefugte Dritte gelangen, da sonst ernste Sicherheitsprobleme und rechtliche Konsequenzen drohen.

Obwohl Quellcode nicht unbedingt direkt personenbezogene Daten enthält, so kann er trotzdem Rückschlüsse zulassen auf Personen oder Organisationen, die mit dem Code arbeiten¹²⁷. Es muss daher ein Umfeld geschaffen werden, in dem volle Kontrolle und Transparenz über den Datenfluss bestehen, und keine Schwachstellen in der Datenverarbeitung auftreten können. Einige Problemstellungen sind die Blackbox-Problematik von LLMs und Cloud-Computing-Umgebungen sowie die Frage der Datenübertragung an interne oder externe Server.

4.2 Grundlagen des Datenschutzes im Kontext von Softwareentwicklung

Innerhalb der Europäischen Union gilt die Datenschutz-Grundverordnung (DSGVO) als zentrale Rechtsnorm für Datenschutzangelegenheiten. In ihr werden die Grundbedingungen geregelt, unter welchen personenbezogene Daten verarbeitet werden dürfen. Sie regelt darüber hinaus, welche Pflichten bei den Datenverarbeitern bestehen.

Im Kontext dieser Arbeit sind insbesondere folgende Artikel der DSGVO von Bedeutung: Art. 6 (Rechtmäßigkeit der Verarbeitung), Art. 32 (Sicherheit der Verarbeitung) und Art. 35 (Datenschutz-Folgenabschätzung).

¹²⁶Vgl. Qazi 2022, S. 1-5

¹²⁷Vgl. Horlboge et al. 2022, S. 1 ff.

Art. 6 DSGVO — Rechtmäßigkeit der Verarbeitung

Hier werden die Bedingungen festgelegt, unter welchen eine Verarbeitung von personenbezogenen Daten rechtmäßig ist. Die Zulässigkeit wird bedingt durch sechs Rechtsgrundlagen, diese sind:

1. Eine Einwilligung der betroffenen Personen muss vorliegen.
2. Die Verarbeitung ist erforderlich zur Erfüllung eines Vertrages mit der betroffenen Person.
3. Weitere rechtliche Verpflichtungen müssen erfüllt werden.
4. Eine Verarbeitung ist zwingend notwendig, um lebenswichtige Interessen einer natürlichen Person zu schützen.
5. Es liegt ein berechtigtes öffentliches Interesse vor oder die Ausübung öffentlicher Gewalt ist erforderlich.
6. Unter Abwägung kann ein berechtigtes Interesse der verantwortlichen Partei eine Verarbeitung rechtfertigen.

Hier wird das Prinzip der Zweckbindung klar formuliert¹²⁸. Es besagt, dass die Verarbeitung personenbezogener Daten nur dann erfolgen darf, wenn dies zu dem Zweck erfolgt, für den sie ursprünglich erhoben wurden. Möchte man die selben Daten später für einen anderen Zweck verwenden, so muss vorerst geprüft werden, ob der neue Zweck mit dem ursprünglichen vereinbar ist. Für diese Prüfung werden in der DSGVO Kriterien genannt¹²⁹. Besteht nach einer Prüfung keine Vereinbarkeit, so ist eine neue explizite Einwilligung zwingend erforderlich. Jede Verarbeitung, die nicht auf mindestens eine dieser Grundlagen zurückgreift, ist unzulässig und stellt einen Verstoß gegen die DSGVO dar.

Art. 32 DSGVO — Sicherheit der Verarbeitung

Artikel 32 verpflichtet die Verantwortlichen und die Auftragsverarbeiter, geeignete technische sowie organisatorische Maßnahmen zu treffen. Das Ziel dieser Maßnahmen ist es, die Vertraulichkeit von Daten zu schützen, sowie die Integrität und Verfügbarkeit der Daten zu gewährleisten.

So ist die Pseudonymisierung und die Verschlüsselung von personenbezogenen Daten bei der Übertragung, Speicherung und Verarbeitung eine der zutreffenden Maßnahmen. Es gehört dazu auch die sichere Speicherung von Daten im Falle eines Ausfalles der Systeme. Die Wiederherstellung z. B. durch Backup-Systeme kann hier als Maßnahme genannt werden. Es muss außerdem regelmäßig geprüft werden, ob alle Maßnahmen weiterhin den Anforderungen der DSGVO entsprechen und ob sie funktionstüchtig sind.

Es gilt generell der risikobasierte Ansatz, der besagt, dass die durchgeführten Maßnahmen dem Risiko der Verletzung der Rechte und Freiheiten betroffener Personen angemessen sein müssen.

¹²⁸Vgl. Art. 5 Abs. 1 lit. b DSGVO

¹²⁹Vgl. Art. 6 Abs. 4 DSGVO

Das bedeutet vereinfacht, je höher das Risiko ist, desto strenger müssen die eingesetzten Maßnahmen sein.¹³⁰

Art. 35 DSGVO — Datenschutz-Folgenabschätzung

Eine sogenannte Datenschutz-Folgenabschätzung (DSFA) ist erforderlich im Falle eines hohen voraussichtlichen Risikos für die Rechte natürlicher Personen bei der Verarbeitung. Typische Fälle solcher hohen Risiken sind z. B. der Einsatz neuer IT-Technologien, automatisierte Entscheidungssysteme, Profiling oder die systematische Überwachung verschiedener Bereiche. In dieser Arbeit wird mit einer Art Blackbox-System gearbeitet, sodass dies möglicherweise eine DSFA erforderlich macht. Dazu trägt auch bei, dass eventuell ein externer Dienstleister in Erwägung gezogen und sensibler Quellcode verarbeitet wird.

Eine DSFA ist im Grunde eine Risikoanalyse derjenigen Risiken und Folgen, die einer Person durch den Einsatz einer Technologie oder eines Systems entstehen können. Damit ist sie eine Vorabprüfung und ein wichtiges Instrument, um die Einhaltung der DSGVO zu gewährleisten. Die DSFA beinhaltet eine vollständige Beschreibung der geplanten Vorgänge, in der die Daten verarbeitet werden sollen. Dann wird festgestellt, wie notwendig diese Form der Verarbeitung tatsächlich ist und in welchem Verhältnis sie zu den Rechten der betroffenen Personen steht. Es folgt so eine Risikobewertung, die entscheidet, welche geplanten Maßnahmen zur Risikominimierung eingesetzt werden müssen.

Zudem besagt dieser Artikel, dass in diesem Fall ein Datenschutzbeauftragter oder eine Datenschutzbeauftragte einbezogen werden muss. Besteht trotz der implementierten Risikominimierungsmaßnahmen ein hohes Restrisiko, so muss eine Aufsichtsbehörde konsultiert werden.¹³¹

BDSG — Ergänzungen zur DSGVO im deutschen Recht

Im deutschen Geltungsbereich der DSGVO ergänzt sie das Bundesdatenschutzgesetz (BDSG). Sie konkretisiert die Vorgaben der DSGVO, wo sie den Mitgliedstaaten einen Gestaltungsspielraum lässt. Aufgrund dieser Klarstellungen ist sie für deutsche Unternehmen von Bedeutung bei der Datenverarbeitung oder für Datenschutzbeauftragte.

In dieser Arbeit relevant ist die dadurch strengere Regelung der Rolle der Datenschutzbeauftragten. So sind diese nicht nur verantwortlich für die Einhaltung von Datenschutzvorgaben, sondern darüber hinaus auch für die Beratung. Die Datenschutzbeauftragten müssen daher frühzeitig in die Prozesse deutscher Unternehmen einbezogen werden. Im Fall dieser Arbeit, einer sensiblen Code-Completion-Lösung, ist eine solche Einbindung verpflichtend und muss dokumentiert werden.

¹³⁰Vgl. Art. 32 Abs. 1 und 2 DSGVO

¹³¹Vgl. Art. 35 Abs. 1, 2, 3 und 7 DSGVO

Das BDSG regelt erweiterte Rechenschaftspflichten. So sind im Vergleich zur DSGVO die Dokumentation und der Verantwortungsnachweis schärfer formuliert. Das gilt insbesondere für die Datenverarbeitung mit hohem Risikofaktor.

Der Paragraph 26 des BDSG erlaubt im Gegensatz zur DSGVO die Datenverarbeitung ausschließlich, wenn sie erforderlich ist für das Beschäftigungsverhältnis. Damit sind allerdings freiwillige Interessen der Verantwortlichen nicht ausreichend. Laut Absatz 2 des Paragraphen muss eine *freiwillige* Einwilligung der Handlung daher klar belegbar sein¹³². Es darf also keine Zwangslage bestehen; es darf kein Nachteil entstehen, wenn keine Einwilligung gegeben wird.

Im Beschäftigungskontext regelt das BDSG nach § 26 Abs. 2, dass eine erneute freiwillige Einwilligung zwingend erforderlich ist, wenn personenbezogene Daten zu einem anderen Zweck verarbeitet werden sollen als ursprünglich angegeben. Diese Regelung gilt insbesondere für personenbezogene Daten von Beschäftigten und darf nicht pauschal auf andere Szenarien übertragen werden¹³³.

Weiter baut die BDSG die DSGVO durch den Paragraphen 38 aus. Dieser besagt im ersten Absatz, dass ein Datenschutzbeauftragter oder eine Datenschutzbeauftragte erforderlich ist, sobald „in der Regel mindestens 20 Personen ständig mit der automatisierten Verarbeitung personenbezogener Daten beschäftig[t] [sind]“¹³⁴.

Im zweiten Satz des ersten Absatzes wird die freiwillige Möglichkeit eingeräumt, dass auch ohne gesetzliche Verpflichtung eine Datenschutzbeauftragte oder ein Datenschutzbeauftragter benannt werden kann. In diesem Fall haben diese dann die vollen Rechte und Pflichten wie verpflichtend bestellte Datenschutzbeauftragte¹³⁵. Jedoch muss in jedem Fall, wenn eine solche Einbestellung erfolgt, muss die zuständige Datenschutzaufsichtsbehörde darüber verpflichtend informiert werden¹³⁶.

LDSG — Spezifika für Sparkassen als öffentliche Stellen der Bundesländer

Die Bundesländer in Deutschland haben weiter die Möglichkeit die bereits bestehenden Datenschutzgesetze auf EU- und Bundesebene zu ergänzen. Das geschieht in Form der Landesdatenschutzgesetz (LDSG), die individuell für jedes Bundesland gelten.

Da sich die S-MS, in dem diese Arbeit durchgeführt wird, im Bundesland Baden-Württemberg befindet, wird hier ausschließlich auf das LDSG Baden-Württemberg eingegangen. Dieser Gesetzestext gilt für alle öffentlichen Stellen in Baden-Württemberg, darunter auch den Kunden der S-MS, den Sparkassen. Sie gelten als kommunale Einrichtungen und sind damit öffentliche Stellen im Sinne des LDSG¹³⁷. Nach diesem Gesetzestext sind öffentliche Stellen dazu verpflichtet, für

¹³²Vgl. § 26 Abs. 2 BDSG

¹³³Vgl. § 26 Abs. 2 BDSG

¹³⁴Vgl. § 38 Abs. 1 S. 1 BDSG

¹³⁵Vgl. § 38 Abs. 1 S. 2 BDSG

¹³⁶Vgl. § 38 Abs. 1 S. 4 BDSG

¹³⁷Vgl. § 2 Abs. 2 LDSG BW

die Wahrung der Interessen betroffener Personen mit angemessenen und spezifischen Maßnahmen zu sorgen¹³⁸. Um dem gerecht zu werden, müssen die Sparkassen von ihren Dienstleistern die gleichwertige Einhaltung der Datenschutzvorgaben verlangen¹³⁹.

Im dritten Abschnitt der LDSG BW werden Lockerungen gegenüber der DSGVO vorgenommen, die nur für öffentliche Stellen gelten. Demnach besteht für öffentliche Stellen eine beschränkte Pflicht, die betroffenen Personen über die Verarbeitung ihrer Daten zu informieren¹⁴⁰.

Konkret besteht keine Pflicht zur Information. . .

1. . . im Falle öffentlicher Sicherheitsgefährdungen bzw. der Benachteiligung von Bund oder Ländern¹⁴¹,
2. . . zur Verhütung oder Verfolgung von Ordnungswidrigkeiten und Straftaten¹⁴²,
3. . . wenn zivilrechtliche Ansprüche beeinträchtigt werden¹⁴³,
4. . . wenn eine Offenlegung gesetzlich untersagt ist oder dem Schutz der betroffenen Person bzw. der Rechte und Freiheiten Dritter entgegensteht¹⁴⁴, oder
5. . . sollte die Information Forschungszwecke unmöglich machen oder erheblich erschweren¹⁴⁵.

Neben der Einschränkung der Informationspflicht ist auch das Auskunftsrecht eingeschränkt. Das bedeutet, dass Personen nicht wie nach der DSGVO ein Recht haben zu erfahren, welche persönlichen Daten über sie gespeichert werden¹⁴⁶. Stattdessen ist diese Auskunft nicht verpflichtend, wenn dadurch die allgemeine Sicherheit, laufende Strafverfolgungen oder rechtliche Ansprüche gefährdet werden¹⁴⁷. Zuletzt wird in Baden-Württemberg auch das Recht auf Vergessenwerden der DSGVO¹⁴⁸ eingeschränkt. Öffentliche Stellen sind nicht verpflichtet, personenbezogene Daten zu löschen, sondern sind teilweise sogar dazu verpflichtet, diese Daten aufzubewahren. Solche Daten werden aufbewahrt, wenn dies ein Landesarchivgesetz oder andere gesetzliche Dokumentationspflichten vorgeben¹⁴⁹. Sie werden ebenfalls aufbewahrt, wenn das Löschen die betroffene Person schaden würde¹⁵⁰ oder wenn die Daten nichtautomatisiert sind und daher nicht oder nur unter hohem Aufwand gelöscht werden können.¹⁵¹

Nach Paragraph 15 des LDSG BW ist es untersagt mithilfe der Daten eine Überwachung von

¹³⁸§ 3 Abs. 1 LDSG BW

¹³⁹§ 3 Abs. 1 S. 3 Nr. 1, 4, 9 LDSG BW

¹⁴⁰Vgl. § 4 Abs. 1 LDSG BW

¹⁴¹Vgl. § 8 Abs. 1 Nr. 1 LDSG BW

¹⁴²Vgl. § 8 Abs. 1 Nr. 2 LDSG BW

¹⁴³Vgl. § 8 Abs. 1 Nr. 3 LDSG BW

¹⁴⁴Vgl. § 8 Abs. 1 Nr. 4 LDSG BW

¹⁴⁵Vgl. § 8 Abs. 1 Nr. 5 LDSG BW

¹⁴⁶Vgl. Art. 15 DSGVO

¹⁴⁷Vgl. § 9 Abs. 1 S. 1 LDSG BW

¹⁴⁸Vgl. Art. 17 DSGVO

¹⁴⁹Vgl. § 10 Abs. 1 LDSG BW

¹⁵⁰Vgl. § 10 Abs. 2 LDSG BW

¹⁵¹Vgl. § 10 Abs. 3 LDSG BW

Mitarbeitenden durchzuführen, solange kein Strafverdacht vorliegt¹⁵². Es dürfen während der Evaluierungen ohne weiteres keine Leistungsprofile erstellt werden oder in einer anderen Weise das konkrete Arbeitsverhalten der Mitarbeitenden zu extrahieren, analysieren oder bewerten, da dies gegen die Datenschutzvorgaben verstoßen würde¹⁵³. Wenn es dennoch notwendig ist, z. B. Logs von Mitarbeitereffizienz oder Ähnlichem zu erstellen, so muss vorher die Einbindung des Personalrats erfolgen. Es gilt die Mitbestimmungspflicht der Mitarbeitenden¹⁵⁴.

Allerdings darf nicht außer Acht gelassen werden, dass ein Dienstleister wie die S-MS meist öffentliche Stellen (hier Sparkassen) aller Bundesländer bedient und damit im Detail alle bundeslandspezifischen LDSGs beachten muss. Es reicht also nicht aus, wenn ausschließlich das LDSG eines Bundeslandes beachtet wird, da sonst rechtliche Ungereimtheiten entstehen können.

4.3 Datenschutztechnische Risiken bei Code-Completion-Systemen

Beim Arbeiten mit Code-Completion-Systemen sind vielerlei technische und rechtliche Risiken aus Sicht des Datenschutzes anzutreffen. Es ist wichtig, sich über diese Risiken bewusst zu sein und diese mit entsprechenden Maßnahmen zu minimieren, um die rechtliche Einhaltung von Datenschutzvorgaben zu gewährleisten.

Es wird erneut betont, dass es sich bei Quellcode, besonders Produktivcode, um sensible Daten handelt, die verarbeitet werden. Sie müssen unter allen Umständen vor unbefugtem Zugriff geschützt werden. Das Risiko besteht zu jeder Zeit, dass eine unbeabsichtigte Offenlegung solcher Daten an Dritte erfolgt. Wenn externe Dienstleister, Server oder Modelle verwendet werden, besteht hier eine große Gefahr, da die Kontrolle über den Datenfluss verloren wird¹⁵⁵. Ohne klare vertragliche Absprache und angemessene Schutzmaßnahmen, kann dieses Risiko aus Unternehmensperspektive nicht in Kauf genommen werden. Eine Weitergabe an Dritte ist jedoch nicht grundsätzlich dadurch auszuschließen und ist möglicherweise sogar notwendig oder sinnvoll im Kontext von Code-Completion-Systemen. Wenn es möglich ist, sollte darauf Wert gelegt werden, die Weitergabe an Dritte grundlegend zu vermeiden, solange kein triftiger Grund dafür besteht.

Doch auch wenn die Daten vollständig im internen Netzwerk bleiben, bestehen große Risiken. Selbst im lokalen Bereich werden Daten durch verschiedene Speichermethoden und -orte verarbeitet. Es kann daher zu versehentlichen Speicherungen in Caches, Backups, Logs oder anderen Diagnosedaten kommen. Eine Speicherung unter diesen Bedingungen ist womöglich nicht Datenschutzkonform und kann daher einen Verstoß gegen die Datenschutzvorgaben der Europäischen Union, des Bundes oder der Länder darstellen.

Admins oder andere befugte Mitarbeitende könnten solche Daten in Logs o.ä. einsehen, ohne dass

¹⁵²Vgl. § 15 Abs. 5 LDSG BW

¹⁵³Vgl. § 15 Abs. 7 LDSG BW

¹⁵⁴Vgl. § 15 Abs. 8, § 74, § 75 Abs. 4 Nr. 13 LDSG BW

¹⁵⁵Vgl. Sheggam, Zhang, X. 2024, S. 1 f., 6

dies bemerkt wird. Es erfordert somit eine technische als auch organisatorische Schutzmaßnahme gegen einen solchen Zugriff durch Mitarbeitende.

Ähnlich kann es auch zu unbeabsichtigten Übertragungen an Dritte durch unbewusste Telemetrie bei *SaaS*- oder *PaaS*-Lösungen kommen oder durch Trainingsdaten-Feedback von Plattformen im Umgang mit LLMs oder genereller KI¹⁵⁶. Sensible Inhalte können so Teil von Trainingsprozessen anderer Unternehmen werden und damit langfristig in andere Systeme gelangen oder Informationen über das eigene Unternehmen preisgeben. Umgekehrt können auch Code-Fragmente fremder Unternehmen versehentlich ohne Einwilligung und Lizenzierung in das eigene System gelangen. Hierbei handelt es sich um Urheberrechtsverletzungen und es kann zu unklaren Eigentumsverhältnissen kommen. Das stellt ein großes Compliance-Risiko dar und kann zu schwerwiegenden rechtlichen Konsequenzen führen.

Wie im Abschnitt 4.2 ausgeführt, muss akribisch darauf geachtet werden, dass die vorliegende Einwilligung nicht für andere Zwecke als ausdrücklich vereinbart missbraucht wird. Ein solcher Verstoß kann bei Code-Completion-Systemen schnell vorliegen, unter anderem beim Training von Modellen oder deren Einbeziehung in andere Systeme.

4.4 Interne vs. externe Server und deren rechtliche Bewertung

Die Wahl der Serverinfrastruktur ist entscheidend für die datenschutzrechtliche Bewertung des Artefaktes. Es wird ein besonderes Augenmerk auf die Hosting-Variante gelegt, die zur Verarbeitung von Daten verwendet wird. Im Idealfall werden alle Daten gar nicht an externe Server übertragen, wenn es das interne Netzwerk zulässt. So müssen sich keine Gedanken über die sorgfältige Einhaltung von Rechtsgrundlagen über mehrere Vertragspartner hinweg gemacht werden, deren Vertrauen jederzeit in Frage gestellt werden kann.

Lässt sich dennoch nicht vermeiden auch externe Server einzubinden, so gibt es auch hier eine Reihe von Aspekten, die das Sicherheitsniveau bestimmen. Es ist sinnvoll, solche Server bzw. Dienstleister zu wählen, die im gleichen Rechtsraum agieren wie das eigene Unternehmen. Das gilt für alle Ebenen der Datenschutzverordnungen, insbesondere allerdings für die DSGVO, da sie die allgemeinen Grundlangen festlegt.

Mit einem Nicht-EU-Land zu arbeiten, sollte daher, soweit möglich, vermieden werden. Innerhalb der EU-Staaten gestaltet sich die Rechtslage deutlich einfacher und kontrollierter, denn hier sind staatliche Aufsichtsbehörden eingeschaltet, die die Einhaltung der Datenschutzvorgaben überwachen. Es empfiehlt sich außerdem, aufgrund von Latenzzeiten und der Stabilität der politischen Beziehung der Standorte, geographisch nahegelegene Server zu verwenden.

Der Europäische Gerichtshof hat, als gutes Beispiel dafür, am 16.07.2020 das Urteil „Schrems II“ gefällt. Zur Folge hatte das, dass das Privacy Shield-Abkommen zwischen den USA und der

¹⁵⁶Vgl. Ylitalo 2024, S. 40 f.

EU ungültig wurde¹⁵⁷. Seitdem ist es erheblich schwieriger, den Datenaustausch mit den USA zu legitimieren, da in den USA zumeist keine gleichwertigen Datenschutzvorgaben in Kraft sind. Ohne strenge zusätzliche Schutzmaßnahmen wie der End-to-End-Verschlüsselung, Anonymisierung oder andere Transportverschlüsselungen, ist eine Datenübertragung in die USA nicht mehr zulässig¹⁵⁸. Da die USA ein Big Player im Bereich der Cloud-Services und der KI sind, hat dieses Urteil eine weitreichende Folge auf die gesamte Branche.

¹⁵⁷Vgl. Gerichtshof der Europäischen Union 2020

¹⁵⁸Vgl. European Data Protection Board (EDPB) 2020, S. 24 f.

5 Methoden der Projektkonzeptionierung

5.1 Ziel und Bedeutung der Konzeptionierung

Zunächst soll die Frage geklärt werden, warum eine Konzeptionierung notwendig ist und warum sie sauber gestaltet werden sollte.

Damit die Gefahr vermieden wird, am Bedarf der Entwickler und Entwicklerinnen vorbeizuentwickeln, ist es sinnvoll, vor Beginn der Entwicklung ein klares Zielbild zu formulieren. Mit einem solchen Zielbild ist die Vorstellung des Soll-Zustandes gemeint, das die Erwartungen und Anforderungen an das System erfüllt. Es bietet einen Referenzpunkt, an dem sich alle weiteren Schritte in der Arbeit orientieren können. Anschließend muss aus diesem Zielbild eine technische Konzeption als Brücke zwischen Theorie und Praxis abgeleitet werden. Damit ergibt sich der klare Weg hin zum fertigen Produkt, mit einzelnen Schritten und portionierten Zielen.

Es gilt grundsätzlich, dass jede klare Planung mögliche Mehraufwände reduziert, die durch Missverständnisse oder technische Inkompatibilitäten entstehen können. Dazu gehört es auch, durch die Planung eine Flexibilität zu schaffen, sollten später Änderungen von Rahmenbedingungen oder Methoden notwendig werden. So wird sichergestellt, dass die vorhandenen Ressourcen effizient genutzt werden und nicht an redundanten Aufgaben verloren gehen. Das bedeutet, dass jeder Schritt im Plan so nachvollziehbar und transparent wie möglich gestaltet werden sollte. Jede Entscheidung sollte daher klar begründet und dokumentiert werden und Strukturen mit eindeutigen Verantwortlichkeiten geschaffen werden. Das hilft, die Entwicklung, Wartung und anschließende Weiterentwicklung stringent zu halten und außerdem zu steuern.

Damit Aufgaben und Verantwortlichkeiten in angemessenen Zeiträumen und angemessenem Aufwand erledigt werden können, muss eine Komplexitätsreduktion stattfinden. Das bedeutet, dass das Gesamtsystem in handhabbare Komponenten zerlegt wird, sodass diese dann modular in einem iterativen Prozess entwickelt werden können. Agile Methoden bieten sich hier an, um diese einzelnen Schritte zu koordinieren und eine Qualitätssicherung zu gewährleisten.

Um die Akzeptanz der fertigen Lösung zu garantieren, müssen im Zentrum der Planung die Bedürfnisse der Nutzerinnen und Nutzer stehen. In diesem Fall bedeutet das, dass das System spezifisch auf die Anforderungen von Entwicklern und Entwicklerinnen abgestimmt werden muss. Damit diese auch tatsächlich erfüllt werden, müssen die Entwickler und Entwicklerinnen in den Planungsprozess einbezogen werden und nach Möglichkeiten gesucht werden, wie diese Feedback zum aktuellen Stand der Implementierung geben können.

In dieser Arbeit befindet sich bereits eine Lösung, die diese Anforderungen nicht erfüllt. Daher muss zusätzlich geprüft werden, wo konkret die Schwächen der Lösung liegen und nach Feedback der Nutzer und Nutzerinnen gesucht werden. Darüber hinaus ist es auch wichtig, die Planung unter Berücksichtigung verschiedener Perspektiven zu gestalten. Beispiele solcher Perspektiven sind die IT- und Datensicherheits-, die DX-, sowie die Entwicklungsperspektive.

Genauer werden nun die potenziellen Gefahren einer unzureichenden Planung beschrieben, um die Relevanz der Konzeptionierung zu unterstreichen.

Niedrige Nutzerakzeptanz

Die Nutzer bzw. die Nutzerinnen bestimmen über den Erfolg des Systems. Wenn es dazu kommt, dass die Lösung nicht nach ihren Bedürfnissen entwickelt wird, so könnte diese abgelehnt werden. Das führt zum Versagen des Projektes und damit zu enormer Ressourcenverschwendung sowie Frustrationspotenzial oder sogar Imageschäden, wenn die Enttäuschung auf die Organisation übertragen wird.

Fehlfokus bei der Entwicklung

Wurden Anforderungen und Ziele nicht klar definiert oder schlecht kommuniziert, so kann es dazu kommen, dass Features entwickelt werden, die nicht den Bedürfnissen der Nutzer und Nutzerinnen entsprechen. Das führt zu einer niedrigen Nutzerakzeptanz.

Ressourcenverschwendung

Ressourcen wie Zeit, Geld und Arbeitszeit können durch mangelnde Planung und Priorisierung redundant eingesetzt werden, was zu unnötigen Kosten führt.

Technische Inkonsistenzen

Die Gefahr von Inkompatibilitäten wird befeuert durch unklare technische Vorgaben sowie der schlechten Abstimmung zwischen Arbeitsgruppen oder gar der vollständig isolierten Entwicklung. Das führt zu Mehraufwänden, die behoben werden müssen und damit zu Ressourcenverschwendung.

Projektverzögerungen

Wird unsauber konzeptioniert, können unrealistische Zeitpläne entstehen. Das geschieht entweder durch eine unterschätzte Komplexität der Aufgaben oder durch eine unzureichende Risikobetrachtung. So können zeitliche Engpässe entstehen, die zu verzögerten Abgaben oder zu verminderter Qualität führen.

Fehlende Vergleichbarkeit

Im Falle einer nicht ausreichend detaillierten Definition der Anforderungen und Ziele, fehlt später die Grundlage für eine Messung des Projekterfolges. Dadurch kann das Projekt nicht mit anderen Lösungen verglichen werden. Daher sind konsistente Metriken und Standards notwendig; ansonsten ist eine objektive Bewertung nicht möglich und es können keine Lernprozesse für zukünftige Projekte stattfinden.

Sicherheits- und Datenschutzlücken

Eine oberflächliche Planung könnte einige Sicherheitslücken übersehen, woraufhin nicht auf diese im Vorfeld reagiert werden kann. Das führt zu Missbrauch von sensiblen Daten oder zur Gefährdung des Systems durch externe Angriffe.

Entscheidungslücken

Durch unzureichende Planung kommt es zu unklaren Verantwortlichkeiten oder zum Über-

blicksverlust über das gesamte Projekt. Dadurch können wichtige Fragen verloren gehen und damit nicht rechtzeitig beantwortet werden. So entstehen weitere Unklarheiten sowie Kontrollverlust im Projekt.

Fehlende Wiederverwendbarkeit

Damit das Projekt nicht nur auf kurze Sicht, sondern auch auf lange Sicht nützlich ist, muss eine gute Modularisierung und Dokumentation sichergestellt werden. Ist das nicht der Fall, so lässt sich das Projekt möglicherweise kaum oder gar nicht in zukünftigen Vorhaben einsetzen.

Die Konzeption in dieser Arbeit hilft dabei, die identifizierten Anforderungen der Nutzer und Nutzerinnen in ein konkretes technisches Lösungskonzept zu übersetzen. Es wird dazu eine Schwachpunktanalyse durchgeführt und die Limitierungen der bestehenden Refact.ai-Lösung adressiert und gezielt Verbesserungspunkte herausgearbeitet. Mithilfe der Konzeptionierung kann außerdem eine Grundlage für die Evaluierbarkeit der Lösung geschaffen werden. Nur mit einem klar definierten Zielsystem können auch systematische Bewertungen stattfinden. Transparenz ist dabei ein wichtiger Aspekt, der eine Grundlage für die wissenschaftliche Nachvollziehbarkeit schafft.

Das Ziel der Verknüpfung von Theorie und Praxis wird durch die Konzeptionierung greifbar gemacht. Es stellt sicher, dass die theoretischen Erkenntnisse nachvollziehbar und sinnvoll in die praktische Umsetzung einfließen können. Iterativ soll durch die Konzeptionierung Feedback in das Design des Projektes einfließen, um schrittweise die gegebenen Ziele zu erreichen. Dazu wird eine gute Skalierbarkeit, Wartbarkeit sowie Modularität der Lösung angestrebt durch eine entsprechende Vorbereitung.

Zuletzt sind auch die unternehmensspezifischen Anforderungen anhand einer klaren Planung zu berücksichtigen. Insbesondere die strengen Datenschutzvorgaben der S-MS müssen in der Konzeptionierung berücksichtigt werden.

Anschließend lässt sich festhalten, dass eine saubere Konzeptionierung als Bindeglied zwischen der Theorie und der konkreten Umsetzung im Projekt dient. Sie hilft dabei technische, methodische und organisatorische Entscheidungen fundiert treffen zu können. In dieser Arbeit wird eine Konzeptionierung durchgeführt, um die Komplexität eines Code-Completion-Systems zu bändigen und die Gefahr von Chaos und Missverständnissen zu minimieren. Abgestimmt soll sie an den Besonderheiten der S-MS sein, eine Bewertung und Reflexion ermöglichen und nachhaltige Verbesserungen ermöglichen. Es wird darauf abgezielt, einen Prototypen zu entwickeln, der sich aus dem Prozess heraus ergibt.

5.2 Methodisches Rahmenwerk: Design Science Research (DSR)

In der Wirtschaftsinformatik ist eine wichtige und weit verbreitete Methode die des DSR. Dabei handelt es sich um ein forschungsorientiertes Gestaltungsverfahren, das darauf abzielt, Artefakte wie Systeme, Modelle oder Methoden zu entwickeln, die in der Praxis anwendbar sind. In

dieser Arbeit beinhaltet ist das zu erstellende Artefakt ein Code-Completion-System. Mit DSR wird dieses entwickelt, implementiert und anschließend evaluiert. Demnach handelt es sich um einen typischen Anwendungsfall für DSR, da es sich um eine praktische Lösung handelt, die auf theoretischen Erkenntnissen basiert.

DSR wird von Hevner et al. (2004) als Prozess mit sieben Leitlinien definiert, die den gesamten Forschungsprozess leiten. Diese werden in deren Arbeit „Design Science in Information Systems Research“ wie folgt beschrieben:

1. **Design als Artefakt:** Die Design-Science-Forschung muss ein funktionsfähiges Artefakt in Form eines Konzepts, Modells, einer Methode oder Instanz hervorbringen.
2. **Problemrelevanz:** Ziel der Design-Science-Forschung ist es, technologiegestützte Lösungen für wichtige und relevante betriebliche Probleme zu entwickeln.
3. **Artefaktbewertung:** Der Nutzen, die Qualität und die Wirksamkeit eines Design-Artefakts müssen durch gut durchgeführte Evaluationsmethoden rigoros nachgewiesen werden.
4. **Forschungsbeitrag:** Effektive Design-Science-Forschung muss klare und überprüfbare Beiträge zu Design-Artefakten, Designgrundlagen und/oder Designmethodologien liefern.
5. **Forschungsmethodik:** Design-Science-Forschung stützt sich auf die Anwendung rigoroser Methoden sowohl bei der Konstruktion als auch bei der Evaluation des Artefakts.
6. **Design als Suchprozess:** Die Suche nach einem effektiven Artefakt erfordert den Einsatz geeigneter Mittel, um gewünschte Ziele unter Einhaltung der Gegebenheiten im Problemkontext zu erreichen.
7. **Kommunikation der Forschung:** Design-Science-Forschung muss sowohl für technologieorientierte als auch für managementorientierte Zielgruppen wirksam präsentiert werden.¹⁵⁹.

Weiter kann der DSR-Prozess nach Hevner in drei Phasen unterteilt werden:

1. **Problemidentifikation und Zieldefinition:** Die erste Phase beinhaltet zunächst die Identifikation des Problems, das gelöst werden soll. Anschließend wird daraus ein Ziel definiert, das das Artefakt erreichen soll¹⁶⁰.
2. **Artefaktentwicklung:** In der zweiten Phase wird das Artefakt entwickelt. Dabei werden verschiedene Design- und Evaluationsmethoden eingesetzt, um sicherzustellen, dass das Artefakt den definierten Anforderungen entspricht¹⁶¹.
3. **Evaluation durch Feedback und Metirken:** In der dritten und letzten Phase wird das erschaffene Artefakt evaluiert. Es wird geprüft, ob es den definierten Anforderungen und

¹⁵⁹Vgl. Hevner et al. 2004, S. 83

¹⁶⁰Vgl. Hevner et al. 2004, S. 79

¹⁶¹Vgl. Hevner et al. 2004, S. 77-79

Zielen entspricht. Hierfür werden Metriken und Feedback-Schleifen genutzt.¹⁶².

Das alles wird explizit unter Berücksichtigung der unternehmensspezifischen Anforderungen durchgeführt¹⁶³.

Nachfolgend werden drei Phasen des DSR-Prozesses im Kontext dieser Arbeit definiert, sodass sie in späteren Kapiteln des Praxisteils angewendet werden können.

5.3 Phase 1: Problemanalyse und Zieldefinition

Als erster Schritt muss erklärt werden, warum bei der bestehenden Lösung mit *Refact.ai* ein Problem und damit ein Handlungsbedarf besteht. Es werden zunächst technische Defizite und Limitierungen der Lösung beschrieben, die eine Übereinstimmung mit den Anforderungen der Entwickler und Entwicklerinnen der S-MS verhindern. Dazu ist eine generelle Kommunikation mit diesen notwendig, um zu verstehen, was sie benötigen und was sie von der bestehenden Lösung halten.

Als nächster Schritt folgt eine Erfassung sämtlicher organisatorischer und regulatorischer Anforderungen, die wichtig für die Erstellung des Artefaktes sind; insbesondere den Datenschutz betreffend. Mit diesen beiden Schritten sind genügend Informationen gesammelt, um die Ableitung eines klar definierten Zielbildes zu ermöglichen. Sie dient als Grundlage für die weiteren Phasen des DSR-Prozesses. Damit geprüft werden kann, ob das Zielbild erreicht wurde, muss dieses gekoppelt sein an spezifische Kriterien, die messbar oder bewertbar sind. Das soll in dieser Arbeit durch eine Kombination aus Performance-Tests, Feedback der Nutzer und Nutzerinnen sowie eines Tests mit Programmieraufgaben geschehen.

5.4 Phase 2: Artefaktentwicklung — Konzeption und Prototyping

In der zweiten Phase des DSR-Prozesses wird das Artefakt entwickelt. Es wird dazu eine Testumgebung erstellt, die aus einem abgesonderten Server besteht, auf dem das Backend des Systems installiert und konfiguriert wird. Für die Tester wird ein Prototyp-Plugin erstellt, der ausschließlich mit dem Testserver kommunizieren kann und keinerlei Konfigurationsmöglichkeit für den Nutzer und die Nutzerin bietet. Damit soll sichergestellt werden, dass einheitliche Bedingungen für alle Tester und Testerinnen bestehen. Nur so kann vergleichbares Feedback eingeholt werden.

Wichtig bei der Einrichtung dieser beiden Komponenten ist die Berücksichtigung der Sicherheitsstufen für die Datenübertragung. Produktivcode wird unter der Sicherheitsstufe 3 eingestuft. Damit muss die Umgebung entsprechend konfiguriert werden, um keine internen Richtlinien zu verletzen. Es wird aus dem Grund Produktivcode verwendet, um die Testiterationen möglichst realistisch und nah an der tatsächlichen späteren Umgebung zu gestalten.

¹⁶²Vgl. Hevner et al. 2004, S. 85

¹⁶³Vgl. Hevner et al. 2004, S. 79

Hier wird ebenfalls entschieden, welche technischen Komponenten konkret verwendet werden sollen und weshalb. Darunter beispielsweise die gewählten Modelle und die gewählte Plattform. Da mit vielen Änderungen (Austausch von Komponenten und Modellen) zu rechnen ist, soll konkret so entworfen werden, dass eine hohe Modularität sowie Flexibilität gewährleistet ist. Das spart Aufwand und erhöht die Kompatibilität. Der Prototyp ist außerdem nur sinnvoll, wenn er später in eine Produktivumgebung integriert werden kann. Das erfordert eine gute Skalierbarkeit, Wartbarkeit und Integrationsfähigkeit. Iterativ soll in dieser Phase aus der initialen Lösung der Prototyp entstehen mit kontinuierlicher Verfeinerung und Verbesserung.

5.5 Phase 3: Evaluation mit Fokus auf Nutzerfeedback

Abschließend folgt in der dritten Phase die Evaluation des Ergebnisses der durchgeführten Schritte. Es wird der qualitative Evaluationsprozess konkretisiert, in dem verschiedene Testmethoden eingesetzt werden sollen, wie in Phase 1 beschrieben. Für die Beschaffung von direktem Feedback sollen Fragebögen erstellt werden, die die Tester und Testerinnen nach der Nutzung einer Prototyp-Konfiguration ausfüllen sollen. Damit wird die Nutzerwahrnehmung und -akzeptanz für die aktuelle Einstellung erfasst. Darunter sollen Fragen zur Qualität der Vorschläge, Usability und der Akzeptanz des Systems erstellt werden.

Für die technische Evaluation werden zum einen Performance-Tests erstellt und die einzelnen Konfigurationen miteinander verglichen. Zum anderen werden Programmieraufgaben mit aufsteigender Komplexität erstellt, die zu einem Score führen, der zum Vergleich der Konfigurationen genutzt werden kann.

Dies ergibt verschiedene Kriterien, die eine Gesamtbewertung des Prototypen ermöglichen und die Auswahl der besten Konfiguration ermöglichen. Aus dem Ergebnis der Evaluation können dann potenzielle Verbesserungen abgeleitet werden. Dies ist wiederum der erste Schritt in der nachfolgenden DSR-Iteration.

5.6 Fazit der Konzeptionierung und DSR-Anwendung

Die Konzeptionierung in diesem Kapitel diente dazu, eine fundierte Grundlage für die spätere praktische Umsetzung zu schaffen. Insbesondere wurde dabei sichergestellt, dass alle entwickelten Lösungsideen den Anforderungen der S-MS entsprechen. Zudem wurde sichergestellt, dass technische als auch organisatorische Rahmenbedingungen berücksichtigt werden.

Mithilfe der strukturierten DSR-Anwendung wurden die drei Phasen des DSR-Prozesses auf die Fragestellung dieser Arbeit übertragen. Das dadurch entwickelte Vorgehen erlaubt so eine nachvollziehbare und iterativ-zielgerichtete Gestaltung des geplanten Code-Completion Artefakts.

Zentral ist dabei die Idee der Identifikation bestehender technischer Defizite und Limitierungen, die konzeptionell adressiert und im Prototypen korrigiert werden können. Das Artefakt beinhaltet

eine kontrollierte Testumgebung, sowie die Planung von Evaluationsprozessen. Sie stellen sicher, dass alle Kriterien, sowohl technische als auch entwicklerorientierte, berücksichtigt werden.

Die geschaffene Struktur bietet die Grundlage für die praktische Umsetzung des Lösungsansatzes in den nachfolgenden Praxiskapiteln.

6 Analyseverfahren von Plattformen

6.1 Zielsetzung der Plattformanalyse

Um herauszufinden, ob eine bestehende oder potenzielle Plattform geeignet für den Einsatz im unternehmensspezifischen Kontext, hier der S-MS, ist. Durch Plattformanalysen sollen objektiv nachvollziehbare Entscheidungsgrundlagen geschaffen werden, um die besten Entscheidungen für den späteren Prototypen zu treffen. Sie liefert die analytische Vorarbeit für spätere Phasen im DSR-Zyklus — die Implementierung und Evaluation des Artefakts.

Die Plattformanalyse ist die methodische Grundlage für die Kapitel 9 (Analyse der aktuellen Plattform Refact.ai) und 10 (Analyse alternativer Plattformen). Mit ihr wird sichergestellt, dass keine willkürlichen Entscheidungen getroffen werden, sondern dass ein reproduzierbares Verfahren zur Bewertung der Plattformen angewendet wird.

Anders als bei der Evaluation von LLM-Modellen, wie in Kapitel 7 (Evaluationsverfahren für KI-Modelle) genauer erläutert wird, stehen keinerlei generierte Vervollständigungen selbst im Fokus, sondern die Gesamtfunktionalität der Systemarchitektur. Zentrale Fragestellungen sind unter anderem die Frage, welche Mechanismen zur Bedienung des Systems bestehen, wie leicht die Integration der Plattform in interne Systeme fällt und wie gut weitere konkrete Bewertungskriterien erfüllt werden, die im nachfolgenden Abschnitt 6.2 (Bewertungskriterien für Plattformen) festgelegt sind.

Relevant für die Plattformanalyse ist die Betrachtung der Bewertungskriterien in einem realistischen Unternehmenskontext und nicht ausschließlich in einem akademischen Kontext. So soll die tatsächliche Eignung in der Praxis bewertet werden, nicht die theoretische Eignung.

6.2 Bewertungskriterien für Plattformen

Um eine Bewertung vornehmen zu können, müssen vorher aus den Anforderungen an die Plattformen konkrete Kriterien abgeleitet werden, die dann in der Analyse verwendet werden. Sie dienen dem Zweck sicherzustellen, dass funktionale, rechtliche, organisatorische sowie technische Anforderungen in der Bewertung abgebildet werden.

Es folgt eine Übersicht ausgewählter Kriterien, die in dieser Arbeit von Relevanz sind, in der Tabelle 1 (Bewertungskriterien für Plattformen).

Diese Kriterien sind abgeleitet aus den Anforderungen, die in Kapitel 2 (Marktentwicklung und technologische Trends von Chatbots und Code-Completion-Systemen), Abschnitt 2.3 und 2.4 beschrieben werden. Diese Kriterien allein reichen jedoch noch nicht für eine sinnvolle Bewertung aus; sie müssen zuvor in ein gegenseitiges Verhältnis gesetzt werden.

Kriterium	Beschreibung
Funktionalität	Welche Aufgaben kann die Plattform übernehmen? Gibt es API-Zugänge? Welche Codeformate werden unterstützt?
IDE-Kompatibilität	Lässt sich das System in genutzte IDEs einbinden? Erkennt es Projektstruktur, Cursorposition, Imports etc.?
Datenschutz	Werden Daten intern verarbeitet? Gibt es Zugriffsbeschränkungen? Ist der Betrieb DSGVO-konform möglich?
UX/DX	Wie leicht lässt sich das System bedienen? Können Nutzer und Nutzerinnen es anpassen? Stört es den Workflow?
Erweiterbarkeit	Ist die Architektur modular? Lässt sich die Plattform anpassen oder erweitern?
Integration	Funktioniert die Plattform mit Tools wie Git, CI/CD oder Jira?
Skalierbarkeit	Wie verhält sich das System bei hoher Last oder vielen Nutzern und Nutzerinnen?
Performance	Wie schnell kommt ein Vorschlag zurück? Gibt es Wartezeiten?

Tab. 1: Bewertungskriterien für Plattformen

6.3 Kriterienbasierte Bewertungsverfahren

Mit den vorher definierten Kriterien des letzten Abschnitts wurde eine solide Grundlage geschaffen, Stärken und Schwächen der Plattformen zu identifizieren. Es fehlt noch eine Möglichkeit, wie die Kriterien gewichtet werden, sodass ihre Wertung nicht willkürlich ist und nur auf einzelne Meinungen zurückzuführen ist. Mithilfe von quantifizierbaren Verfahren können Bewertungen vergleichbar und nachvollziehbar gemacht werden. Aus der Summe der gewichteten Kriterien wird ein Gesamtbild der Eignung erzeugt, die dann zu einer Entscheidung verhilft.

Eine sinnvolle Möglichkeit der Gewichtung ist die Nutzwertanalyse anhand eines Scoring-Modells. Hierbei erhält jedes Kriterium eine Gewichtung basierend auf den Prioritäten des Unternehmens und eine Bewertungsskala, meist ein Punktesystem von 1 bis 5. Die Punkte werden zu einer Gesamtpunktzahl aggregiert und die einzelnen Punkte können in einem Diagramm visualisiert werden. Geeignete Darstellungsformen für die Ergebnisse sind beispielsweise Spider Charts oder Heatmaps. Mit diesen fällt die Interpretation der Ergebnisse leichter und es kann besser verglichen werden. Das Scoring-Modell soll in dieser Arbeit Verwendung finden.

6.4 Feature-Mapping

Abseits der rein kriterienbasierten Bewertung liegt es im Interesse des Bewertenden auch die Fülle der Features sowie die Marktposition der Plattformen zu betrachten. Mithilfe dieser Betrachtung kann ein direkter Vergleich mit der bestehenden Plattform im Unternehmen, hier Refact.ai,

stattfinden.

Dieses Verfahren wird mithilfe von Feature-Mapping durchgeführt, das ein Abgleich der angebotenen, mit den benötigten Funktionalitäten ist. Abseits der Funktionen, die essenziell benötigt werden, können auch solche identifiziert werden, die überflüssig sind bzw. nicht benötigt werden. Damit kann entschieden werden, ob vor der Implementierung der Plattform Anpassungen fällig sind oder ob sie direkt einsetzbar ist.

7 Evaluationsverfahren für KI-Modelle

Hier werden die theoretischen Grundlagen für die Evaluationsverfahren von LLM-Modellen, spezifisch die Code-Completion Aufgabe, gegeben. Es wird darauf eingegangen, was die konkrete Zielsetzung der Evaluationen ist, auf was eingegangen werden soll und wie konkret die Erfüllung der Anforderungen gemessen werden soll.

7.1 Zielsetzung der Evaluationsverfahren

In diesem Kapitel wird darauf abgezielt, anhand von Evaluationsverfahren die Bewertung von LLM-Modellen für die Code-Completion Aufgabe systematisch zu fundieren. Sie sollen so hinsichtlich ihrer Eignung, ihrem Potential als Hilfswerkzeug für Softwareentwicklerinnen und -entwickler, sowie ihrer Erfüllung weiterer Anforderungen bewertet werden. Zur Fundierung der Bewertung werden die Dimensionen und Methoden der Evaluation festgelegt und erläutert, sodass sie in der Praxis anwendbar sind. Anschließend wird darauf eingegangen, wie die Ergebnisse gewichtet werden und welche Kombinationen gut oder schlecht abschneiden.

7.2 Evaluationsdimensionen

Damit überhaupt eine Bewertung stattfinden kann, muss davor festgelegt werden, welche Kriterien eine Rolle spielen und welcher Natur sie sind. In der Tabelle 2 (Evaluationsdimensionen für die Bewertung von KI-Modellen zur Code Completion) werden die in dieser Arbeit festgelegten Dimensionen für die Code-Completion LLM-Modelle dargestellt.

7.3 Evaluationsmethoden für KI-Modelle

Es wird in dieser Arbeit ein Zusammenspiel mehrerer Evaluationsverfahren für die LLM-Modelle betrachtet. Diese sind:

A) Qualitatives Nutzerfeedback:

Es soll eine Gruppe von Testern des Artefakts gebildet werden, die dieses nutzen und anschließend einen Fragebogen für jede Test-Iteration ausfüllen. So können subjektive Eindrücke über das Artefakt gesammelt werden, wie „Wie hilfreich war die Vervollständigung?“. Als Antwortmöglichkeiten werden sowohl Skalen von 1-5, als auch offene Textfelder verwendet, um detaillierte Rückmeldungen zu erhalten

B) Taskbasierte Evaluation:

Bei dieser Methode sollen die Modelle in einem Programmieraufgaben-Wettbewerb gegeneinander antreten. Die Aufgaben werden aufsteigend schwerer bzw. komplexer. Wenn die Modelle eine

Evaluationsdimension	Aspekte / Unterkriterien
Vorschlagsqualität	› Stimmt das Ergebnis technisch? › Passt der Vorschlag zur Aufgabe, oder geht er am Ziel vorbei? › Wird der Code kürzer, klarer oder schneller verständlich?
Kontextualisierung	Wie gut erkennt das System Zusammenhänge im Projekt? Dazu zählen etwa benachbarter Code, Importpfade oder Dateistruktur.
Anpassungsfähigkeit	Lässt sich das Modell spürbar anpassen, wie Stilvorgaben, projektinterne Namenskonventionen oder Nutzerpräferenzen?
Sicherheitsaspekte	› Fantasiert das Modell Dinge, die es nicht wissen kann? › Gibt es Verzerrungen, z. B. durch Trainingsdaten? › Produziert es Code, der falsch, gefährlich oder schlampig ist? › Trägt es bei zu Misstrauen, Fehlern oder Blackbox-Gefühl?
Performance	› Wie lange dauert es, bis etwas erscheint? › Rechnet das Modell effizient oder frisst es Ressourcen?

Tab. 2: Evaluationsdimensionen für die Bewertung von KI-Modellen zur Code Completion

Aufgabe fehlerfrei gelöst haben, werden entsprechend Punkte vergeben. Am Ende kann so die Gesamtpunktzahl verglichen werden. Zudem können weitere Metriken wie der Zeitaufwand, die Fehlerquote und die subjektive Qualität der Lösung mit einbezogen werden.

C) Benchmarkdaten und Metriken:

Diese Methode kann hilfreich sein, um die Modelle direkt zu vergleichen, allerdings muss die Gefahr im Bewusstsein gehalten werden, dass Modelle spezifisch auf öffentlich verfügbare Benchmarks overfittet werden können. Das führt dazu, dass die Modelle scheinbar besser abschneiden, als sie es in Realsituationen würden. Daher ist es zumindest wichtig, mehrere Benchmarks mit verschiedenen Metriken zu betrachten. Diese Metriken sollten ebenfalls kritisch untersucht werden¹⁶⁴.

7.4 Kombination und Gewichtung der Ergebnisse

Beim Arbeiten mit mehreren Evaluationsverfahren ist es notwendig, diese zu kombinieren und gewichten, um daraus möglichst zutreffende Aussagen zu treffen. Wichtig ist, dass die Gewichtungskriterien klar begründet werden und nicht willkürlich sind. Der finale Score wird ausgehend dieser gewichteten Kriterien in einer Ergebnisdarstellung wie einem Spider-diagramm oder einem aggregierten Score dargestellt. Ausgehend davon kann dann eine Entscheidung für das Projekt

¹⁶⁴Vgl. Reuel et al. 2024, S. 1 ff, 8

getroffen, implementiert und getestet werden.

7.5 Fazit

Zusammengefasst soll durch eine Kombination mehrerer Evaluationsverfahren eine fundierte Entscheidung getroffen werden, welches LLM-Modell am besten für das Artefakt geeignet ist. Eine einzelne Evaluationsmethode könnte Ausreißer im Datensatz nicht erfassen und würde so zu einer Fehlentscheidung führen. So eine Vielfalt der Perspektiven kann sowohl technische als auch menschliche Faktoren gleichzeitig berücksichtigen und schafft eine umfassendere Grundlage für die Entscheidungsfindung. Mithilfe des Design Science Research Ansatzes wird aus dieser Evaluation eine iterative Verbesserung des Artefaktes durchgeführt. Der Prozess wird so lange wiederholt, bis die Anforderungen der Nutzer und Nutzerinnen, sowie die weiteren Anforderungen des Betriebs erfüllt sind. Diese Arbeit gibt damit lediglich den Startschuss und beinhaltet die erste Iteration dieses Prozesses, der allerdings darüber hinaus weiterhin verfolgt wird. Es handelt sich somit also keineswegs um eine einmalige Messung bzw. Durchführung.

8 Marktforschung und Unternehmensanforderungen

8.1 Ziel der Marktforschung

Anhand einer umfassenden Marktforschung soll ein systematischer Überblick über die aktuell verfügbaren Plattformen und Code-Completion-Modelle geschaffen werden. Damit soll eine Grundlage für nachfolgende Analysen und Vergleiche geschaffen werden, um so anhand von festgelegten Unternehmensprioritäten und -anforderungen diejenigen zu identifizieren, die das größten Potenzial für die S-MS haben. Dann kann mit der Produktion des Prototypen begonnen werden.

Die Plattform- und Modellrecherche werden getrennt durchgeführt, die Ergebnisse in Feature-Tabellen zusammengefasst und diese werden dann in diesem Kapitel kurz diskutiert. Für die Suche nach geeigneten Produkten und ihren Eigenschaften wird eine dezentralisierte Recherche durchgeführt, wobei die Daten von verschiedenen Quellen zusammengetragen werden. Durch diese Parallelarbeit und Einbeziehung mehrerer Personen erfolgt die Recherche schneller und umfassender. Außerdem wird die Gefahr von Fehlinformationen durch Dopplungen und mündliche Diskussion reduziert.

Mit diesen Feature-Tabellen wird in den nachfolgenden Kapiteln weiter gearbeitet.

8.2 Vorstellung der analysierten Plattformen

Im Anhang dieser Arbeit befinden sich die Feature-Tabellen für die gefundenen Plattformen, zu denen nun jeweils eine Vorstellung gegeben wird. Es handelt sich konkret um die Tabellen 3 bis 8.

Refact.AI

Refact.AI ist die Plattform, die zur Zeit der Erstellung dieser Arbeit von der S-MS für ihre Code-Completion-Lösung verwendet wird. Sie stellt die Basis für alle weiteren Vergleiche dar und auf sie wird spezifisch eingegangen in der Ist-Analyse im Kapitel 9. Die Hauptkomponenten von Refact.AI sind Open Source, weshalb sie für lokale Implementierungen geeignet ist. Obwohl es möglich ist, diese Plattform in Form eines Plugins in JetBrains IDEs und Visual Studio Code zu integrieren, hat sich dies in der Vergangenheit als schwierig und arbeitsaufwendig herausgestellt. Über die lokale Installation hinaus stellt Refact.AI auch Cloud-Services zur Verfügung für Unternehmen, sodass eine einfache Integration möglich ist auf Kosten der Flexibilität, Kontrolle und Anpassbarkeit.

Amazon CodeWhisperer

Bei Amazon CodeWhisperer handelt es sich um eine Cloud-basierte AWS Lösung, die mithilfe des AWS Toolkits in verschiedene IDEs und andere AWS-Dienste integriert werden kann. Diese Plattform ist proprietär und nicht Open Source. Dadurch ist es nicht möglich, lokale Installationen und Anpassungen vorzunehmen. Dazu besteht eine Abhängigkeit von einem externen Unternehmen. Es ist jedoch tief in das Ökosystem von AWS integriert, weshalb sich die Nutzung mit anderen AWS-Diensten anbietet.

Continue.dev

Continue.dev ist eine Open-Source-Plattform, die aufgrund ihrer MIT-Lizenzierung kostenlos lokal installiert und beliebig angepasst werden kann. Sie wird mithilfe eines Plugins in die IDEs integriert und benötigt kein schwerwiegendes Setup. Sie kommuniziert direkt über eine API mit dem LLM-Backend. Auch Continue.dev bietet eine Cloud-Variante für kleine Teams oder Unternehmen an, sodass die Pflege und Wartung nicht selbst übernommen werden muss. Mit Continue.dev können Inline-Vervollständigungen und Chats mit unterschiedlichen LLMs durchgeführt werden.

Codeium (Windsurf)

Obwohl Codeium ursprünglich ein Open-Source-Projekt war, ist es heute eine proprietäre Plattform und unter dem Namen „Windsurf“ bekannt. Dieses ist nach eigenen Angaben für mehr als 40 IDEs verfügbar und es wird eine eigene IDE mit voller Integration und weiteren Funktionen angeboten. Mit einer kostenlosen Version ist es nur limitiert nutzbar, da jeder Prompt eine bestimmte Anzahl von Credits kostet, die monatlich aufgeladen werden. Viele Features sind zudem nicht aktiviert. Mit bezahlten Plänen werden diese Credits aufgeladen und Features freigeschaltet. Eine Anbindung lokaler LLMs ist nicht möglich.

CodeT5

Es handelt sich um CodeT5 und CodeT5+ genommen um ein eigenes vortrainiertes LLM und nur bedingt um eine Plattform. Dieses Modell kann jedoch direkt lokal in die IDE integriert werden und ist Open Source. Eine zentrale Verwaltung und Wartung wird nicht direkt unterstützt und für eine Server-Installation sind vermutlich zusätzliche Schritte notwendig. Da es sich nicht direkt um eine Plattform handelt, können keine anderen Modelle genutzt werden.

GitHub Copilot

Dieses Produkt ist eine Zusammenarbeit zwischen GitHub, OpenAI, sowie Microsoft und ist eine proprietäre Lösung. Die Integration ist vertieft in Visual Studio Code, einer IDE von Microsoft, und ist als Plugin verfügbar. Auch für andere IDEs ist es verfügbar, jedoch nicht so tief integriert.

Es basiert auf dem Codex-Modell von OpenAI, das wiederum auf GPT-3.5 basiert. Keine anderen Modelle können verwendet werden und eine lokale Installation ist ebenfalls ohne Weiteres ausgeschlossen. Stattdessen muss für die Verwendung ein kostenpflichtiges Abonnement abgeschlossen werden. Ausschließlich die Enterprise-Variante bietet eine kontrollierte Self-Hosting-Option an, die eingeschränkt in ihrer Anpassbarkeit ist. Es bietet sowohl Inline-Vervollständigungen als auch eine Chatfunktion mit anderen Modellen von OpenAI an. Diese Lösung gilt als eine der besten und am weitesten verbreiteten Lösungen auf dem Markt.

Sourcegraph Cody

Als Teil einer ganzen Reihe von Sourcegraph-Produkten ist Cody eine proprietäre Lösung, die zwar keine echten Code-Completion-Funktionen anbietet, jedoch eine Chat-Funktion in die IDE integriert. Damit ist es möglich herkömmliche Chat-LLMs zu verwenden, um den Code zu editieren oder Vorschläge zu erhalten. Mit einem Knopf kann der Code dann angewendet werden. Obwohl zwar viele Modelle ausgewählt werden können, je nach Abonnement-Plan, bleibt die Anpassbarkeit und Kontrolle sehr niedrig, da der Code nicht bearbeitet werden kann und keine eigenen Modelle angebunden werden können. Die Plattform bietet neben der Plugin-Integration auch eine Weboberfläche und CLI-Tools an.

Replit Ghostwriter

Replit bietet mit Ghostwriter eine proprietäre Lösung an, die tief in die eigene Replit-Toolbox integriert ist und ist vollständig Cloud-basiert. Da Replit eine eigene Web-basierte IDE ist, ist eine lokale Installation nicht möglich und es gibt keine Plugins zur Integration in andere IDEs. Die Nutzung ist damit nur über diese eigene Umgebung möglich und nur unter kostenpflichtigem Abonnement nutzbar.

Tabnine

Tabnine bietet erst ab ihrem Enterprise-Plan eine on-premise Lösung an und ist damit mit hohen Kosten eines Abonnements verbunden. Die Software ist jedoch weiterhin nicht Open Source und kann nicht angepasst werden. Die darin verwendeten Modelle sind ebenfalls proprietär und können nicht durch eigene Modelle ersetzt werden.

Visual Studio IntelliCode

Zwar ist Visual Studio IntelliCode eine veraltete und proprietäre Lösung, jedoch kann sie heute weiterhin in Visual Studio und Visual Studio Code verwendet werden. Es nutzt keine moderne LLMs und scheitert bei kreativen oder komplexen Aufgaben, zumal es nicht möglich ist, andere Modelle zu verwenden. Auch der Betrieb auf einem eigenen Server ist nicht möglich und es ist nicht anpassbar.

Zencoder

Auch Zencoder ist eine proprietäre Lösung, die keine Self-Hosting-Option vorsieht. Diese Lösung konzentriert sich auf die Automatisierung von generellen Entwicklungsaufgaben, wie der Erstellung von Unit Tests. Es wird ein Chat Assistent sowie eine Inline-Vervollständigung mit vorgegebenen Modellen angeboten. Kleine Teams können diese Lösung kostenlos nutzen, jedoch fallen für Unternehmen Kosten an, abhängig von der Anzahl der Nutzerinnen und Nutzer, sowie dem gewählten Plan. Es wird als Plugin in die IDEs integriert und ist nicht Open Source.

8.3 Vorstellung der Marktsituation für Code-Completion-Modelle

Unter den verfügbaren Modellen auf dem aktuellen Markt sind sowohl viele proprietäre als auch Open Source Modelle zu finden. Da es eine unabdingbare Voraussetzung ist, dass sowohl die Plattform als auch die Modelle lokal betrieben werden können, fallen daher Cloud-basierte Lösungen aus der Auswahl. Es bleiben so lediglich die Open Source Modelle übrig, die dann auf einem eigenen Backend mit REST-APIs wie vLLM oder TGI betrieben werden können.

Aus der Marktforschung haben sich viele Modelle als potenziell geeignet herausgestellt. Die große Auswahl an Modellen wurde deshalb schrittweise durch bestimmte Kriterien eingegrenzt. Das erste Kriterium ist die Modellgröße. Da die Modelle nur auf einem Server mit begrenzten Ressourcen, nämlich 24GB VRAM, betrieben werden können, kommen nur jene Modelle in Frage, die unter dieser Grenze in Betrieb genommen werden können. Um zu berechnen, wie viel Speicherplatz ein Modell verbraucht, muss die Anzahl der Parameter des Modells mit dem Speicherbedarf des Präzisionsformats multipliziert werden.

$$\text{Speicherbedarf (in Bytes)} = \frac{\text{Anzahl der Parameter} \times \text{Bits pro Parameter}}{8}$$

Es ergibt sich beispielsweise für ein Modell mit zwei Milliarden Parameter für jede Präzision der folgende Speicherbedarf wie in Tabelle 9 dargestellt.

Jedoch ist in diesem Fall der maximale Speicherbedarf gegeben, jedoch die Anzahl der Parameter gesucht. Daher ergibt sich die Formel:

$$\text{Anzahl der Parameter} = \frac{\text{Speicherbedarf (in Bytes)} \times 8}{\text{Bits pro Parameter}}$$

In der Tabelle 10 im Anhang werden nun die maximalen Anzahlen der Parameter für die verschiedenen Präzisionsformate dargestellt, sodass sie jeweils unter 24GB VRAM bleiben.

Es ist zu beachten, dass dies jedoch nur der theoretische Bedarf ist und nicht das volle und tatsächliche Bild wiedergibt. In der Realität wird darüber hinaus noch Speicherplatz für die Kontextlänge, sowie Attention- und Cache-Mechanismen benötigt. Das können je nach Kontextlänge mehrere GB sein.

Bei den Quantisierungsformaten werden neben den Parametern noch zusätzliche Metadaten benötigt, die in jeden Layer bzw. Block gespeichert werden müssen, sodass die Funktionalität erhalten bleibt. Auch das erfordert eine große Menge an Speicherplatz, die nicht in der Tabelle dargestellt ist. Obwohl es gewissen Einstellungsspielraum gibt, ist die Fragmentierung der Speicherblöcke ein Grund dafür, dass der vorhandene VRAM nicht vollständig genutzt werden kann.

Aus diesen und weiteren nicht genannten Gründen kann das theoretische Maximum nicht ohne Weiteres hingenommen werden. Am Ende hilft lediglich der praktische Test durch trial-and-error, ob ein Modell tatsächlich auf dem Server betrieben werden kann. Dennoch kann die Tabelle 10 im Anhang als grobe Orientierungshilfe nützlich sein, besonders bei den unquantisierten Formaten, da diese noch näher an den theoretischen Werten sind.

Auf dem lokalen Testserver konnte ein 24-Milliarden-Parameter-Modell mithilfe von 4-Bit-AWQ-Quantisierung gestartet werden. Es nutzt dabei 21,48 GB VRAM. Obwohl das Modell so praktisch in Betrieb genommen werden kann, bleibt wenig VRAM übrig für die Kontextlänge sowie die Attention- und Cache-Mechanismen. Das kann dazu führen, dass das Modell bei einer Anfrage mit langer Kontextlänge wegen Speicherplatzmangel abstürzt. Ein 22-Milliarden-Parameter-Modell mit gleicher Quantisierung nutzte mit 21,45 GB nahe zu gleich viel Speicherplatz, obwohl es 2 Milliarden Parameter weniger hat. Daran zeigt sich klar, dass jedes Modell individuelle Eigenschaften hat und die tatsächliche Speicherbelastung stark von den theoretischen Werten abweichen kann.

Mit etwa 22 bis 24 Milliarden Parametern ist jedoch klar eine Grenze erreicht, sodass alle größeren Modelle aus der Auswahl fallen.

Um eine faire Vergleichbarkeit zu schaffen, werden in diesem Projekt alle Modelle, auch die kleineren, mit 4-Bit-AWQ-Quantisierung betrieben. In der Praxis kann jedoch die Quantisierung bei kleinen Modellen weggelassen werden; so erhöht sich zusätzlich ihre Leistung. Dafür benötigen sie jedoch mehr Speicherplatz. Es gilt daher abzuwägen, wie hoch die Anfragenlast im Vergleich zur erwünschten Leistung ist.

Ein weiteres Eingrenzungskriterium ist die Leistung der Modelle in öffentlichen Benchmarks. Allerdings ist hier Vorsicht geboten, denn solche Benchmarks können nicht immer die reale Leistungsfähigkeit der Modelle abbilden. Ohne konkrete Beweise muss daher davon ausgegangen werden, dass die aufgelisteten Modelle spezifisch auf Evaluationskriterien wie IFEval, BBH, MATH, HumanEval oder weitere trainiert wurden. In einem kompetitiven Markt muss ein solches Verhalten erwartet werden.

Mit diesen Aspekten im Hinterkopf konnten einige Modelle ausgewählt werden, die ein hohes Potenzial aufweisen.

Qwen/Qwen2.5-Coder-[14B/7B]-Instruct-AWQ

Dieses Modell wird sowohl in der 14B- als auch in der 7B-Variante betrachtet. Es handelt sich um eine eigenständige Qwen-Architektur von Alibaba Cloud, das von GPT-4 Features inspiriert wurde. Es weist ein langes Kontextfenster von 128k Tokens auf und wurde spezifisch auf Code- und Mathematik-Aufgaben trainiert.

casperhausen/mistral-small-24b-instruct-2501-awq

Bei diesem Modell von Mistral AI handelt es sich um eine eigene Mistral-Architektur, die verwandt mit der LLaMA-Architektur ist. Zu seinen Besonderheiten zählen die Sliding Window Attention und die Multi-Query Attention. Im Vergleich zum klassischen Transformer ist es effizienter und schneller. Es ist besonders gut darin viel Leistung bei wenig VRAM-Verbrauch zu liefern. Es handelt sich dabei um kein Modell, das spezifisch für Code-Completion trainiert wurde. Stattdessen handelt es sich um ein Chat-Modell, das in der S-MS bereits für ein anderes Projekt verwendet wird. Es soll genauer betrachtet werden, da so verglichen werden kann, ob sich ein eigenständiges Code-Completion-Modell lohnt oder ob stattdessen das Chat-Modell direkt verwendet werden kann.

TechxGenus/DeepSeek-Coder-V2-Lite-Instruct-AWQ

DeepSeek AI hat ein Mixture-of-Experts-Ansatz verfolgt und ein Modell geschaffen das spezialisiert ist auf Code und ein langes Kontextfenster von 128k Tokens hat. Obwohl dieses 15,7 Milliarden Parameter hat, werden während der Inferenz tatsächlich nur 2,4 Milliarden Parameter aktiv verwendet¹⁶⁵.

solidrust/Codestral-22B-v0.1-hf-AWQ

Codestral ist ein Fork von Mistral AIs eigener Mistral-Basis, jedoch sind die genauen Details nicht bekannt. Dieser ist im Gegensatz zu den Basismodellen von Mistral AI spezifisch auf Code optimiert.

8.4 Unternehmensanforderungen und -prioritäten

Wegen der sensiblen Daten, die in der S-MS verarbeitet werden, steht die Datensicherheit an oberster Stelle. Die Quellcodedaten und eventuell miteingebettete kundenspezifische Daten dürfen keineswegs an Dritte weitergegeben werden. Daraus leitet sich eine Affinität zu on-premise Lösungen gegenüber Cloud-Services ab für dieses Projekt.

Die eingesetzten Modelle und Plattformen müssen bestimmte technische Anforderungen erfüllen. Dazu zählt eine schnelle Reaktionszeit, sowie die Gewährleistung hoher Qualität der Vervollstän-

¹⁶⁵Vgl. DeepSeek-AI et al. 2024, S. 5

digungen. Gleichzeitig sind rechtliche Risiken möglichst minimal zu halten. Es wird spezifisch gesucht nach einer Plattform, die Inline Code-Completion anbietet und mit der es möglich ist eigene Modelle anzubinden. Erwünscht ist auch eine Chat-Funktion. Dadurch können die bereits vorhandenen Chat-Modelle im Betrieb ebenfalls verwendet werden. Außerdem sollte eine ausreichende Kontextlänge gegeben sein, sodass relevante Ausschnitte aus dem gesamten Projekt entnommen werden können.

Gewünscht, aber optional ist die Möglichkeit, dass das ausgewählte Modell auf die eigenen Daten trainiert werden kann.

Organisatorisch sollen die Plattformen und Modelle eine leichte Wartung und Skalierbarkeit anbieten. Eine Integration in bestehende DevOps-Prozesse ist aktuell nicht notwendig.

9 Analyse der aktuellen Plattform Refact.ai

Dieses Kapitel analysiert die Code-Completion-Plattform *Refact.ai* anhand definierter Kriterien. Ziel des Kapitels ist es, die Stärken und insbesondere die Schwächen dieser Plattform zu identifizieren. So soll damit die Grundlage für den späteren Vergleich mit alternativen Plattformen und damit auch für die Konzeptionierung einer neuen Lösung geschaffen werden. Alle Informationen stammen aus öffentlich zugänglichen Quellen und beinhalten den Funktionsumfang, die technische Architektur und Aspekte zu Datenschutz und Integration.

Aus den gesammelten Daten wurde eine Feature-Matrix erstellt, die im Anhang in Tabelle 11 zu finden ist. Im zweiten Teil dieses Kapitels werden konkret die bestehenden Schwächen zusammengefasst, die den Anreiz geschaffen haben, eine alternative Plattform zu finden.

Funktionalität

Refact.ai liefert eine Reihe an Funktionen wie die Code-Vervollständigung, einen integrierten KI-Chat, automatisierte Code-Refactorings, Bugfixing, Dokumentationserstellung sowie Codegenerierung aus Textbeschreibungen. Die Kommunikation läuft über eine eingebaute REST-API und On-Premise-Installationen sind ebenfalls möglich. Darüber hinaus wird eine Agenten-Funktion angeboten, die ganze Entwicklungsaufgaben autonom erarbeiten kann.

IDE-Kompatibilität

Mithilfe eines Plugins kann die Refact.ai-Plattform in viele beliebte IDEs eingebunden werden, darunter auch JetBrains IDEs und Visual Studio Code. Sie erkennt die Projektstruktur, sowie Abstract Syntax Trees — das sind strukturierte, vereinfachte Darstellungen des Codes als Baum. Zum Kontext werden außerdem die Cursorposition und eingefügte Bibliothek-Importe hinzugezogen. Die Kontextanalyse erfolgt über eine Vektor-Datenbank. Sie ermöglicht es relevante Informationen über das gesamte Projekt hinweg effizient in die Vorschläge einzubeziehen.

Datenschutz

Die Plattform sichert den Datenschutz durch drei Privacy-Level. Auf Level 2 können externe Zugriffe auf lokale Dateien und auf externe Dienste erfolgen. So kann eine maximale Funktionalität erreicht werden. Bei Level 1 wird die externe Kommunikation verboten. Nur interne Modelle oder ein Self-Hosted Refact.ai-Server können verwendet werden. Damit bleibt der Code vollständig im eigenen Netzwerk und ist somit DSGVO-konform. Auf der sichersten Stufe 0 wird selbst der Zugriff auf lokale Dateien verboten und die Kommunikation auf lokale Dienste wird ebenfalls verboten. Damit sind zwar die meisten Features inaktiv, darunter auch die Code-Completion, jedoch wird so maximale Sicherheit geschaffen. Sollte dies gewünscht sein, können generell auch

Zugriffsbeschränkungen und Datei-Exklusionen konfiguriert werden, um sensible Bereiche gezielt zu schützen.

User Experience und Developer Experience

Es wird eine unkomplizierte Integration der Plattform mittels eines Plugins in die IDEs angeboten. Die Bedienung wird mit Bedienelementen und dedizierten Fenstern schnell und einfach gehalten. Darunter fallen Funktionen wie ein Chat-Fenster, Kontextmenüs und Autocomplete-Dropdowns. Die Vorschläge erscheinen direkt an der Cursorposition und können per Hotkeys oder Mausklicks abgelehnt oder akzeptiert werden. Durch ein Menü können andere unterstützte Modelle ausgewählt werden (z. B. GPT-4, Claude oder eigene Refact-Modelle); Eine Einbindung eigener Modell-Backends ist nicht vorgesehen. Für die Inline-Code-Completion gibt es Modi wie Explain, Bugfix oder Refactor, die eine zielgerichtete Bearbeitung bzw. Hilfestellung für ausgewählte Codebereiche ermöglichen.

Erweiterbarkeit

Refact.ai bietet über Open-Source-Komponenten, wie etwa die IDE-Plugins, Anpassungs- und Erweiterungsmöglichkeiten an. Die Kernplattform inklusive der verwendeten Modelle und Serverkomponenten ist jedoch proprietär. Es besteht die native Möglichkeit ein Fine-Tuning auf der eigenen Codebasis vorzunehmen. Dies ist jedoch nur in der Enterprise-Variante möglich. Zudem stehen eine API und eine CLI zur Verfügung, um die Plattform auch außerhalb einer IDE zu verwenden und beispielsweise in eigene Workflows einzubinden. Eine direkte, native Einbindung in CI/CD-Pipelines wird zwar nicht angeboten, jedoch könnte dies über API- oder CLI-basierte Lösungen technisch realisiert werden.

Integration in bestehende Tool-Landschaften

Über die Agenten-Funktionalität kann Refact.ai mit weiteren Werkzeugen interagieren, darunter beispielsweise GitHub und GitLab, verschiedene Datenbanken, Docker oder auch der Chrome-Browser. Diese Integrationen erfolgen über sogenannte Tool-Adapter innerhalb des Agenten, welche die Kommunikation mit externen Systemen ermöglichen. Mithilfe dieser Anbindungen lassen sich Build-, Test- und Deployment-Prozesse automatisieren sowie End-to-End-Tests durchführen. Kritische Aktionen, die Änderungen an der Codebasis betreffen, erfordern dabei die explizite Bestätigung durch den Nutzer, um Fehlkonfigurationen oder unbeabsichtigte Änderungen zu vermeiden.

Skalierbarkeit

Durch die On-Premise-Installationsfähigkeit können horizontale Skalierungen, also über mehrere GPUs und Instanzen, vorgenommen werden. Damit eignet sich die Plattform sowohl für kleine Teams als auch für große Unternehmen. Die Kontextverarbeitung erfolgt über eine Vektor-

Datenbank mit semantischer Suche, die relevante Informationen aus dem Projekt effizient bereitstellt. Dieses Vorgehen entspricht in ihrer Methodik den Mechanismen des RAG-Ansatzes.

Performance

Nach Angaben von Refact.ai können durch Caching-Mechanismen sowie ein stringentes Kontext-Management sowohl eine Vervollständigung, als auch ein KI-gestützter Chat nahezu in Echtzeit erfolgen. Die Voraussetzung dafür ist, dass das eingesetzte Modell selbst eine geringe Latenz unterstützt. Die Plattform beschreibt eine hohe Reaktionsgeschwindigkeit auch bei größeren Projekten und Anfragen. Unabhängige Benchmarks zur Bestätigung dieser Performance-Angaben liegen jedoch nicht vor.

Besondere Merkmale

Refact.ai bietet besondere Features an, wie zum Beispiel die Agenten-Funktionalität, die Möglichkeit des Fine-Tunings auf die eigene Codebasis sowie die eigene Modellwahl und -anbindung. Dies unterscheidet die Plattform von vielen anderen Lösungen die es aktuell auf dem Markt gibt.

9.1 Schwächenanalyse der bestehenden Plattform Refact.ai

Aus der Recherche und Analyse der Refact.ai-Plattform ergeben sich einige Schwächen, die aus der Perspektive der S-MS zu Ungereimtheiten führen. Diese Limitationen werden nun im Folgenden zusammengefasst.

Limitierte Anpassbarkeit und Modellabhängigkeit

Ein großes Problem mit der aktuellen Plattform ist die nicht ausreichende Flexibilität, insbesondere bei der Auswahl und Integration von LLM-Modellen. Obwohl einige Modelle direkt von Refact.ai zur Verfügung gestellt werden, bleibt die Anbindung alternativer oder sogar eigener Modelle nativ ausgeschlossen. Es bedarf erheblicher Umwege und Konfigurationsaufwände, um andere Modelle zu integrieren. Durch solche elementaren Eingriffe in die Codebasis der Plattform, erhöht sich der Wartungsaufwand und die Gefahr von Bugs. Es wird eine „Bring Your Own LLM“ Strategie angestrebt, die nicht vorgesehen ist. Daraus resultiert eine Abhängigkeit des Unternehmens von bestimmten Modellanbietern, was die Preisgestaltung, Kontrolle und Datenschutzfragen betrifft. Besonders wegen den hohen Datenschutzanforderungen ist die Plattform auf lange Sicht nicht tragbar.

Eingeschränkte Erweiterbarkeit und Integrationen

Da Teile der Plattform proprietär sind, leidet die Erweiterbarkeit der Plattform. Die On-Premise-Installation sowie eine gewisse Freiheit der Konfiguration sind zwar gegeben, allerdings fehlen

Mechanismen für zukünftige Integrationen in eigene Workflows. Weitere Kontextquellen oder anderweitige Informationen, wie die Ticket-Daten, können nicht ohne Weiteres eingebunden werden. Die Integration in eigene CI/CD-Pipelines oder Dokumentationen ist ebenfalls nur über Umwege realisierbar, was besonders in komplexen Entwicklungsumgebungen eine Schwäche darstellt.

Begrenztes Feintuning und fehlende Modulanpassung

Das Fine-Tuning-Feature der Plattform ist ausschließlich der Enterprise-Version vorbehalten. Die Modellanpassung auf die eigene Codebasis kann somit nur unter hohen Kosten erfolgen. Konkrete Einstellungen dabei sind vom Nutzer direkt nicht anpassbar und an das vorgegebene Modell-Setup gebunden. Eigene Prompt-, Retrieval- und Kontextmanagement-Strategien sind durch diese proprietäre Architektur nur begrenzt möglich, oder sogar gar nicht umsetzbar. Insgesamt erschwert dies eine optimale Hilfestellung durch die KI-Assistenz bei internen Standards und Workflows.

Agenten-Funktionalität ohne vollständige Toolsteuerung

Möchte das Unternehmen die Agenten-Funktionalität nutzen, stößt es in der Praxis schnell an Grenzen- die Tools können nur einfache Operationen innerhalb der Codebasis durchführen. Die Möglichkeit tiefergehende Tools in der Zukunft integrieren zu können, liegt jedoch im Interesse der S-MS. Beispiele für interessante Einsatzgebiete sind das autonome Einbinden externer APIs, sowie die Shell-Nutzung, sowie Anfragen an verfügbare Datenbanksysteme. Für komplexe Entwicklungsaufgaben sind die verfügbaren Funktionalitäten von Refact.ai möglicherweise nicht ausreichend.

Geringe Transparenz und fehlende Open-Source-Basis

Es besteht zwar eine volle Einsicht in die Open-Source-Komponenten, doch die Transparenz der Kernplattform und verwendeten Modelle bleibt trüb, da diese proprietär sind. Anpassungen der Kernlogik sind nicht möglich und über die Entscheidungsprozesse der KI gibt es keine Einsicht. So fehlt die Möglichkeit, detaillierte Dokumentationen für Auditierungen zu erstellen und eine langfristige Nachvollziehbarkeit zu schaffen, was besonders im Finanzsektor und bei öffentlichen Auftraggebern wie den Sparkassen problematisch sein kann.

10 Analyse alternativer Plattformen

Zuvor wurden bereits einige Plattformen recherchiert und jeweils eine Feature-Matrix erstellt. Danach wurde die Ist-Analyse der aktuell genutzten Refact.ai-Plattform durchgeführt. In diesem Kapitel soll zunächst die Auswahl mithilfe einer Sammlung an Ausschlusskriterien eingegrenzt werden. Anschließend sollen die verbleibenden Plattformen genauer betrachtet und mit der Refact.ai-Plattform verglichen werden.

10.1 Präemptiver Ausschluss ungeeigneter Plattformen

Auf den Grundlagen, die im Kapitel 8 (Marktforschung und Unternehmensanforderungen) gelegt wurden, erfolgt nun die Eingrenzung und Suche nach geeigneten Plattformen. Es werden zunächst die Ausschlusskriterien vorgestellt, diese werden dann auf die recherchierten Plattformen angewendet. Dies führt schließlich im nächsten Abschnitt zur genaueren Betrachtung der verbleibenden Plattformen und zur Auswahl der Soll-Plattform.

Ausschlusskriterien

Aus den Anforderungen an den Datenschutz, der freien Auswahl von Modellen und der Anpassbarkeit der gesamten Lösung ergeben sich die folgenden Ausschlusskriterien:

- › Self-Hosting/ vollständig lokale Betriebsmöglichkeit
- › Freie Modellwahl und Unterstützung eigener LLM-Backends
- › Offene oder überprüfbare Plattformbasis (Open Source oder dokumentierte Transparenz)
- › Kompatibilität mit gängigen IDEs (mindestens JetBrains und Visual Studio Code)

Mithilfe dieser Kriterien soll sichergestellt werden, dass keine Ressourcen an Plattformen verschwendet werden, die von Grund auf nicht den Anforderungen entsprechen. Es wird also ausgeschlossen, dass Plattformen in die engere Auswahl kommen, die nicht alle Anforderungen erfüllen.

Bewertung der Plattformen

Die Tabelle 13 bewertet alle recherchierten Plattformen anhand der Ausschlusskriterien und zieht ein Fazit. Sie ist im Anhang zu finden.

Anhand der Tabelle wird deutlich, dass lediglich die Plattform Continue.dev alle, doch die anderen Plattformen mindestens eines der Kriterien nicht erfüllen. Refact.ai und die Enterprise-Version von GitHub Copilot konnten allerdings einige der Kriterien teilweise oder vollständig erfüllen; Keine Plattform jedoch alle.

Damit ist klar, dass im nachfolgenden Abschnitt Continue.dev als einzige Plattform genauer betrachtet und mit Refact.ai direkt verglichen wird.

10.2 Verbesserungspotenziale durch Continue.dev (On-Premise)

Continue.dev ist in der Lage, die bestehenden Schwächen der Refact.ai-Plattform anzusprechen. Zu diesen gehört die Anpassbarkeit, Integrationstiefe, Modellflexibilität und auch die Erweiterbarkeit. Nun wird näher erläutert, wie konkret die Potenziale von Continue.dev zur Verbesserung der Lösung im On-Premise-Betrieb führen.

Hohe Modellflexibilität und Bring-Your-Own-LLM

Da Continue.dev das „Bring Your Own LLM“ Konzept verfolgt, unterstützt es eine Vielzahl an Backends durch eine API-basierte Architektur. Damit können sowohl lokale, als auch cloudbasierte Modelle angebunden und genutzt werden, was maximale Flexibilität und Anpassbarkeit bietet. Welches Modell nun genutzt werden soll und wo es betrieben wird, liegt damit vollständig unter der Kontrolle des Nutzers.

Offene Erweiterbarkeit und Integrationstiefe

Aktuell befindet sich Continue.dev auf dem Weg, die Konfigurationsdatei von .json zu .yaml zu ändern. In jedem Fall kann jedoch frei konfiguriert werden. Zudem kann sogar der gesamte Code an sich angepasst werden, da Continue.dev auf einer Open-Source-Basis beruht. So können auch Hard-Codings umgesetzt und ganze Funktionen entfernt, bearbeitet oder hinzugefügt werden. Es bietet damit die Möglichkeit, die Plattform in interne Werkzeuge wie Ticketsysteme, Datenquellen oder CI/CD-Prozesse mit relativ geringem Arbeitsaufwand anzusprechen.

Tiefe Integrationen, Einstellungen und Automatisierungen sind so im Vergleich zu Refact.ai möglich, bei dem dies z. T. nur sehr eingeschränkt möglich ist.

Feingranulare Anpassbarkeit von Prompts, Regeln und Workflows

Anders als Refact.ai ermöglicht Continue.dev auch die Anpassung der Prompts sowie ihren Regeln und der Assistenzlogik. Spezifische Guidelines und Konventionen zu definieren ist ebenfalls leicht umsetzbar. Das gilt auch für die Automatisierung von wiederkehrenden Aufgaben des Entwicklungsprozesses.

Insgesamt kann Continue.dev in hohem Maße an die Bedürfnisse des Unternehmens angepasst werden.

Agent-Funktionalität mit erweiterter Toolsteuerung

Continue.dev verfügt über eine integrierte Agenten-Funktionalität. Diese ermöglicht es, auch komplexe Entwicklungsaufgaben durchzuführen und übernimmt durch diese interaktive und autonome Nutzung viele Aufgaben, die sonst manuell durchgeführt werden müssten. Es kann dabei Teilschritte selbstständig ausführen und externe Tools sowie eigene Logik nutzen. Aufgaben, wie das Lesen und Schreiben von Dateien, das Suchen im Projekt sowie die Ausführung von Befehlen in der Konsole gehören zu den vielen Möglichkeiten. Damit geht die Plattform weit über die einfache Code-Vervollständigung hinaus und tritt in die Rolle des aktiven Entwicklungsassistenten ein.

Transparenz und Nachhaltigkeit durch Open-Source-Basis

Es ist möglich, den gesamten Code von Continue.dev einzusehen und zu bearbeiten; zudem ist es unter Apache 2.0 lizenziert. Das ist eine wertvolle Eigenschaft, wenn es um die Kontrolle des Datenflusses und um die Auditierbarkeit geht. So können auch bei sicherheitskritischen Umgebungen die regulatorischen Anforderungen erfüllt werden.

Auch die Nachhaltigkeit ist gegeben, da sich das Unternehmen jederzeit dazu entscheiden kann, die Plattform an neue Technologien oder Anforderungen anzupassen.

Fazit zur Verbesserung gegenüber Refact.ai

Insgesamt werden einige zentrale Schwächen von Refact.ai durch den Einsatz von Continue.dev gelöst und die Lösung entspricht damit den Anforderungen des Unternehmens besser. Die Plattform geht über die aktuellen Anforderungen hinaus und bietet eine Grundlage für zukünftige Anpassungen und Erweiterungen. Mit der Plattform kann ein einfacher und sicherer On-Premise-Betrieb in Gang gesetzt werden.

11 Evaluation vielversprechender Modelle

11.1 Zielsetzung der Evaluation

Es soll eine systematische Evaluation erfolgen, sodass eine fundierte Auswahl über den Einsatz geeigneter Code-Completion-Modelle getroffen werden kann. Verschiedene LLM-Modelle unterscheiden sich primär in ihrer Leistungsfähigkeit und ihrem Kontextverständnis. Darüber hinaus unterscheiden sie sich auch in ihrer Eignung für spezifische Unternehmensanforderungen. Fehlentscheidungen sollen vermieden werden, um die Qualität der angewandten Modelle in der Produktivumgebung sicherzustellen. Ein Blick auf öffentliche Benchmarks reicht nicht aus, um die reale Leistung der Modelle zu bewerten. Zum einen liegt das an der Möglichkeit, dass Modelle spezifisch auf die Benchmarks trainiert werden können und sie damit nicht mehr repräsentativ sind. Zum anderen liegt das daran, dass Benchmarks nicht dem Nutzungsszenario in einem Betrieb entsprechen..

Daher ist das Ziel der Evaluation, die Leistungsfähigkeit selbst zu erfassen, insbesondere die Qualität der Vervollständigungen und die Problemlösungskompetenz. Ein weiteres Ziel ist die Erfassung der wahrgenommenen Akzeptanz bei Nutzerinnen und Nutzern der einzelnen Modelle.

Dazu werden zunächst zwei Methoden verwendet. Die erste Methode ist die Aufgabenprüfung, in der Modelle Probleme lösen müssen und für ihre Lösungen Punkte erhalten. Die zweite Methode ist eine Nutzerbefragung durch einen Fragebogen nach jedem Einsatz.

Diese beiden Methoden sind im Sinne des DSR-Ansatzes Teil eines iterativen Prozesses, bei dem die Ergebnisse der Evaluationsverfahren als Grundlage für nachfolgende Entwicklungsphasen dienen.

11.2 Methodik: Prüfung der Modelle durch Code-Challenges

In der ersten Evaluationsmethode wird eine standardisierte Aufgabenprüfung durchgeführt. Alle Modelle müssen die gleichen Aufgaben zu den Programmiersprachen Java, Python und TypeScript lösen. Es gibt aufsteigend fünf Schwierigkeitsgrade, die, entsprechend ihrem Level, eine bestimmte Punktzahl bei korrekter Lösung einbringen.

Dazu werden die Aufgaben nacheinander als Aufgabenstellung an die Modelle übermittelt. Ihre Antworten werden dann mit Unit-Tests auf ihre funktionale Richtigkeit getestet. Schlägt der Test fehl, so werden keine Punkte vergeben; bei Erfolg wird die Punktzahl dem Gesamtergebnis hinzugefügt.

Am Ende werden die Gesamtpunktzahlen der einzelnen Modelle miteinander verglichen. So soll eine realistische Einschätzung der Modellfähigkeiten ermöglicht werden.

11.3 Methodik: Nutzerfeedback zum Praxiseinsatz

Um unter fairen Rahmenbedingungen Feedback zu allen Modellen zu erhalten, werden die Namen der Modelle den Testerinnen und Testern nicht bekannt gegeben. Sie erhalten einen Prototypen des Continue.dev Systems, das von ihnen über den gesamten Testzeitraum genutzt werden soll. Im Hintergrund wird in festgelegten Intervallen das betriebene Modell auf dem Backendserver gewechselt. Zum Schluss jeder Iteration, sollen die Nutzerinnen und Nutzer am Ende jeder Iteration einen Fragebogen zu ihren Erfahrungen ausfüllen.

Im geplanten Fragebogen sollen nur wenige Fragen gestellt werden, um eine hohe Rücklaufquote zu gewährleisten. Mögliche Fragen könnten wie folgt aussehen:

- › Wie hilfreich waren die Vorschläge des Modells? (Skala 1-5)
- › Wie oft haben Sie die Vorschläge genutzt? (Skala 1-5)
- › Haben die Vorschläge Ihren Workflow unterstützt oder gestört? (Skala 1-5)

Mithilfe des Fragebogens soll die Einschätzung der Nutzerinnen und Nutzer zur Qualität und Nützlichkeit der Vorschläge sowie zur wahrgenommenen Akzeptanz des Modells.

Diese zweite Evaluationsmethode wird aus Zeitgründen nicht im Rahmen dieser Arbeit durchgeführt. Sie dient jedoch als Startpunkt und theoretische Grundlage für die spätere Durchführung. Die Befragung kann so ohne zeitlichen Druck durchgeführt werden, was eine genauere Analyse der Ergebnisse ermöglichen soll.

12 Ergebnisse und Entscheidung

12.1 Ergebnisse der Plattformanalyse

Als erster Schritt wurde nach einer umfassenden Sammlung von Informationen eine eingeschränkte Auswahl von Plattformen getroffen. Die Auswahl erfolgte auf Basis der folgenden vier Inklusionskriterien:

- › Self-Hosting möglich
- › Freie Modellwahl
- › Open-Source-Basis
- › IDE-Kompatibilität mit Visual Studio Code und JetBrains IDEs

Durch diese Kriterien wurde klar, dass lediglich eine Plattform als Alternative zu Refact.ai in Frage kommt: *Continue.dev*. Daraufhin wurde diese Plattform genauer betrachtet und es wurden die relevanten Vorteile gegenüber Refact.ai herausgearbeitet.

Beim Vergleich der beiden Plattformen wurde besonders auf die folgenden Punkte geachtet:

- › Anpassbarkeit und Modellflexibilität
- › Datenschutz und Auditierbarkeit
- › Integrationsmöglichkeiten und Erweiterbarkeit

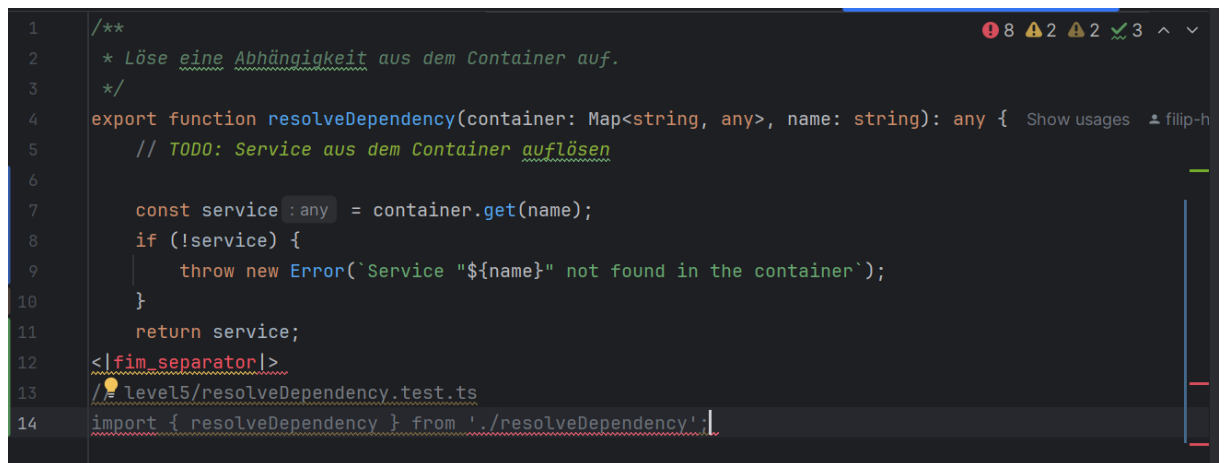
Es wurden genau diese Punkte von der S-MS als relevant betrachtet aufgrund ihrer Geschäftsbeziehung mit öffentlichen Stellen und der damit verbundenen Verantwortung für die Daten. Damit dies glatt verläuft, ist eine hohe Anpassbarkeit und Flexibilität der Plattform notwendig. Die Integrationsmöglichkeit und Erweiterbarkeit sind insbesondere wichtig, sodass die Plattform nachhaltig und effizient gewartet werden kann. In allen drei Punkten hat *Continue.dev* klare Vorteile gegenüber Refact.ai.

12.2 Durchführung und vorläufige Ergebnisse der Modellevaluation

Zur Vorarbeit wurden drei Klausuren für die Programmiersprachen Java, Python und TypeScript erstellt. Es wurden jeweils fünf Schwierigkeitsstufen definiert, die von einfach bis anspruchsvoll reichen. Angefangen bei den Java-Aufgaben stellte sich jedoch schnell heraus, dass die Aufgaben insgesamt zu einfach waren, sodass sich im Verlauf der Evaluation kaum eindeutige Leistungsunterschiede feststellen ließen. Daraufhin wurden die Aufgaben der anderen beiden Sprachen verschärft und anspruchsvoller gestaltet.

Bei der Erstellung der Aufgaben wurde darauf geachtet, dass die Modelle so einfach wie möglich getestet werden können. Das beinhaltet die Atomisierung der Aufgaben, sodass immer nur genau eine Methode bzw. Funktion implementiert werden soll. Zum anderen wurde darauf geachtet, dass die Implementierung genau am Ende oder kurz davor erfolgt. So haben selbst Modelle eine gute Chance, wenn sie nicht in der Lage sind zu verstehen, dass der Code an der aktuellen Cursor-Position eingefügt werden soll. Die Plattform gibt immer den gesamten Inhalt der Dateien an die Modelle weiter, sodass sie zum Teil dachten, dass sie entweder eine neue Aufgabe schreiben oder einen Docstring generieren sollten.

Außerdem fiel auf, dass das Codestral-22B-Modell nicht in der Lage war, korrekt mit dem Prototypen der Plattform zu kommunizieren. Funktionale Tags wurden daher nicht erkannt und wurden daher fälschlich in den Vorschlag für den Nutzer bzw. die Nutzerin übernommen. Ein aufgezeichnetes Beispiel dieses Problems ist in Abbildung 1 zu sehen.



```

1  /**
2   * Löse eine Abhängigkeit aus dem Container auf.
3   */
4  export function resolveDependency(container: Map<string, any>, name: string): any { Show usages  filip-h
5      // TODO: Service aus dem Container auflösen
6
7      const service :any = container.get(name);
8      if (!service) {
9          throw new Error(`Service "${name}" not found in the container`);
10     }
11     return service;
12     <|fim_separator|>
13     // level5/resolveDependency.test.ts
14     import { resolveDependency } from './resolveDependency';

```

Abb. 1: Beispiel für einen Fehler bei Codestral (Screenshot).

In den Abbildungen 2 bis 4 sind die Ergebnisse der Evaluation für die drei Programmiersprachen dargestellt.

Die volle Punktzahl der Java Klausur beträgt 28 Punkte, die Modelle erzielten dabei folgende Ergebnisse:

1. **Codestral 22B** – 28 Punkte
1. **Qwen2.5-Coder-7B** – 28 Punkte
2. **Qwen2.5-Coder-14B** – 24 Punkte
3. **Mistral Small 24B** – 8 Punkte

Die volle Punktzahl der Python Klausur beträgt 43 Punkte, die Modelle erzielten dabei folgende Ergebnisse:

1. **Codestral 22B** – 33 Punkte
1. **Qwen2.5-Coder-14B** – 33 Punkte
2. **Qwen2.5-Coder-7B** – 23 Punkte
3. **Mistral Small 24B** – 5 Punkte

Die volle Punktzahl der TypeScript Klausur beträgt 65 Punkte, die Modelle erzielten dabei folgende Ergebnisse:

1. **Codestral 22B** – 29 Punkte
2. **Qwen2.5-Coder-14B** – 28 Punkte
3. **Qwen2.5-Coder-7B** – 24 Punkte
4. **Mistral Small 24B** – 2 Punkte

Jedoch ist bei diesem Ergebnis zu beachten, dass die Modelle Codestral 22B und Mistral Small 24B externe Nachhilfe erhalten haben. Zwar hatten sie korrekten Code in den Vorschlägen geliefert, jedoch mussten oft nachträglich überschüssige Zeilen durch Inkompatibilität mit dem Prototypen entfernt werden. Zudem musste oft eine nachfolgende geschlossene geschweifte Klammer entfernt werden, da die Prototypen nicht in der Lage waren, ihre Vorschläge an den richtigen Stellen zu orientieren. Gegebenenfalls wurde außerdem ein Kommentar hinzugefügt, um vorgeschlagene Platzhalter-Return-Anweisungen zu verhindern.

12.3 Erkenntnisse und Ableitungen

Aus den vorliegenden Ergebnissen wird deutlich, dass der Einsatz der Continue.dev Plattform in der S-MS sinnvoll ist. Nachdem diese Erkenntnis gewonnen wurde folgte daraus die Entscheidung, einen Prototypen zu entwickeln. Der daraus entstandene Prototyp wurde dann für die Evaluation der Modelle verwendet und war zum Großteil funktionsfähig. Einige Anpassungen wurden jedoch vorgenommen. Außerdem wurden einige Fehler entdeckt, die es noch zu beheben gilt.

In der Evaluation der Modelle schneidet Codestral 22B numerisch am besten ab. Allerdings muss im Hinterkopf behalten werden, dass dieses Modell nicht zuverlässig genutzt werden konnte. Ohne externe Unterstützung wäre die Mehrheit der erzielten Punkte vermutlich nicht erreicht worden. Subjektiv wurde festgestellt, dass Qwen2.5-Coder-14B und Qwen2.5-Coder-7B die souveränsten Vorschläge gemacht haben. Sie erzielten gute Punktzahlen, und ihre Vorschläge mussten nicht nachträglich bearbeitet werden.

Daraus ergibt sich die Erkenntnis, dass Qwen2.5-Coder-14B derzeit die beste Wahl darstellt — zumindest solange Codestral 22B noch nicht sauber in den Evaluationsprozess eingebunden werden kann.

13 Reflexion und Ausblick

In der vorliegenden Arbeit wurde das Ziel verfolgt, ein Code-Completion-System anhand konkreter Unternehmensanforderungen zu konzipieren sowie passende Evaluationsverfahren für Plattformen und Modelle durchzuführen. Auf dieser Grundlage sollte ein Prototyp entwickelt werden, der dann iterativ getestet und verbessert werden sollte. Es handelt sich bei diesem wiederholenden Prozess um einen Design Science Research-Zyklus. Die erste Iteration wurde in dieser Arbeit bearbeitet.

Es folgen nun kritische Reflexionen zur Durchführung und den Ergebnissen. Damit soll eine Perspektive für mögliche Weiterentwicklungen aufgezeigt werden.

13.1 Grenzen der Evaluation

Dank der erfolgreichen Evaluation ausgewählter Plattformen und Modelle konnte ein Continue.dev-basierter Prototyp entwickelt und LLM-Modelle getestet werden. Die Umsetzung der Methodik blieb jedoch in Tiefe und Breite eingeschränkt, da die zur Verfügung stehenden Serverressourcen und das Zeitfenster begrenzt waren. Die Klausuraufgaben für die Modellevaluation geben nur einen Teil der erforderlichen Informationen wieder. Daher konnte zwar kein finales Ergebnis erzielt, aber ein solider Anhaltspunkt gewonnen werden. Allein mit diesen Aufgaben ließ sich der reale Nutzungskontext nur unzureichend abbilden; stattdessen konnten lediglich objektiv beobachtbare Ergebnisse erhoben werden. Wie gut die jeweiligen Modelle in der tatsächlichen Anwendung performen, bleibt somit weiterhin unklar.

Die Gestaltung der Klausuren ist verbesserungswürdig, insbesondere bei den Java- und Python-Aufgaben. Hier ergaben sich durch Erfahrung und Reflexion klare Schwächen und Ungereimtheiten. So waren die Java-Aufgaben zunächst zu einfach gewählt, und rückblickend entsprachen die Python-Aufgaben teilweise nicht den angegebenen Schwierigkeitsgraden. Die erzielte Punktzahl ist demnach in der ersten Iteration kritisch zu betrachten. Eine zukünftige Auswertung könnte zu belastbareren Ergebnissen führen.

13.2 Offene technische Perspektiven

Der Verlauf des Projekts zeigte, dass mit den erarbeiteten Modellen eine grundlegende Inbetriebnahme möglich wäre. Allerdings gibt es noch viele Anhaltspunkte, die zu einer verbesserten Modellleistung und damit zu höherer Vorschlagsqualität führen könnten. In zukünftigen DSR-Iterationen könnte an einer Anbindung von Reranking-Modellen zur Auswahl des besten von mehreren Vorschlägen geforscht werden. Außerdem könnten Embedding-Modelle helfen, die Codebasis semantisch besser zu verstehen, um gezieltere Vorschläge zu generieren.

Besonders in den vergangenen Monaten sind zunehmend sogenannte Reasoning-Modelle aufkommen. Diese sind in der Lage, Denkprozesse zu simulieren, Aufgaben in mehrere Teilschritte zu gliedern und diese nacheinander zu lösen. Obwohl dies in der Regel zu längeren Laufzeiten führt, ist es sinnvoll, die Eignung solcher Modelle im Code-Completion-Kontext zu prüfen. Alternativ könnte damit auch die Chat-Funktion der Continue.dev-Plattform verbessert werden, da dort längere Antwortzeiten weniger kritisch sind.

Ein weiterer Trend, der im Markt zu beobachten ist, ist die Entwicklung der Code-Completions hin zur sogenannten Agentic AI. Erste Experimente können bereits in Aktion gesehen werden, wie beispielsweise bei GitHub Copilot. Dabei sollen selbstständig agierende Agenten ganze Aufgaben eines Entwicklungszyklus übernehmen können. Um mit der fortschreitenden Forschung Schritt zu halten und frühzeitig disruptive Technologien zu identifizieren, ist es sinnvoll, auch solche Trends zeitnah genauer zu betrachten.

13.3 Fazit und Ausblick

Diese Arbeit leistet einen Beitrag zur Integration moderner Ansätze für Code-Completion-Systeme im datensensiblen Unternehmensumfeld. Sie ist als Teil eines langfristigen, iterativen Prozesses zu verstehen, bei dem deutlich wird, dass eine einmalige Durchführung allein nicht ausreicht. Mehrere Iterationen sind notwendig, um das volle Potenzial von Code-Completion-Systemen auszuschöpfen. Die entwickelte Lösung soll daher kontinuierlich weiterentwickelt und um neue Komponenten ergänzt werden – insbesondere im Hinblick auf sich wandelnde Anforderungen, technologische Fortschritte und zukünftige Einsatzszenarien.

Sowohl innerhalb der S-Management Services GmbH als auch im Kontext öffentlicher Forschung und Entwicklung bietet diese Arbeit eine tragfähige theoretische und praktische Grundlage für die schrittweise Erarbeitung leistungsfähiger, anpassbarer und verantwortungsvoll einsetzbarer Systeme im datensensiblen Unternehmensumfeld.

Damit ist ein Ausgangspunkt geschaffen, der nicht als Abschluss, sondern als Einladung zur Weiterarbeit zu verstehen ist.

Anhang

Anhangverzeichnis

Funktion	Refact.AI (Kostenlos)	Amazon CodeWhisperer
Betreiber/Hersteller	Refact, Inc.	Amazon Web Services (AWS)
Open Source / Proprietär / Hybrid	Hybrid (Kern Open Source)	Proprietär
Unterstützte IDEs	VS Code, JetBrains IntelliJ, Neovim, Sublime Text	Visual Studio Code, JetBrains IntelliJ, AWS Cloud9, AWS Lambda Konsole
Unterstützte Programmiersprachen	Sehr viele	Python, Java, JavaScript, TypeScript, C#, Go, Rust, PHP, Ruby, Kotlin, C, C++, Shell, SQL, Scala
Art der Integration	Plugin/Extension	Plugin/Extension (AWS Toolkit)
Unterstützt Self-Hosting / On-Premise?	Ja, vollständig lokal möglich	Nein (Cloud-basiert)
Datenschutz-Funktionen	Lokal = Code/Daten verlassen Maschine nicht	Datenverschlüsselung (at rest und in transit), Opt-out der Datenteilung, Enterprise-Tier ohne Sammlung, IAM-Integration
Integration externer Modelle / Fine-Tunes	Ja, Integration lokaler Modelle möglich	Nicht explizit erwähnt
Skalierbarkeit	Lokal begrenzt	Für Einzelentwickler und Unternehmen (verschiedene Tiers)
Anbindung an DevOps	Git (IDE), weitere potenziell	Integration mit AWS Services (Lambda, S3, DynamoDB)
Besonderheiten	In-File Chat, Code-Aktionen, Unterstützung lokaler LLMs, Agentic-Mode in Entwicklung	Inline-Vorschläge, Integration mit AWS, Code-Sicherheits-Scans, Referenzverfolgung, Bias-Vermeidung
Latenz / Performance	Abhängig von lokaler Hardware/Modell (schnell bei geeignetem Setup)	Realtime-Vorschläge
Lizenzierung und Kostenmodell	Kostenlos für lokale Nutzung, optionale Cloud/Enterprise-Dienste kostenpflichtig	Individuelles Tier kostenlos, Professional Tier kostenpflichtig (Preis nicht öffentlich)

Tab. 3: Vergleich von KI-gestützten Code Completion Tools: Teil 1

Funktion	Continue.dev (Kostenlos)	Codeium (Free Tier)
Betreiber /Hersteller	Continue.dev (Open Source Community)	Windsurf (ehemals Exafunction)
Open Source / Proprietär / Hybrid	Open Source (MIT)	Ursprünglich Open Source, jetzt mit kostenlosem Cloud-Plan
Unterstützte IDEs	VS Code, JetBrains IntelliJ, VS Codium, Neovim (experimentell)	VS Code, JetBrains Suite, Eclipse u.a. (über 40)
Unterstützte Pro- grammiersprachen	Abhängig von integrierten LLMs (breit)	Über 70
Art der Integration	Plugin/Extension	Plugin/Extension
Unterstützt Self-Hosting / On-Premise?	Ja, vollständig lokal möglich	Tendenziell Cloud-basiert im kostenlosen Plan
Datenschutz- Funktionen	Lokal = Code/Daten verlassen Maschine nicht	Potenziell Datenübertragung (Cloud) im kostenlosen Plan
Integration externer Modelle / Fine-Tunes	Ja, Integration verschiedener LLMs (lokal/remote), Fine-Tuning möglich	Im kostenlosen Plan eingeschränkt
Skalierbarkeit	Lokal begrenzt	Einzelnutzer (kostenlos)
Anbindung an DevOps	Git (IDE), weitere potenziell	Primär IDE-Integration (kostenlos)
Besonderheiten	Chat-Interface, Code-Bearbeitung, Multi-Step-Operationen, erweiterbar	Supercomplete, Previews, Linter Integration
Latenz / Performance	Abhängig von lokaler Hardware/LLM (optimiert für Geschwindigkeit)	Kann von Cloud-Anbindung abhängen (schnell bei guter Verbindung)
Lizenzierung und Kostenmodell	Vollständig kostenlos und Open Source (MIT)	Kostenloser Plan für Einzelnutzer (potenziell eingeschränkt), kostenpflichtige Pläne mit mehr Features

Tab. 4: Vergleich von KI-gestützten Code Completion Tools: Teil 2

Funktion	CodeT5	GitHub Copilot
Betreiber /Hersteller	Open Source	GitHub in Zusammenarbeit mit OpenAI und Microsoft
Open Source / Proprietär / Hybrid	Open Source	Proprietär
Unterstützte IDEs	Nicht explizit erwähnt	VS Code, JetBrains IntelliJ, Visual Studio, Neovim, GitHub Codespaces
Unterstützte Programmiersprachen	Java, Python, JavaScript, Ruby, Go, PHP	Sehr viele, darunter Python, JavaScript, TypeScript, Ruby, Go, C#, C++
Art der Integration	Online und Offline Version	Plugin/Extension
Unterstützt Self-Hosting / On-Premise?	Offline Version für Datensicherheit	Nein (nur Enterprise über GitHub Enterprise Cloud)
Datenschutz-Funktionen	Offline Version für Datensicherheit	Richtlinienverwaltung für Unternehmen, Blockieren von Vorschlägen, die mit öffentlichem Code übereinstimmen (Enterprise), Audit-Logs, Opt-out für Telemetrie möglich
Integration externer Modelle / Fine-Tunes	Open Source, daher potenziell möglich	Ja (GitHub Copilot Enterprise bietet Zugriff auf feinabgestimmte, private Modelle)
Skalierbarkeit	Abhängig von der Implementierung	Für Einzelpersonen, Unternehmen und Großunternehmen verfügbar
Anbindung an DevOps	Nicht explizit erwähnt	GitHub-nativ, Azure DevOps
Besonderheiten	Code-zu-Code-Übersetzung, Code-Zusammenfassung, Generierung von Code aus natürlichen Sprachbeschreibungen	Chat-Funktionalität, Agent-Modus (VS Code), Code Review, Next Edit Suggestions, Integration von Drittanbieter-Erweiterungen
Latenz / Performance	Schnell	Generell niedrige Latenz, gelegentliche Probleme möglich
Lizenzierung und Kostenmodell	Kostenlos	Individuell, Business, Enterprise (ab \$10/Monat pro Benutzer), kostenlos für Studenten, Lehrer und Open-Source-Maintainer

Tab. 5: Vergleich von KI-gestützten Code Completion Tools: Teil 3

Funktion	Sourcegraph Cody (Free Tier)	Replit Ghostwriter
Betreiber / Hersteller	Sourcegraph	Replit
Open Source / Proprietär / Hybrid	Bietet kostenlosen Cloud-Plan	Teil der Online-IDE Replit
Unterstützte IDEs	VS Code, JetBrains, Visual Studio (experimentell), Eclipse (experimentell)	Nur in Replit Online-IDE
Unterstützte Programmiersprachen	Viele	Über 50
Art der Integration	Plugin/Extension, Web-Oberfläche, CLI	Integriert in Replit IDE
Unterstützt Self-Hosting / On-Premise?	Tendenziell Cloud-basiert im kostenlosen Plan	Unterstützt Self-Hosting von Modellen
Datenschutz-Funktionen	Potenziell Datenübertragung (Cloud) im kostenlosen Plan	Replit teilt keinen Code aus den AI Mode Features
Integration externer Modelle / Fine-Tunes	Im kostenlosen Plan eingeschränkt	Unterstützt Self-Hosting von Modellen
Skalierbarkeit	Einzelnutzer (kostenlos)	Für Lernende, Hobby-Entwickler und Teams
Anbindung an DevOps	Primär IDE-Integration (kostenlos)	Integrierte Hosting- und Deployment-Funktionen in Replit
Besonderheiten	Chat-basierte Unterstützung, Kontext-Filter, Anpassbare Prompts, DeepCody AI Engine	AI-gestützte Code-Generierung, Debugging, Code-Erklärung, Semantische Suche, Replit Agent
Latenz / Performance	Kann von Cloud-Anbindung abhängen (schnell bei guter Verbindung)	Schnell, insbesondere schneller als GitHub Copilot laut Nutzerberichten
Lizenzierung und Kostenmodell	Kostenloser Plan für Einzelnutzer (potenziell eingeschränkt), kostenpflichtige Enterprise-Pläne	Kostenloser Plan mit eingeschränkten AI-Funktionen, Replit Core (ab \$20/Monat) für volle AI-Funktionen

Tab. 6: Vergleich von KI-gestützten Code Completion Tools: Teil 4

Funktion	Tabnine Pro	Visual Studio IntelliCode
Betreiber / Hersteller	Tabnine	Microsoft
Open Source / Proprietär / Hybrid	Proprietär mit Option zur lokalen Modellierung	Proprietär
Unterstützte IDEs	Über 60, einschließlich VS Code, JetBrains IntelliJ, Visual Studio	Visual Studio, Visual Studio Code
Unterstützte Programmiersprachen	Über 80	C#, XAML, C++, JavaScript, TypeScript, Visual Basic, Java, SQL
Art der Integration	Plugin/Extension	Standardmäßig in Visual Studio enthalten, Extension für VS Code
Unterstützt Self-Hosting / On-Premise?	Ja, über Private Deployments (Single-Tenant SaaS, VPC, On-Premise via Kubernetes, Air-Gapped)	Läuft lokal auf der Maschine
Datenschutz-Funktionen	GDPR-konform, SOC 2 Type 2, ISO 9001, Zero Data Retention (optional), IP-Schutz, Admin-Zensurkontrollen, Audit-Logs	Private Code bleibt privat
Integration externer Modelle / Fine-Tunes	Flexibilität bei Modellen (eigene, Drittanbieter, Open Source), Fine-Tuning optional	Training auf eigenem Code für C# möglich
Skalierbarkeit	Für Einzelentwickler und Unternehmen (Pro und Enterprise Pläne)	Für Einzelentwickler und Teams
Anbindung an DevOps	Git-agnostisch, Integration mit Jira Cloud und Data Center	Azure DevOps
Besonderheiten	Lokale und Cloud-Modelle, Team-Level AI-Modelle, Code Provenance und Attribution, AI-gestützter Chat, AI Agents für verschiedene SDLC-Schritte	AI-gestützte IntelliSense, Whole-Line Autocompletions, Erkennung wiederholter Edits, Quick Actions
Latenz / Performance	Schnell, Hybridmodell (lokal und Cloud) möglich	Schnell, da lokal ausgeführt
Lizenzierung und Kostenmodell	Kostenlose Version für Einzelentwickler, Pro- und Enterprise-Pläne (ab \$12/Monat pro Benutzer)	Kostenlos für alle Visual Studio Benutzer

Tab. 7: Vergleich von KI-gestützten Code Completion Tools: Teil 5

Funktion	Zencoder
Betreiber /Hersteller	Zencoder
Open Source / Proprietär / Hybrid	Proprietär
Unterstützte IDEs	VS Code, JetBrains (bald)
Unterstützte Pro- grammiersprachen	Über 70
Art der Integration	Plugin/Extension
Unterstützt Self-Hosting / On-Premise?	Nicht explizit erwähnt
Datenschutz- Funktionen	Nicht explizit erwähnt
Integration externer Modelle / Fine-Tunes	Nicht explizit erwähnt
Skalierbarkeit	Für Startups bis Enterprise
Anbindung an DevOps	Integration mit DevOps-Tools
Besonderheiten	Repo Grokking, AI Agents (Code Repair, Doku-Generierung, Unit-Test-Generierung), Agentic Chat, Docstring-Generierung, Code Completion, Unit Test Generation, Code Repair
Latenz / Performance	Schnell
Lizenzierung und Kostenmodell	Free Plan, Business Plan (ab \$19/Nutzer/Monat), Enterprise Plan (ab \$39/Nutzer/Monat), Custom Solutions für Großunternehmen

Tab. 8: Vergleich von KI-gestützten Code Completion Tools: Teil 6

Präzisionsformat	Bits pro Parameter	Berechnung (für 2 Mrd. Parameter)	Speicherbedarf
FP32 (Float32)	32	$\frac{2 \times 10^9 \times 32}{8} = 8 \times 10^9$ Bytes	8.00 GB
BF32 (BFloat32)	32	wie FP32	8.00 GB
FP16 (Float16)	16	$\frac{2 \times 10^9 \times 16}{8} = 4 \times 10^9$ Bytes	4.00 GB
BF16 (BFloat16)	16	wie FP16	4.00 GB
INT8 / 8-bit Quant.	8	$\frac{2 \times 10^9 \times 8}{8} = 2 \times 10^9$ Bytes	2.00 GB
4-bit Quantisierung	4	$\frac{2 \times 10^9 \times 4}{8} = 1 \times 10^9$ Bytes	1.00 GB
2-bit Quantisierung	2	$\frac{2 \times 10^9 \times 2}{8} = 0.5 \times 10^9$ Bytes	0.50 GB
1-bit Quantisierung	1	$\frac{2 \times 10^9 \times 1}{8} = 0.25 \times 10^9$ Bytes	0.25 GB

Tab. 9: Beispiel: Speicherbedarf eines 2B-Parameter-Modells bei verschiedenen Präzisionsformaten

Präzisionsformat	Bits pro Parameter	Berechnung (für 24 GB VRAM)	Maximale Anzahl Parameter
FP32 (Float32)	32	$\frac{24 \times 1024^3 \times 8}{32} \approx 6.44 \times 10^9$	ca. 6,44 Mrd.
BF32 (BFloat32)	32	wie FP32	ca. 6,44 Mrd.
FP16 (Float16)	16	$\frac{24 \times 1024^3 \times 8}{16} \approx 12.88 \times 10^9$	ca. 12,88 Mrd.
BF16 (BFloat16)	16	wie FP16	ca. 12,88 Mrd.
INT8 / 8-bit Quant.	8	$\frac{24 \times 1024^3 \times 8}{8} \approx 25.77 \times 10^9$	ca. 25,77 Mrd.
4-bit Quantisierung	4	$\frac{24 \times 1024^3 \times 8}{4} \approx 51.55 \times 10^9$	ca. 51,55 Mrd.
2-bit Quantisierung	2	$\frac{24 \times 1024^3 \times 8}{2} \approx 103.11 \times 10^9$	ca. 103,11 Mrd.
1-bit Quantisierung	1	$\frac{24 \times 1024^3 \times 8}{1} \approx 206.22 \times 10^9$	ca. 206,22 Mrd.

Tab. 10: Maximale Anzahl der Parameter bei 24 GB VRAM für verschiedene Präzisionsformate

Kriterium	Ausprägung / Bewertung	Besondere Merkmale / Stärken
Funktionalität	Autocomplete (Fill-in-the-Middle), KI-Chat, Explain, Bugfix, Refactor, Dokumentation, Codegenerierung, API-Zugang, Self-Hosting, Agent-Funktion	Autonomer KI-Agent für mehrstufige Aufgaben, 25+ Programmiersprachen, API und Bring-Your-Own-Key (BYOK)
IDE-Kompatibilität	Plugins für VS Code, JetBrains, Visual Studio (Beta), Neovim (Beta), Sublime (Beta); erkennt Projektstruktur, AST, Imports, Cursorposition	Kontextverständnis durch Vektor-Datenbank und AST-Analyse, FIM-Completion
Datenschutz	Drei Privacy-Level, Self-Hosting möglich, DSGVO-konformer Betrieb, Zugriffsbeschränkungen und Datei-Exklusion konfigurierbar	Kein Code-Training auf Nutzerdaten, volle Kontrolle über Datenfluss und Modellwahl
UX / DX	Direkte IDE-Integration, intuitive Bedienung, schnelle Vorschläge, nicht störend, anpassbare Modelle, Explain- und Refactor-Funktionen	Hoher Lerneffekt durch erklärbare Vorschläge, Workflow-nahe Interaktion
Erweiterbarkeit	Open-Source (BSD), modulare Architektur, Fine-Tuning auf eigene Codebasen, modellagnostisch (GPT, Claude, StarCoder, Code Llama), API und CLI	Custom Models möglich, eigene Plugins und Tool-Integrationen erweiterbar
Integration	GitHub, GitLab, Docker, PostgreSQL, MySQL, Chrome, PDB, Shell-Kommandos	Automatisierter Build, Test, Deployment, End-to-End-Testing, Agent nutzt externe Tools
Skalierbarkeit	Multi-GPU und vLLM-Backend, Self-Hosting skalierbar (horizontal, mehrere Instanzen), hohe Tokenlimits, Vektorindexierung für große Codebasen	Geeignet für Einzelentwickler bis große Teams, Lastverteilung über Serverinstanzen
Performance	Schnelle Autocomplete (Millisekunden), Chat-Antworten im Sekundenbereich, Caching, Retrieval-Augmented Generation, effizientes Kontextmanagement	32k-64k Token Kontextfenster, reaktionsschnell auch bei großen Projekten
Zusätzliche Highlights	Autonomer KI-Agent, Fine-Tuning auf Unternehmenscode, Open-Source-Basis, granularer Datenschutz, flexible Modellwahl, RAG mit Vektor-Datenbank	Innovativer Agent-Ansatz, projektweites Kontextverständnis, hohe Anpassbarkeit

Tab. 11: Feature-Matrix Refact.AI (Ist-Analyse)

Kriterium	Ausprägung / Bewertung	Besondere Merkmale / Stärken
Funktionalität	Chat, Autocomplete, Edit, Agent; Codegenerierung, Erklärungen, Refactorings (auch mehrfile); keine externe API; modellunabhängig (LLM wählbar)	Autonomer Agent; flexible Modellwahl (lokal oder extern); BYOL (Bring Your Own LLM)
IDE-Kompatibilität	Plugins für VS Code und JetBrains IDEs; erkennt Cursor, Projektstruktur und Imports; semantische Code-Indexierung (Embeddings)	Kontextverständnis über Vektorindex; @-Referenzen und Codebase-Suche für gezielten Kontextzugriff
Datenschutz	Lokale Verarbeitung; kein Code-Versand an Hersteller; anonyme, abschaltbare Telemetrie; DSGVO-konform bei Self-Hosting	On-Prem-Proxy für Schlüsselverwaltung; volle Datenhoheit
UX / DX	Direkt in IDE integriert (Sidebar, Inline-Suggestions); Steuerung per Shortcuts; Kontextzufuhr über Markierung oder @-Referenzen	Streaming-Ausgabe, Apply-Buttons; anpassbare Modelle und Prompts; geringe Workflow-Störung
Erweiterbarkeit	Modular (YAML); eigene Modelle, Prompts, Regeln, Provider und Slash-Kommandos definierbar; Open Source (Apache 2.0)	Community-Hub für Assistants und Bausteine; flexibel erweiterbar, auch für interne Systeme
Integration	Kontext-Provider für GitLab Issues, Confluence, Doku, Repos; Anbindung an interne Tools (z.B. CI/CD) über Custom-Provider	Flexibel für interne Wissensbasen und Ticket-Systeme adaptierbar
Skalierbarkeit	Verteilte Nutzung (lokale Plugin-Instanzen); Skalierung über Backend-Modelle (lokal/Cluster); große Codebasen durch Embedding-basierte Suche handhabbar	Enterprise-Optionen für zentrale Config und Key-Management
Performance	Schnelle Kompletion (modellabhängig); Chat und Edit gestreamt; asynchrone Berechnung; schnelle semantische Kontextbereitstellung	Caching via Code-Index; Teilakzeptanz beschleunigt Interaktion
Zusätzliche Highlights	Agent mit Tool-Nutzung (Dateisystem, Suche, Kontext); Multi-LLM-Support (verschiedene Modelle für verschiedene Aufgaben); semantische Code-Suche	Open Source, BYOL-Strategie, Community-Hub für KI-Bausteine

Tab. 12: Feature-Matrix Continue.dev (On-Premise-Nutzung)

Plattform	Self-Hosting?	Modellwahl frei?	Open Source?	IDE-Integration?	Fazit
Refact.AI	Teilweise	Eingeschränkt	Hybrid	Ja	Modellbindung, Vendor-Lock-in
Amazon CodeWhisperer	Nein	Nein	Nein	Ja	Ausschluss: Cloud-only
GitHub Copilot (nicht Enterprise)	Nein	Nein	Nein	Ja	Ausschluss: Cloud-only
GitHub Copilot Enterprise	Teilweise	Teilweise	Nein	Ja	Keine echte Kontrolle, teuer
Codeium / Windsurf	Nein	Nein	Nein	Ja	Ausschluss: Cloud, kein BYO LLM
Tabnine (Enterprise)	Ja	Nein	Nein	Ja	Kein eigenes Modell möglich
Sourcegraph Cody	Nein	Nein	Nein	Ja	Cloud-only, Chat statt Inline Completion
Replit Ghostwriter	Nein	Nein	Nein	Nur Replit IDE	Komplett inkompatibel
Zencoder	Nein	Nein	Nein	Ja	Kein Self-Hosting
Continue.dev	Ja	Ja	Ja	Ja	Erfüllt alle Anforderungen

Tab. 13: Bewertung der Plattformen anhand definierter Ausschlusskriterien

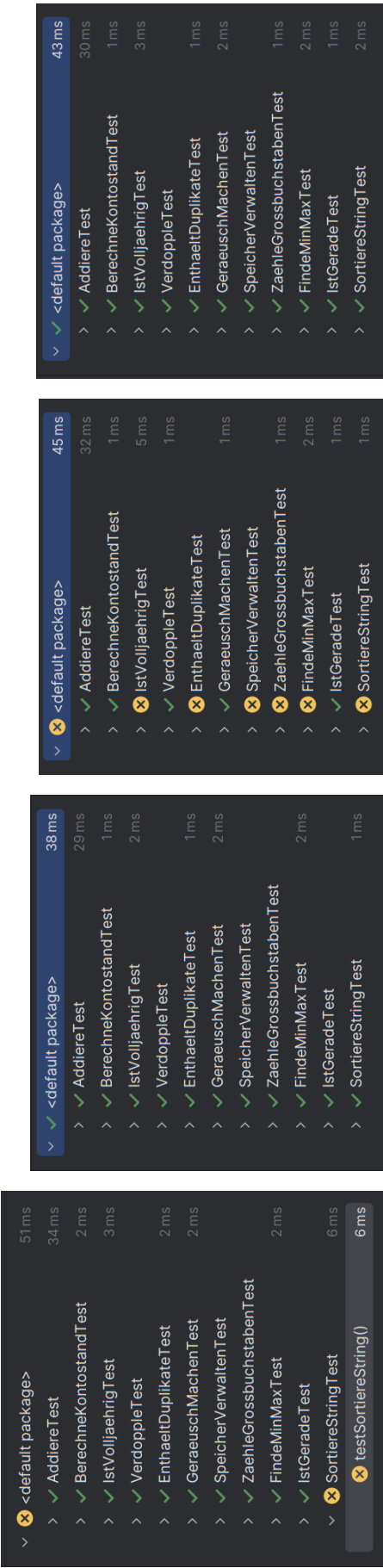


Abb. 2: Vergleich der Java Testergebnisse mehrerer Modelle (Bildschirmaufnahmen)



Abb. 3: Vergleich der Python Testergebnisse mehrerer Modelle (Bildschirmaufnahmen)

(d) Codestral 22B

Abb. 4: Vergleich der TypeScript Testergebnisse mehrerer Modelle (Bildschirmaufnahmen)

Literaturverzeichnis

Hinweis: Bei der Erstellung dieser Arbeit wurde die offizielle L^AT_EX-Vorlage der DHBW verwendet. Aufgrund einer technischen Einschränkung war es trotz umfangreicher Recherche und Anpassungsversuche nicht möglich, unterstrichene Titel wie booktitle oder journaltitle korrekt mit automatischem Zeilenumbruch darzustellen. Da die Zitierrichtlinien jedoch die Unterstreichung dieser Felder fordern, führten lange Titel stellenweise zu einem Überlaufen über den rechten Seitenrand. Um die Lesbarkeit zu gewährleisten, wurde das Format in solchen Fällen angepasst. Ich bitte um Verständnis, da eine standardkonforme Umsetzung technisch nicht möglich war.

- Abdellatif, Ahmad; Badran, Khaled; Costa, Diego Elias; Shihab, Emad (2022):** A Comparison of Natural Language Understanding Platforms for Chatbots in Software Engineering. In: *IEEE Transactions on Software Engineering* 48.8, S. 3087–3102. ISSN: 0098-5589, 1939-3520, 2326-3881. DOI: 10.1109/TSE.2021.3078384. URL: <https://ieeexplore.ieee.org/document/9426404/> (Abruf: 23.04.2025).
- Abraham, Ajith (2005):** „Rule-Based Expert Systems“. In: *Handbook of Measuring System Design*. Hrsg. von Peter H. Sydenham; Richard Thorn. 1. Aufl. Chichester, England: John Wiley & Sons, Ltd. ISBN: 978-0-470-02143-9 978-0-471-49739-4. DOI: 10.1002/0471497398.mm422. URL: <https://onlinelibrary.wiley.com/doi/10.1002/0471497398.mm422> (Abruf: 19.04.2025).
- Acharya, Deepak Bhaskar; Kuppan, Karthigeyan; Divya, B. (2025):** Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey. In: *IEEE Access* 13, S. 18912–18936. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3532853. URL: <https://ieeexplore.ieee.org/abstract/document/10849561> (Abruf: 19.04.2025).
- Al-Amin, Md; Ali, Mohammad Shazed; Salam, Abdus; Khan, Arif; Ali, Ashraf; Ullah, Ahsan; Alam, Md Nur; Chowdhury, Shamsul Kabir (2024):** History of generative Artificial Intelligence (AI) chatbots: past, present, and future development. DOI: 10.48550/arXiv.2402.05122. arXiv: 2402.05122[cs]. URL: <http://arxiv.org/abs/2402.05122> (Abruf: 15.04.2025).
- Annepaka, Yadagiri; Pakray, Partha (2025):** Large language models: a survey of their development, capabilities, and applications. In: *Knowledge and Information Systems* 67.3, S. 2967–3022. ISSN: 0219-1377, 0219-3116. DOI: 10.1007/s10115-024-02310-4. URL: <https://link.springer.com/10.1007/s10115-024-02310-4> (Abruf: 17.04.2025).
- Anthropic (2025):** Claude 3.7 Sonnet. URL: <https://www.anthropic.com/claude/sonnet> (Abruf: 24.04.2025).
- Aslam, Farhan (2023):** The Impact of Artificial Intelligence on Chatbot Technology: A Study on the Current Advancements and Leading Innovations. In: *European Journal of Technology* 7.2, S. 62–72. ISSN: 2520-0712. DOI: 10.47672/ejt.1561. URL: https://www.researchgate.net/publication/373138851_The_Impact_of_Artificial_Intelligence_on_Chatbot_Technology_A_Study_on_the_Current_Advancements_and_Leading_Innovations (Abruf: 13.04.2025).

- Bahdanau, Dzmitry; Cho, Kyunghyun; Bengio, Yoshua (2014):** Neural Machine Translation by Jointly Learning to Align and Translate. In: *CoRR*. ICLR 2015, S. 1–15. DOI: 10.48550/arXiv.1409.0473. arXiv: 1409.0473[cs]. URL: <http://arxiv.org/abs/1409.0473> (Abruf: 16.04.2025).
- Bangalore, S.; Johnston, M. (2003):** Balancing data-driven and rule-based approaches in the context of a multimodal conversational system. In: *2003 IEEE Workshop on Automatic Speech Recognition and Understanding (IEEE Cat. No.03EX721)*. 2003 IEEE Workshop on Automatic Speech Recognition and Understanding. St Thomas, VI, USA: IEEE, S. 221–226. ISBN: 978-0-7803-7980-0. DOI: 10.1109/ASRU.2003.1318444. URL: <http://ieeexplore.ieee.org/document/1318444/> (Abruf: 21.04.2025).
- Barker, Virginia E.; O'Connor, Dennis E.; Bachant, Judith; Soloway, Elliot (1989):** Expert systems for configuration at Digital: XCON and beyond. In: *Communications of the ACM* 32.3, S. 298–318. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/62065.62067. URL: <https://dl.acm.org/doi/10.1145/62065.62067> (Abruf: 13.04.2025).
- Bettis, Wyatt (2024):** AI News Values: Axiologies Guiding Content Production in Google Gemini, Microsoft Copilot, and Newspapers. Diss. Johns Hopkins University. URL: <https://jscholarship.library.jhu.edu/handle/1774.2/70344> (Abruf: 12.04.2025).
- Blinn, Andrew; Li, Xiang; Kim, June Hyung; Omar, Cyrus (2024):** Statically Contextualizing Large Language Models with Typed Holes. In: *Proceedings of the ACM on Programming Languages* 8 (OOPSLA2), S. 468–498. ISSN: 2475-1421. DOI: 10.1145/3689728. URL: <https://dl.acm.org/doi/10.1145/3689728> (Abruf: 23.04.2025).
- Buchanan, Bruce G.; Smith, Reid G. (1988):** Fundamentals of Expert Systems. In: *Annual Review of Computer Science* 3 (Volume 3, 1988). Publisher: Annual Reviews, S. 23–58. ISSN: 8756-7016. DOI: 10.1146/annurev.cs.03.060188.000323. URL: <https://www.annualreviews.org/content/journals/10.1146/annurev.cs.03.060188.000323> (Abruf: 19.04.2025).
- Canbek, Nil Goksel; Mutlu, Mehmet Emin (2016):** On the track of Artificial Intelligence: Learning with Intelligent Personal Assistants. In: *Journal of Human Sciences* 13.1, S. 592–601. ISSN: 2458-9489. URL: <https://www.j-humansciences.com/ojs/index.php/IJHS/article/view/3549> (Abruf: 21.04.2025).
- Casheekar, Avyay; Lahiri, Archit; Rath, Kanishk; Prabhakar, Kaushik Sanjay; Srinivasan, Kathiravan (2024):** A contemporary review on chatbots, AI-powered virtual conversational agents, ChatGPT: Applications, open challenges and future research directions. In: *Computer Science Review* 52.100632, S. 24. ISSN: 1574-0137. DOI: 10.1016/j.cosrev.2024.100632. URL: <https://www.sciencedirect.com/science/article/pii/S1574013724000169> (Abruf: 11.04.2025).
- Celia Dolores Benitez; Montes Serrano (2024):** The Integration and Impact of Artificial Intelligence in Software Engineering. In: *International Journal of Advanced Research in Science, Communication and Technology*, S. 791–805. ISSN: 2581-9429. DOI: 10.48175/IJARSCT-19190. URL: <http://ijarsct.co.in/Paper19190.pdf> (Abruf: 19.04.2025).

- Choi, Wan Chong; Chang, Chi In (2025):** Advantages and Limitations of Open-Source Versus Commercial Large Language Models (LLMs): A Comparative Study of DeepSeek and OpenAI's ChatGPT. DOI: 10.20944/preprints202503.1081.v1. URL: <https://www.preprints.org/manuscript/202503.1081/v1> (Abruf: 18.04.2025).
- Contreras, Albert; Guerra, Esther; De Lara, Juan (2024):** Conversational Assistants for Software Development: Integration, Traceability and Coordination: In: *Proceedings of the 19th International Conference on Evaluation of Novel Approaches to Software Engineering*. 19th International Conference on Evaluation of Novel Approaches to Software Engineering. Angers, France: SCITEPRESS - Science und Technology Publications, S. 27–38. ISBN: 978-989-758-696-5. DOI: 10.5220/0012561600003687. URL: <https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0012561600003687> (Abruf: 12.04.2025).
- Daniel Ajiga; Patrick Azuka Okeleke; Samuel Olaoluwa Folorunsho; Chinedu Ezeigweneme (2024):** Enhancing software development practices with AI insights in high-tech companies. In: *Computer Science & IT Research Journal* 5.8, S. 1897–1919. ISSN: 2709-0051, 2709-0043. DOI: 10.51594/csitrj.v5i8.1450. URL: <https://fepbl.com/index.php/csitrj/article/view/1450> (Abruf: 22.04.2025).
- Danowitz, Andrew; Kelley, Kyle; Mao, James; Stevenson, John P.; Horowitz, Mark (2012):** CPU DB: recording microprocessor history. In: *Communications of the ACM* 55.4, S. 55–63. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/2133806.2133822. URL: <https://dl.acm.org/doi/10.1145/2133806.2133822> (Abruf: 20.04.2025).
- Davis, Andrew (2016):** Conditional Computation in Deep and Recurrent Neural Networks. Diss. Knoxville: University of Tennessee. URL: https://trace.tennessee.edu/utk_graddiss/3907.
- DeepSeek-AI; Zhu, Qihao; Guo, Daya; Shao, Zhihong; Yang, Dejian; Wang, Peiyi; Xu, Runxin; Wu, Y.; Li, Yukun; Gao, Huazuo; Ma, Shirong; Zeng, Wangding; Bi, Xiao; Gu, Zihui; Xu, Hanwei; Dai, Damai; Dong, Kai; Zhang, Liyue; Piao, Yishi; Gou, Zhibin; Xie, Zhenda; Hao, Zhewen; Wang, Bingxuan; Song, Junxiao; Chen, Deli; Xie, Xin; Guan, Kang; You, Yuxiang; Liu, Aixin; Du, Qiushi; Gao, Wenjun; Lu, Xuan; Chen, Qinyu; Wang, Yaohui; Deng, Chengqi; Li, Jiashi; Zhao, Chenggang; Ruan, Chong; Luo, Fuli; Liang, Wenfeng (2024):** DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. DOI: 10.48550/arXiv.2406.11931. arXiv: 2406.11931[cs]. URL: <http://arxiv.org/abs/2406.11931> (Abruf: 26.04.2025).
- Deryugina, O. V. (2010):** Chatterbots. In: *Scientific and Technical Information Processing* 37.2, S. 143–147. ISSN: 1934-8118. DOI: 10.3103/S0147688210020097. URL: <https://doi.org/10.3103/S0147688210020097> (Abruf: 14.04.2025).
- Desolda, Giuseppe; Esposito, Andrea; Greco, Francesco; Tucci, Cesare; Buono, Paolo; Piccinno, Antonio (2025):** Understanding User Mental Models in AI-Driven Code Completion Tools: Insights from an Elicitation Study. Version Number: 1. DOI: 10.48550/ARXIV.2502.02194. URL: <https://arxiv.org/abs/2502.02194> (Abruf: 17.04.2025).

- Dewey, Marco (2024)**: Large Language Models and Software Testing. Diss. Matieland: Stellenbosch University. URL: <https://scholar.sun.ac.za/server/api/core/bitstreams/ed3a0d90-749c-420d-9d25-a8909f8ad192/content> (Abruf: 23.04.2025).
- European Data Protection Board (EDPB) (2020)**: Recommendations 01/2020 on measures that supplement transfer tools to ensure compliance with the EU level of protection of personal data. Brüssel: European Data Protection Board (EDPB), S. 1–38. URL: https://www.edpb.europa.eu/sites/default/files/consultation/edpb_recommendations_202001_supplementarymeasurestransferstools_en.pdf (Abruf: 26.04.2025).
- Farrow, Elissa (2019)**: To augment human capacity—Artificial intelligence evolution through causal layered analysis. In: *Futures* 108, S. 61–71. ISSN: 0016-3287. DOI: 10.1016/j.futures.2019.02.022. URL: <https://www.sciencedirect.com/science/article/pii/S001632871830332X> (Abruf: 19.04.2025).
- For Good AI Inc. (2025)**: Features | Zencoder – The AI Coding Agent. Zencoder. URL: <https://zencoder.ai/product/features> (Abruf: 19.04.2025).
- Gao, Ze (2023)**: A review on statistical language and neural network based code completion. In: *Applied and Computational Engineering* 22.1, S. 233–239. ISSN: 2755-2721, 2755-273X. DOI: 10.54254/2755-2721/22/20231222. URL: <https://www.ewadirect.com/proceedings/ace/article/view/4827> (Abruf: 17.04.2025).
- Geng, Jiahui (2023)**: Taking Computation to Data: Integrating Privacy-preserving AI techniques and Blockchain Allowing Secure Analysis of Sensitive Data on Premise. Accepted: 2023-08-29T13:54:57Z ISBN: 9788284391779 ISSN: 1890-1387. Doctoral thesis. Stavanger: University of Stavanger, Norway. URL: <https://uis.brage.unit.no/uis-xmlui/handle/11250/3086242> (Abruf: 18.04.2025).
- Gerichtshof der Europäischen Union (2020)**: Der Gerichtshof erklärt den Beschluss 2016/1250 über die Angemessenheit des vom EU-US-Datenschutzschild gebotenen Schutzes für ungültig. Pressemitteilung des Gerichtshofs der Europäischen Union. Luxemburg: Pressemitteilung des Gerichtshofs der Europäischen Union. URL: <https://curia.europa.eu/jcms/upload/docs/application/pdf/2020-07/cp200091de.pdf>.
- Google DeepMind (2025)**: Gemini 2.5 Pro. Google DeepMind. URL: <https://deepmind.google/technologies/gemini/pro/> (Abruf: 24.04.2025).
- Guo, Song-Bin; Meng, Yuan; Lin, Liteng; Zhou, Zhen-Zhong; Li, Hai-Long; Tian, Xiao-Peng; Huang, Wei-Juan (2024)**: Artificial intelligence alphafold model for molecular biology and drug discovery: a machine-learning-driven informatics investigation. In: *Molecular Cancer* 23.1, S. 223. ISSN: 1476-4598. DOI: 10.1186/s12943-024-02140-6. URL: <https://doi.org/10.1186/s12943-024-02140-6> (Abruf: 12.04.2025).
- Hamilton, James (2009)**: Cooperative Expendable Micro-Slice Servers (CEMS) : Low Cost, Low Power Servers For Internet-Scale Services. In: URL: <https://www.scribd.com/document/61467287/10-1-1-145> (Abruf: 20.04.2025).
- Hassan, Mohammad Asif (2024)**: Impact of adopting AI tools by software developers towards productivity and sustainability. Accepted: 2024-12-10T06:05:19Z. Diss. Lappeenranta-Lahti

- University of Technology. URL: <https://lutpub.lut.fi/handle/10024/168640> (Abruf: 23.04.2025).
- Hevner, Alan R.; March, Salvatore T.; Park, Jinsoo; Ram, Sudha (2004):** Design Science in Information Systems Research. In: *MIS Quarterly* 28.1. Publisher: Management Information Systems Research Center, University of Minnesota, S. 75–105. ISSN: 0276-7783. DOI: 10.2307/25148625. URL: <https://www.jstor.org/stable/25148625> (Abruf: 05.04.2025).
- Hinton, Geoffrey; Deng, Li; Yu, Dong; Dahl, George; Mohamed, Abdel-rahman; Jaitly, Navdeep; Senior, Andrew; Vanhoucke, Vincent; Nguyen, Patrick; Sainath, Tara; Kingsbury, Brian (2012):** Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups. In: *IEEE Signal Processing Magazine* 29.6, S. 82–97. ISSN: 1053-5888. DOI: 10.1109/MSP.2012.2205597. URL: <http://ieeexplore.ieee.org/document/6296526/> (Abruf: 21.04.2025).
- Hirschberg, Julia; Manning, Christopher D. (2015):** Advances in natural language processing. In: *Science* 349.6245, S. 261–266. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.aaa8685. URL: <https://www.science.org/doi/10.1126/science.aaa8685> (Abruf: 21.04.2025).
- Horlboge, Micha; Quiring, Erwin; Meyer, Roland; Rieck, Konrad (2022):** I still know it's you! On Challenges in Anonymizing Source Code. Version Number: 2. DOI: 10.48550/ARXIV.2208.12553. URL: <https://arxiv.org/abs/2208.12553> (Abruf: 26.04.2025).
- Hubert; Dreyfus, Stuart (1986):** Why Expert Systems Do Not Exhibit Expertise. In: *IEEE Expert* 1.2, S. 86–90. ISSN: 0885-9000, 2374-9407. DOI: 10.1109/MEX.1986.4306957. URL: <https://ieeexplore.ieee.org/document/4306957/> (Abruf: 19.04.2025).
- Isidor and the VS Code team (2025):** Introducing GitHub Copilot agent mode (preview). Visual Studio Code. URL: <https://code.visualstudio.com/blogs/2025/02/24/introducing-copilot-agent-mode> (Abruf: 19.04.2025).
- Jampala, Rahul; Kola, Devisri Santosh; Gummadi, Adithya Nagendra; Bhavanam, Meghana; Rani Pannerselvam, Ithaya (2024):** The Evolution of Voice Assistants: From Text-to-Speech to Conversational AI. In: *2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*. 2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT). Bengaluru, India: IEEE, S. 1332–1338. ISBN: 979-8-3503-2753-3. DOI: 10.1109/IDCIoT59759.2024.10467739. URL: <https://ieeexplore.ieee.org/document/10467739/> (Abruf: 21.04.2025).
- Jordan, M. I.; Mitchell, T. M. (2015):** Machine learning: Trends, perspectives, and prospects. In: *Science* 349.6245, S. 255–260. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.aaa8415. URL: <https://www.science.org/doi/10.1126/science.aaa8415> (Abruf: 14.04.2025).
- Kantek, Petr (2023):** AI-driven Software Development Source Code Quality. Diplomová práce. Masaryk University, Faculty of Informatics. URL: https://theses.cz/id/9oaezy/?lang=en#panel_bibtex (Abruf: 23.04.2025).

- Keshamoni, Kumar (2023):** „ChatGPT: An Advanced Natural Language Processing System for Conversational AI Applications—A Comprehensive Review and Comparative Analysis with Other Chatbots and NLP Models“. In: *ICT Systems and Sustainability*. Hrsg. von Milan Tuba; Shyam Akashe; Amit Joshi. Bd. 765. Series Title: Lecture Notes in Networks and Systems. Singapore: Springer Nature Singapore, S. 447–455. ISBN: 978-981-99-5651-7 978-981-99-5652-4. DOI: 10.1007/978-981-99-5652-4_40. URL: https://link.springer.com/10.1007/978-981-99-5652-4_40 (Abruf: 21.04.2025).
- Al-Kharusi, Younis; Khan, Ajmal; Rizwan, Muhammad; Bait-Suwailam, Mohammed M. (2024):** Open-Source Artificial Intelligence Privacy and Security: A Review. In: *Computers* 13.12. Number: 12 Publisher: Multidisciplinary Digital Publishing Institute, S. 311. ISSN: 2073-431X. DOI: 10.3390/computers13120311. URL: <https://www.mdpi.com/2073-431X/13/12/311> (Abruf: 13.04.2025).
- Lajko, Mark; Csuvik, Viktor; Gyimothy, Tibor; Vidacs, Laszlo (2024):** Automated Program Repair with the GPT Family, including GPT-2, GPT-3 and CodeX. In: *Proceedings of the 5th ACM/IEEE International Workshop on Automated Program Repair*. APR '24. New York, NY, USA: Association for Computing Machinery, S. 34–41. ISBN: 979-8-4007-0577-9. DOI: 10.1145/3643788.3648021. URL: <https://dl.acm.org/doi/10.1145/3643788.3648021> (Abruf: 16.04.2025).
- Levitan, Peter (2015):** ActiveBuddy & SmarterChild & Advertising. Peter Levitan & Co. URL: <https://peterlevitan.com/activebuddy-smarterchild-advertising-7485/> (Abruf: 15.04.2025).
- Li, Yichen; Peng, Yun; Huo, Yintong; Lyu, Michael R. (2024):** Enhancing LLM-Based Coding Tools through Native Integration of IDE-Derived Static Context. In: *Proceedings of the 1st International Workshop on Large Language Models for Code*. LLM4Code '24: 1st International Workshop on Large Language Models for Code. Lisbon Portugal: ACM, S. 70–74. ISBN: 979-8-4007-0579-3. DOI: 10.1145/3643795.3648392. URL: <https://dl.acm.org/doi/10.1145/3643795.3648392> (Abruf: 27.04.2025).
- Li, Youjia; Shi, Jianjun; Zhang, Zheng (2024):** An Approach for Rapid Source Code Development Based on ChatGPT and Prompt Engineering. In: *IEEE Access* 12, S. 53074–53087. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3385682. URL: <https://ieeexplore.ieee.org/document/10493021/> (Abruf: 23.04.2025).
- Li, Yujia; Choi, David; Chung, Junyoung; Kushman, Nate; Schrittwieser, Julian; Leblond, Rémi; Eccles, Tom; Keeling, James; Gimeno, Felix; Lago, Agustin Dal; Hubert, Thomas; Choy, Peter; d’Autume, Cyprien de Masson; Babuschkin, Igor; Chen, Xinyun; Huang, Po-Sen; Welbl, Johannes; Goyal, Sven; Cherepanov, Alexey; Molloy, James; Mankowitz, Daniel J.; Robson, Esme Sutherland; Kohli, Pushmeet; de Freitas, Nando; Kavukcuoglu, Koray; Vinyals, Oriol (2022):** Competition-Level Code Generation with AlphaCode. In: Publisher: arXiv Version Number: 1. DOI: 10.48550/ARXIV.2203.07814. URL: <https://arxiv.org/abs/2203.07814> (Abruf: 17.04.2025).
- Liang, Ming; Xie, Xiaoheng; Zhang, Gehao; Zheng, Xunjin; Di, Peng; jiang wei, wei; Chen, Hongwei; Wang, Chengpeng; Fan, Gang (2024):** REPOFUSE: Repository-Level

- Code Completion with Fused Dual Context. Version Number: 2. DOI: 10.48550/ARXIV.2402.14323. URL: <https://arxiv.org/abs/2402.14323> (Abruf: 24.04.2025).
- Liao, Simona (2025)**: Introducing Code Referencing for GitHub Copilot Completions in Visual Studio. Visual Studio Blog. URL: <https://devblogs.microsoft.com/visualstudio/introducing-code-referencing-for-github-copilot-completions-in-visual-studio/> (Abruf: 27.04.2025).
- Lomshakov, Vadim; Kovalchuk, Sergey; Omelchenko, Maxim; Nikolenko, Sergey; Aliev, Artem (2023)**: „Fine-Tuning Large Language Models for Answering Programming Questions with Code Snippets“. In: *Computational Science – ICCS 2023*. Hrsg. von Jiří Mikyška; Clélia De Mulatier; Maciej Paszynski; Valeria V. Krzhizhanovskaya; Jack J. Dongarra; Peter M.A. Sloot. Bd. 14074. Series Title: Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, S. 171–179. ISBN: 978-3-031-36020-6 978-3-031-36021-3. DOI: 10.1007/978-3-031-36021-3_15. URL: https://link.springer.com/10.1007/978-3-031-36021-3_15 (Abruf: 23.04.2025).
- Maletic, J.I.; Marcus, A. (2001)**: Supporting program comprehension using semantic and structural information. In: *Proceedings of the 23rd International Conference on Software Engineering. ICSE 2001*. 23rd International Conference on Software Engineering. ICSE 2001. Toronto, Ont., Canada: IEEE Comput. Soc, S. 103–112. ISBN: 978-0-7695-1050-7. DOI: 10.1109/ICSE.2001.919085. URL: <http://ieeexplore.ieee.org/document/919085/> (Abruf: 22.04.2025).
- Melo, Glaucia; Law, Edith; Alencar, Paulo; Cowan, Don (2020)**: Exploring Context-Aware Conversational Agents in Software Development. Version Number: 1. DOI: 10.48550/ARXIV.2006.02370. URL: <https://arxiv.org/abs/2006.02370> (Abruf: 21.04.2025).
- Meta (2025)**: The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. Meta AI. URL: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/> (Abruf: 24.04.2025).
- Mikolov, Tomas; Chen, Kai; Corrado, Greg; Dean, Jeffrey (2025)**: Efficient Estimation of Word Representations in Vector Space. In: ISSN: 1301.3781. URL: <https://arxiv.org/abs/1301.3781> (Abruf: 16.04.2025).
- Moraes, Rodrigo; Valiati, João Francisco; Gavião Neto, Wilson P. (2013)**: Document-level sentiment classification: An empirical comparison between SVM and ANN. In: *Expert Systems with Applications* 40.2, S. 621–633. ISSN: 09574174. DOI: 10.1016/j.eswa.2012.07.059. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0957417412009153> (Abruf: 21.04.2025).
- Muggleton, Stephen (2024)**: Alan Turing and the development of Artificial Intelligence | Request PDF. In: 27.1, S. 3–10. ISSN: 0921-7126. DOI: 0.3233/AIC-130579. URL: https://www.researchgate.net/publication/262290526_Alan_Turing_and_the_development_of_Artificial_Intelligence (Abruf: 13.04.2025).
- Naseem, Usman; Razzak, Imran; Khan, Shah Khalid; Prasad, Mukesh (2021)**: A Comprehensive Survey on Word Representation Models: From Classical to State-of-the-Art Word Representation Language Models. In: *ACM Transactions on Asian and Low-Resource*

- Language Information Processing* 20.5, S. 1–35. ISSN: 2375-4699, 2375-4702. DOI: 10.1145/3434237. URL: <https://dl.acm.org/doi/10.1145/3434237> (Abruf: 16.04.2025).
- Natale, Simone (2019)**: If software is narrative: Joseph Weizenbaum, artificial intelligence and the biographies of ELIZA. In: *New Media & Society* 21.3, S. 712–728. ISSN: 1461-4448, 1461-7315. DOI: 10.1177/1461444818804980. URL: <https://journals.sagepub.com/doi/10.1177/1461444818804980> (Abruf: 13.04.2025).
- Nettur, Suresh Babu; Karpurapu, Shanthi; Nettur, Unnati; Gajja, Likhith Sagar; Myneni, Sravanthy; Dusi, Akhil (2025)**: The Role of GitHub Copilot on Software Development: A Perspective on Productivity, Security, Best Practices and Future Directions. Version Number: 1. DOI: 10.48550/ARXIV.2502.13199. URL: <https://arxiv.org/abs/2502.13199> (Abruf: 27.04.2025).
- Nickolls, John; Buck, Ian; Garland, Michael; Skadron, Kevin (2008)**: Scalable parallel programming with CUDA. In: *ACM SIGGRAPH 2008 classes*. SIGGRAPH '08. New York, NY, USA: Association for Computing Machinery, S. 1–14. ISBN: 978-1-4503-7845-1. DOI: 10.1145/1401132.1401152. URL: <https://dl.acm.org/doi/10.1145/1401132.1401152> (Abruf: 14.04.2025).
- OpenAI (2025)**: Introducing GPT-4.5. URL: <https://openai.com/index/introducing-gpt-4-5/> (Abruf: 24.04.2025).
- Oyekunle Claudius Oyeniran; Adebunmi Okechukwu Adewusi; Adams Gbolahan Adeleke; Lucy Anthony Akwawa; Chidimma Francisca Azubuko (2022)**: Ethical AI: Addressing bias in machine learning models and software applications. In: *Computer Science & IT Research Journal* 3.3, S. 115–126. ISSN: 2709-0051, 2709-0043. DOI: 10.51594/csitrj.v3i3.1559. URL: <https://fepbl.com/index.php/csitrj/article/view/1559> (Abruf: 19.04.2025).
- Patwardhan, Narendra; Marrone, Stefano; Sansone, Carlo (2023)**: Transformers in the Real World: A Survey on NLP Applications. In: *Information* 14.4, S. 242. ISSN: 2078-2489. DOI: 10.3390/info14040242. URL: <https://www.mdpi.com/2078-2489/14/4/242> (Abruf: 21.04.2025).
- Pettersson, Mats (2025)**: Developer Productivity : Exploring factors that affect developer productivity and how they impact team performance and prioritization. Diss. Karlstad: Karlstad University. URL: <https://urn.kb.se/resolve?urn=urn:nbn:se:kau:diva-102790> (Abruf: 27.04.2025).
- Pinto, Gustavo; De Souza, Cleidson; Rocha, Thayssa; Steinmacher, Igor; Souza, Alberto; Monteiro, Edward (2024)**: Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes. In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI*. CAIN '24. New York, NY, USA: Association for Computing Machinery, S. 81–91. ISBN: 979-8-4007-0591-5. DOI: 10.1145/3644815.3644949. URL: <https://doi.org/10.1145/3644815.3644949> (Abruf: 16.04.2025).

- Podduturi, Santhosh (2025):** AI-Driven Code Optimization: Leveraging ML to Refactor Legacy Codebases. In: *North American Journal of Engineering Research* 6.1. URL: <https://najer.org/najer/article/view/115>.
- Puspitaningrum, Diyah (2021):** A Study of English-Indonesian Neural Machine Translation with Attention (Seq2Seq, ConvSeq2Seq, RNN, and MHA): A Comparative Study of NMT on English-Indonesian. In: *6th International Conference on Sustainable Information Engineering and Technology 2021*. SIET '21: 6th International Conference on Sustainable Information Engineering and Technology 2021. Malang Indonesia: ACM, S. 271–280. ISBN: 978-1-4503-8407-0. DOI: 10.1145/3479645.3479703. URL: <https://dl.acm.org/doi/10.1145/3479645.3479703> (Abruf: 21.04.2025).
- Qazi, Farhan A. (2022):** Insecure Application Programming Interfaces (APIs) in Zero-Trust Networks. AAI28966153 ISBN-13: 9798790650468. Diss. Capitol Technology University.
- Qwen (2025):** Qwen2.5-Max: Exploring the Intelligence of Large-scale MoE Model. Qwen. Section: blog. URL: <https://qwenlm.github.io/blog/qwen2.5-max/> (Abruf: 24.04.2025).
- Rautiainen, Olli (2023):** GitHub Enterprise and Migration of CI/CD Pipelines from Azure DevOps to GitHub. Diss. Kuopio: Savonia University of Applied Sciences. URL: <http://www.theseus.fi/handle/10024/795837> (Abruf: 19.04.2025).
- Reuel, Anka; Hardy, Amelia; Smith, Chandler; Lamparth, Max; Hardy, Malcolm; Kochenderfer, Mykel J. (2024):** BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices. In: *Advances in Neural Information Processing Systems*. Hrsg. von A. Globerson; L. Mackey; D. Belgrave; A. Fan; U. Paquet; J. Tomczak; C. Zhang. Bd. 37. Curran Associates, Inc., S. 21763–21813. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/26889e8359e7ef8a7f5d77457364ca55-Paper-Datasets_and_Benchmarks_Track.pdf.
- Savelka, Jaromir; Agarwal, Arav; Bogart, Christopher; Sakr, Majd (2023):** Large Language Models (GPT) Struggle to Answer Multiple-Choice Questions About Code: In: *Proceedings of the 15th International Conference on Computer Supported Education*. 15th International Conference on Computer Supported Education. Prague, Czech Republic: SCITEPRESS - Science und Technology Publications, S. 47–58. ISBN: 978-989-758-641-5. DOI: 10.5220/0011996900003470. URL: <https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0011996900003470> (Abruf: 16.04.2025).
- Schaaf, Nina; Wiedenroth, Saskia Johanna; Wagner, Philipp (2021):** Erklärbare KI in der Praxis. Anwendungsorientierte Evaluation von xAI-Verfahren. Stuttgart: Fraunhofer-Institut für Produktionstechnik und Automatisierung IPA. URL: <http://publica.fraunhofer.de/dokumente/N-630667.html> (Abruf: 13.04.2025).
- Schmidt, Rainer; Alt, Rainer; Zimmermann, Alfred (2021):** A Conceptual Model for Assistant Platforms. In: Hawaii International Conference on System Sciences. DOI: 10.24251/HICSS.2021.490. URL: <http://hdl.handle.net/10125/71107> (Abruf: 21.04.2025).
- Sergeyuk, Agnia; Titov, Sergey; Izadi, Maliheh (2024):** In-IDE Human-AI Experience in the Era of Large Language Models; A Literature Review. In: *Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments*. IDE '24. New

- York, NY, USA: Association for Computing Machinery, S. 95–100. ISBN: 979-8-4007-0580-9. DOI: 10.1145/3643796.3648463. URL: <https://dl.acm.org/doi/10.1145/3643796.3648463> (Abruf: 12.04.2025).
- Sheggam, Harshith; Zhang, Xiaowen (2024):** Exploring Security Risks and Mitigation Strategies in AI Code Helpers. In: *2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT). Holtsville, NY, USA: IEEE, S. 1–6. ISBN: 979-8-3315-0666-7. DOI: 10.1109/LISAT63094.2024.10807934. URL: <https://ieeexplore.ieee.org/document/10807934/> (Abruf: 26.04.2025).
- Silberling, Amanda (2023):** Twenty years ago, AIM chatbot SmarterChild out-snarked ChatGPT. TechCrunch. URL: <https://techcrunch.com/2023/07/26/twenty-years-ago-aim-chatbot-smarterchild-out-snarked-chatgpt/> (Abruf: 15.04.2025).
- Singh, Jagdish; Joesph, Minnu Helen; Jabbar, Khurshid Begum Abdul (2019):** Rule-based chabot for student enquiries. In: *Journal of Physics: Conference Series* 1228.1, S. 012060. ISSN: 1742-6588, 1742-6596. DOI: 10.1088/1742-6596/1228/1/012060. URL: <https://iopscience.iop.org/article/10.1088/1742-6596/1228/1/012060> (Abruf: 21.04.2025).
- Smutny, Pavel; Bojko, Michal (2024):** Comparative Analysis of Chatbots Using Large Language Models for Web Development Tasks. In: *Applied Sciences* 14.21, S. 10048. ISSN: 2076-3417. DOI: 10.3390/app142110048. URL: <https://www.mdpi.com/2076-3417/14/21/10048> (Abruf: 23.04.2025).
- Solanke, Adedamola Abiodun (2024):** Software Development in the Age of AI: Quantifiable Frameworks for Measuring Enhanced Developer Productivity. In: *ResearchGate* Volume 14 (Issue 4), S. 234–250. ISSN: 2250-1770. URL: https://www.researchgate.net/publication/390585493_Software_Development_in_the_Age_of_AI_Quantifiable_Frameworks_for_Measuring_Enhanced_Developer_Productivity (Abruf: 12.04.2025).
- Souza, Ana Carolina Moises De; Reinehr, Sheila; Malucelli, Andreia; Conte, Tayan; Marczak, Sabrina; Jaccheri, Letizia; Binder, Fabio; Lauschner, Tanara; Cruzes, Daniela (2024):** IT Talent Shortage: Strategies to Mitigate a Blackout. In: *Anais do XXXVIII Simpósio Brasileiro de Engenharia de Software (SBES 2024)*. Simpósio Brasileiro de Engenharia de Software. Brasil: Sociedade Brasileira de Computação, S. 499–509. DOI: 10.5753/sbes.2024.3533. URL: <https://sol.sbc.org.br/index.php/sbes/article/view/30389> (Abruf: 23.04.2025).
- Stalnaker, Trevor; Wintersgill, Nathan; Chaparro, Oscar; Heymann, Laura A.; Penta, Massimiliano Di; German, Daniel M.; Poshyvanyk, Denys (2025):** Developer Perspectives on Licensing and Copyright Issues Arising from Generative AI for Software Development. DOI: 10.48550/arXiv.2411.10877. arXiv: 2411.10877[cs]. URL: <http://arxiv.org/abs/2411.10877> (Abruf: 19.04.2025).
- Sultan, Shairaz (2022):** Smart chatbot. Accepted: 2022-09-15T11:42:58Z. Master thesis. Oslo: Oslo Metropolitan University. URL: <https://oda.oslomet.no/oda-xmlui/handle/11250/3018051> (Abruf: 15.04.2025).

- Sutter, Herb; Larus, James (2005):** Software and the Concurrency Revolution: Leveraging the full power of multicore processors demands new tools and new thinking from the software industry. In: *Queue* 3.7, S. 54–62. ISSN: 1542-7730, 1542-7749. DOI: 10.1145/1095408.1095421. URL: <https://dl.acm.org/doi/10.1145/1095408.1095421> (Abruf: 20.04.2025).
- Szolderits, Chiara Marita (2025):** A New Era of Coding: Investigating GitHub Copilot's Influence on Productivity, Code Quality and Ethics in Software Development. Diss. Klagenfurt: Alpen-Adria-Universität Klagenfurt. URL: <https://netlibrary.aau.at/obvuklhs/content/titleinfo/11700723/full.pdf> (Abruf: 23.04.2025).
- tabnine (2025):** Personalized AI code generator from Tabnine AI code assistant. Tabnine. URL: <https://www.tabnine.com/personalization/> (Abruf: 19.04.2025).
- The Stanford Natural Language Processing Group (2025):** Stanford Log-linear Part-Of-Speech Tagger. Stanford Log-linear Part-Of-Speech Tagger. URL: <https://nlp.stanford.edu/software/tagger.shtml> (Abruf: 10.04.2025).
- Tipirneni, Sindhu; Zhu, Ming; Reddy, Chandan K. (2022):** StructCoder: Structure-Aware Transformer for Code Generation. In: *ACM Trans. Knowl. Discov. Data* 18.3, 70:1–70:20. ISSN: 1556-4681. DOI: 10.1145/3636430. URL: <https://dl.acm.org/doi/10.1145/3636430> (Abruf: 16.04.2025).
- Treude, Christoph; Gerosa, Marco A. (2025):** How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering. DOI: 10.48550/arXiv.2501.08774. arXiv: 2501.08774[cs]. URL: <http://arxiv.org/abs/2501.08774> (Abruf: 12.04.2025).
- Tsikora, Alona (2023):** A FEASIBILITY STUDY OF AVAILABLE NATURAL LANGUAGE CHATBOT TECHNOLOGIES. Diss. Vaasa: VAASAN AMMATTIKORKEAKOULU UNIVERSITY OF APPLIED SCIENCES. 63 S. URL: <https://www.theseus.fi/handle/10024/802337> (Abruf: 15.04.2025).
- Turing, Alan M. (1950):** COMPUTING MACHINERY AND INTELLIGENCE. In: *Mind* LIX.236, S. 433–460. ISSN: 1460-2113, 0026-4423. DOI: 10.1093/mind/LIX.236.433. URL: <https://academic.oup.com/mind/article/LIX/236/433/986238> (Abruf: 19.04.2025).
- Vake, Domen; Šinik, Bogdan; Vičič, Jernej; Tošić, Aleksandar (2025):** Is Open Source the Future of AI? A Data-Driven Approach. In: *Applied Sciences* 15.5. Number: 5 Publisher: Multidisciplinary Digital Publishing Institute, S. 2790. ISSN: 2076-3417. DOI: 10.3390/app15052790. URL: <https://www.mdpi.com/2076-3417/15/5/2790> (Abruf: 13.04.2025).
- Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N.; Kaiser, Łukasz; Polosukhin, Illia (2017):** Attention is all you need. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. Bd. 30. NIPS'17. Red Hook, NY, USA: Curran Associates Inc. ISBN: 978-1-5108-6096-4. DOI: 10.48550/arXiv.1706.03762. (Abruf: 16.04.2025).
- Wallace, Richard S. (2009):** „The Anatomy of A.L.I.C.E.“ In: *Parsing the Turing Test: Philosophical and Methodological Issues in the Quest for the Thinking Computer*. Hrsg. von Robert Epstein; Gary Roberts; Grace Beber. Dordrecht: Springer Netherlands, S. 181–210. ISBN: 978-1-4020-6710-5. DOI: 10.1007/978-1-4020-6710-5_13. URL: https://doi.org/10.1007/978-1-4020-6710-5_13 (Abruf: 14.04.2025).

- Wetstein, Suzanne (2017):** Designing a Dutch financial chatbot. Diss. Amsterdam: VU University Amsterdam. 111 S. URL: <https://vu-business-analytics.github.io/internship-office/reports/report-wetstein.pdf> (Abruf: 15.04.2025).
- Wu, Xingjiao; Xiao, Luwei; Sun, Yixuan; Zhang, Junhang; Ma, Tianlong; He, Liang (2022):** A survey of human-in-the-loop for machine learning. In: *Future Generation Computer Systems* 135, S. 364–381. ISSN: 0167-739X. DOI: 10.1016/j.future.2022.05.014. URL: <https://www.sciencedirect.com/science/article/pii/S0167739X22001790> (Abruf: 19.04.2025).
- Xiao, Tao; Hata, Hideaki; Treude, Christoph; Matsumoto, Kenichi (2024):** Generative AI for Pull Request Descriptions: Adoption, Impact, and Developer Interventions. In: *Proceedings of the ACM on Software Engineering* 1 (FSE), S. 1043–1065. ISSN: 2994-970X. DOI: 10.1145/3643773. URL: <https://dl.acm.org/doi/10.1145/3643773> (Abruf: 17.04.2025).
- Xu, Frank F.; Vasilescu, Bogdan; Neubig, Graham (2022):** In-IDE Code Generation from Natural Language: Promise and Challenges. In: *ACM Trans. Softw. Eng. Methodol.* 31.2, 29:1–29:47. ISSN: 1049-331X. DOI: 10.1145/3487569. URL: <https://dl.acm.org/doi/10.1145/3487569> (Abruf: 12.04.2025).
- Ylitalo, Aleksi (2024):** AI assisted public cloud management. Accepted: 2024-04-05T06:50:23Z. Diss. Oulu: Oulu University of Applied Sciences. 59 S. URL: <http://www.theseus.fi/handle/10024/850830> (Abruf: 26.04.2025).
- Yu, Tao; Yang, Wei; Xu, Junping; Pan, Younghwan (2024):** Barriers to Industry Adoption of AI Video Generation Tools: A Study Based on the Perspectives of Video Production Professionals in China. In: *Applied Sciences* 14.13, S. 5770. ISSN: 2076-3417. DOI: 10.3390/app14135770. URL: <https://www.mdpi.com/2076-3417/14/13/5770> (Abruf: 17.04.2025).
- Zhang, Mingxuan; Yuan, Bo; Li, Hanzhe; Xu, Kangming (2024):** LLM-Cloud Complete: Leveraging Cloud Computing for Efficient Large Language Model-based Code Completion. In: *Journal of Artificial Intelligence General science (JAIGS) ISSN:3006-4023* 5.1, S. 295–326. ISSN: 3006-4023. DOI: 10.60087/jaigs.v5i1.200. URL: <https://ojs.boulibrary.com/index.php/JAIGS/article/view/200> (Abruf: 17.04.2025).
- Zhong, Tianyang; Liu, Zhengliang; Pan, Yi; Zhang, Yutong; Zhou, Yifan; Liang, Shizhe; Wu, Zihao; Lyu, Yanjun; Shu, Peng; Yu, Xiaowei; Cao, Chao; Jiang, Hanqi; Chen, Hanxu; Li, Yiwei; Chen, Junhao; Hu, Huawen; Liu, Yihen; Zhao, Huaqin; Xu, Shaochen; Dai, Haixing; Zhao, Lin; Zhang, Ruidong; Zhao, Wei; Yang, Zhenyuan; Chen, Jingyuan; Wang, Peilong; Ruan, Wei; Wang, Hui; Zhao, Huan; Zhang, Jing; Ren, Yiming; Qin, Shihuan; Chen, Tong; Li, Jiayi; Zidan, Arif Hassan; Jahin, Afrar; Chen, Minheng; Xia, Sichen; Holmes, Jason; Zhuang, Yan; Wang, Jiaqi; Xu, Bochen; Xia, Weiran; Yu, Jichao; Tang, Kaibo; Yang, Yaxuan; Sun, Bolun; Yang, Tao; Lu, Guoyu; Wang, Xianqiao; Chai, Lilong; Li, He; Lu, Jin; Sun, Lichao; Zhang, Xin; Ge, Bao; Hu, Xintao; Zhang, Lian; Zhou, Hua; Zhang, Lu; Zhang, Shu; Liu, Ninghao; Jiang, Bei; Kong, Linglong; Xiang, Zhen; Ren, Yu-**

dan; Liu, Jun; Jiang, Xi; Bao, Yu; Zhang, Wei; Li, Xiang; Li, Gang; Liu, Wei; Shen, Dinggang; Sikora, Andrea; Zhai, Xiaoming; Zhu, Dajiang; Liu, Tianming (2024): Evaluation of OpenAI o1: Opportunities and Challenges of AGI. Version Number: 1. DOI: 10.48550/ARXIV.2409.18486. URL: <https://arxiv.org/abs/2409.18486> (Abruf: 16.04.2025).

Erklärung zur Verwendung generativer KI-Systeme

Bei der Erstellung der eingereichten Arbeit habe ich auf künstlicher Intelligenz (KI) basierte Systeme benutzt:

☒ ja

☐ nein¹⁶⁶

Falls ja: Die nachfolgend aufgeführten auf künstlicher Intelligenz (KI) basierten Systeme habe ich bei der Erstellung der eingereichten Arbeit benutzt:

1. ChatGPT (Modelle GPT-4o und GPT-4.5)
2. ScholarAI (als Custom GPT in ChatGPT)
3. GitHub Copilot

Ich erkläre, dass ich

- › mich aktiv über die Leistungsfähigkeit und Beschränkungen der oben genannten KI-Systeme informiert habe,¹⁶⁷
- › die aus den oben angegebenen KI-Systemen direkt oder sinngemäß übernommenen Passagen gekennzeichnet habe,
- › überprüft habe, dass die mithilfe der oben genannten KI-Systeme generierten und von mir übernommenen Inhalte faktisch richtig sind,
- › mir bewusst bin, dass ich als Autorin bzw. Autor dieser Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.

Die oben genannten KI-Systeme habe ich wie im Folgenden dargestellt eingesetzt:

¹⁶⁶Die Erklärung ist in jedem Fall zu unterzeichnen, auch wenn Sie keine KI-Systeme genutzt haben und Ihr Kreuz bei „nein“ gesetzt haben.

¹⁶⁷U.a. gilt es hierbei zu beachten, dass an KI weitergegebene Inhalte ggf. als Trainingsdaten genutzt und wiederverwendet werden. Dies ist insb. für betriebliche Aspekte als kritisch einzustufen.

Arbeitsschritt in der wissenschaftlichen Arbeit	Eingesetzte(s) KI-System(e)	Beschreibung der Verwendungsweise
Ideenfindung, Gliederung, Formulierungen	ChatGPT	Unterstützung bei der Generierung von Ideen, Ausarbeitung von Argumentationsstrukturen, sprachliche Glättung und Prüfung von Stil, Grammatik und Rechtschreibung. Wiederholte Prüfung auf faktische Richtigkeit und inhaltliche Konsistenz.
Literatursuche	ScholarAI (via ChatGPT)	Generierung semantisch passender Quellen auf Basis eingegebener Freitexte zur Effizienzsteigerung bei der thematischen Recherche.
Texterstellung in $\text{\LaTeX}$, Codeergänzungen	GitHub Copilot	Unterstützung bei der $\text{\LaTeX}$ -Syntax, Vorschläge für Formatierungen und automatisierte Vervollständigung von Codefragmenten (insbesondere bei technischen Passagen und Quelltextbeispielen).

(Ort, Datum)

(Unterschrift)

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: Code Completion unter eigener Kontrolle: Self-Hosted Open-Source-Alternativen für datensensible Unternehmen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

(Ort, Datum)

(Unterschrift)